

密级：公开

北京邮电大学

博士学位论文



题目： 云平台通用组件逻辑漏洞挖掘与分析

关键技术研究

学 号： 2020010220

姓 名： 王子奇

学科专业： 网络空间安全

培养方式： 全日制

导 师： 崔宝江

学 院： 网络空间安全学院

2024 年 05 月 30 日

中国·北京

密级：公开

北京邮电大学

博士学位论文（学术学位）



题目： 云平台通用组件逻辑漏洞挖掘与分析
关键技术研究

学 号： 2020010220

姓 名： 王 子 奇

学科专业： 网络空间安全

培养方式： 全 日 制

导 师： 崔 宝 江

学 院： 网络空间安全学院

2024 年 5 月 30 日

Beijing University of Posts and
Telecommunications

Thesis for Doctor (Academic Degree)



TITLE: Research on Key Technologies of
Detecting and Analyzing Logic Vulnerabilities
in General Components of Cloud Platforms

Student ID: 2020010220

Candidate: Ziqi Wang

Subject: Cyberspace Security

Training Mode: Full-time

Supervisor: Baojiang Cui

Institute: School of Cyberspace Security

May 30, 2024

答辩委员会名单

职务	姓 名	职 称	工 作 单 位
主席	刘文	教授	中国传媒大学
委员	黎琳	教授	北京交通大学
委员	孙运传	教授级高工	北京师范大学
委员	崔宝江	教授	北京邮电大学
委员	罗守山	教授	北京邮电大学
秘书	袁雪敬	讲师	北京邮电大学
答辩日期	2024 年 5 月 27 日		

云平台通用组件逻辑漏洞挖掘与分析关键技术研究

摘 要

随着云计算等新一代信息技术的快速发展,软件开发正经历深刻变革,程序从单体应用向大型分布式系统演进,具有复杂成因本质与外在表现的代码及业务逻辑漏洞更加占据主导地位,并通过供应链波及大量下游云平台相关软组件。然而,现有漏洞挖掘方法大多针对特定编程语言特性进行设计,同时重点关注基础语法语义信息,缺乏对程序及业务语言无关型逻辑链特征的深层次理解,与逻辑漏洞产生机理之间存在矛盾,制约了逻辑漏洞的挖掘与分析水平。

因此,现有逻辑漏洞挖掘与分析技术主要面临如下困境:1)逻辑漏洞的建模与分析能力不足,即无法支撑复杂逻辑漏洞的建模与分析;2)逻辑漏洞的挖掘与利用能力不足,即无法高效地挖掘与利用逻辑漏洞。本文针对上述困境,从逻辑漏洞的分析与挖掘两方面开展研究,针对困境一,研究逻辑漏洞语言无关型特征的分析方法;针对困境二,研究云平台通用组件代码层逻辑漏洞、业务层逻辑漏洞、应用层逻辑漏洞的挖掘方法。具体研究内容与研究成果包括:

1、逻辑漏洞语言无关型特征提取与表征技术

针对当前漏洞分析方法难以提取并表征逻辑漏洞的语言无关型成因本质等问题,本章节首先分析逻辑漏洞的语言无关型特征,分析代码及业务逻辑漏洞的代码层成因,探索该类特征的逻辑本质;进一步从纵向代码层级架构及横向编程语句性质两个视角实现语言无关型特征提取,并利用图数据库进行表征,以支持快速的漏洞复现分析与漏洞模型构建。实验证明该技术能够对逻辑漏洞的语言无关特征进行提取与表征,支持各类逻辑漏洞快速复现验证及分析,包括:代码逻辑型漏洞,如拒绝服务型算法逻辑漏洞;业务逻辑型漏洞,如条件竞争型逻辑漏洞等。本部分研究内容可有效支撑逻辑漏洞的复现分析,支撑后续逻辑漏洞挖掘模型的建立,最终支撑挖掘多项 CVE 漏洞。

2、基于静态程序逻辑理解的代码层逻辑漏洞挖掘

针对当前云平台代码层通用组件逻辑漏洞挖掘方法难以从漏洞成因本质入手挖掘漏洞、随机性强、效果差等问题,首先基于前文所提出的逻辑漏洞语言无关型特征的提取与表征方法,总结代码层算法逻辑漏洞的发生模式;进一步以部分典型类型漏洞为例,提出新型基

于程序逻辑结构及安全策略违背的逻辑漏洞挖掘模型；最后，结合具体的编程语言，进行漏洞语言无关特征模型的实例化，对发生在特定编程语言中的特定类型代码逻辑漏洞进行挖掘。实验证明本文所提出的漏洞挖掘方法能够更高效、更准确地挖掘代码层算法逻辑漏洞，实验发现未公开逻辑漏洞数十个，并申请获得 8 个 CVE 漏洞编号。

3、基于动态程序逻辑推演的业务层逻辑漏洞挖掘

针对当前云平台业务层通用组件逻辑漏洞挖掘方法测试深度不足、测试效果差等问题，本文首先基于可公开获得的业务逻辑描述信息，静态地分析业务层逻辑特性并进行测试组划分，每个测试组关联潜在的逻辑漏洞测试模型；随后分析业务层设计与实现不一致的逻辑缺陷，缓解动态测试深度不足等问题，提升测试的深度与覆盖率；最后构造实例化逻辑漏洞测试样本序列，实现对目标业务的逻辑漏洞测试，并基于状态机稀疏匹配算法检测逻辑漏洞。实验证明本文所提出的基于动态程序逻辑推演的漏洞挖掘方法能够深入测试程序业务逻辑，有效提升测试的覆盖深度与广度，并最终挖掘知名开源公有云平台未公开业务逻辑漏洞多项。

4、基于程序运行时逻辑特征的应用层逻辑漏洞挖掘

针对当前云平台应用层通用组件逻辑漏洞挖掘方法难以发现潜在数据背后的代码执行后门漏洞等问题，本文首先针对此类型应用的后门逻辑漏洞植入方法进行评估，分析当今应用层通用组件，尤其是机器学习预训练模型所面临的逻辑漏洞植入风险；进一步提出针对此类应用的动态执行特性的无侵入式获取方法，获取应用层组件部署时关键调用逻辑特征；进一步基于所获信息构建函数调用栈树形结构，并提出异常调用逻辑的检测方法。实验证明本文所提出的无侵入式执行信息获取方法能够更高效率的获取更多应用加载时关键逻辑特性，所提出的逻辑漏洞检测方法能够对应用层通用组件进行漏洞检测，并最终发现实网环境的十余项攻击。

关键词：云平台通用组件；逻辑特征；逻辑漏洞

RESEARCH ON KEY TECHNOLOGIES OF DETECTING AND ANALYZING LOGIC VULNERABILITIES IN GENERAL COMPONENTS OF CLOUD PLATFORM

ABSTRACT

With the rapid evolution of next-generation information technologies such as cloud computing, software development is undergoing profound transformations. Programs are evolving from single applications to large-scale distributed systems, where code and business logic vulnerabilities, which have complex intrinsic causes and external manifestations, are becoming increasingly dominant and impacting numerous downstream cloud platform-related software components through the supply chain. However, existing vulnerability detection methods are mostly designed based on specific programming language features and primarily focus on basic syntax and semantic information, lacking a deep understanding of language-independent features and logical chains of programs and business logic. This discrepancy with the occurrence principle of logic vulnerability hinders the effectiveness of logic vulnerability detection and analysis.

Thus, current logic vulnerability detection and analysis method are faced with the following challenges: (1) Insufficient modeling and analysis capabilities for logic vulnerabilities, making it difficult to support the modeling and analysis of complex logic vulnerabilities; (2) Insufficient detection and exploitation capabilities for logic vulnerabilities, making it inefficient to detect and exploit these vulnerabilities. This thesis addresses these challenges by focusing on both the analysis and detection of logic vulnerabilities. For the first challenge, we study the language-independent feature extraction and representation technology for logic vulnerabilities. For the second challenge, we explore the detection methods for code-layer, business-layer, and application-layer logic vulnerabilities in cloud platform

generic components. The specific research content and results include:

1. Language-Independent Feature Extraction and Representation Technology for Logic Vulnerabilities

This section addresses the difficulty of existing vulnerability analysis methods in extracting and representing the language-independent nature of logic vulnerabilities. We first analyze the language-independent features of these vulnerabilities, examine the code-level causes of code and business logic vulnerabilities, and explore the logical essence of these features. We then achieve language-independent feature extraction from vertical code hierarchical architecture and horizontal statement properties perspectives and use graph databases for representation to support rapid vulnerability reproduction analysis and model construction. Experiments demonstrate that this method can extract and represent language-independent features of logic vulnerabilities, supporting rapid reproduction and analysis of various logic vulnerabilities, including code logic vulnerabilities such as (DoS) denial of service algorithmic logic vulnerabilities and business logic vulnerabilities such as race condition logic vulnerabilities. This research effectively supports the reproduction analysis of logic vulnerabilities, the establishment of subsequent logic vulnerability detection models, and the detection of multiple CVE vulnerabilities.

2. Code-Layer Component Logic Vulnerability Detecting Based on Static Program Logic Understanding

To address the issues brought in by current cloud platform code-level generic component logic vulnerability detection methods, which often fail to explore vulnerabilities caused by their intrinsic ability of highly random and poor effectiveness, firstly we analyze and summarize the occurrence patterns of algorithmic logic vulnerabilities at the code level based on the our previously proposed method for extracting and representing language-independent features of logic vulnerabilities. We then propose a novel detection model for logic vulnerabilities based on program logical structure and security policy violations. Finally, we instantiate the detection models for specific programming languages to detect specific types of code logic vulnerabilities in these languages. Experiments show that the proposed detection method can more efficiently and accurately detect algorithmic

logic vulnerabilities at the code level, discovering dozens of undisclosed logic vulnerabilities and obtaining 8 CVE vulnerability numbers.

3. Business-Layer Component Logic Vulnerability Detecting Based on Dynamic Program Logic Inference

To address the problems of insufficient testing depth and poor testing effectiveness in current cloud platform business-level generic component logic vulnerability detection methods, this section first analyzes business logic features and divides test groups based on publicly available program logic description documentation, with each test group associated with a potential logic vulnerability testing model. Then we analyze logical defects caused by inconsistencies between business design and implementation, mitigating issues of insufficient testing depth and improving testing depth and coverage. Finally, we construct logic vulnerability test case sequences to test the target business logic vulnerabilities and use state machine sparse matching algorithm to detect logic vulnerabilities. Experiment results show that the proposed dynamic logic inference-based vulnerability detection method can deeply test business logic, effectively improving testing depth and coverage, and ultimately discovering multiple undisclosed business logic vulnerabilities in well-known open-source cloud platforms such as OpenStack.

4. Application-Layer Component Logic Vulnerability Detecting Based on Program Runtime Logic Characteristics

To make up for the deficiency of current logic vulnerability detection methods in detecting hidden code execution backdoor vulnerabilities in cloud platform application-layer general components, this section firstly evaluates the backdoor logic vulnerability implantation methods for such applications and analyzes the risks of these implantation methods faced by current cloud application-level generic components, especially pre-trained machine learning models. Further, we propose a non-intrusive method for obtaining dynamic execution characteristics of such applications, capturing key invocation logic characteristics during deployment. We then propose a method for constructing function call stack tree structure and a method for detecting abnormal invocation logic. Experiments show that the proposed non-intrusive execution information acquisition method can efficiently

capture more key logical characteristics during application loading, and the proposed detection method can detect vulnerabilities in application-level generic components effectively, ultimately discovering dozens of attacks in real network environments.

KEY WORDS: cloud platform general components; logic feature; logic vulnerability

目 录

第一章 绪论.....	1
1.1 选题背景与研究意义.....	1
1.2 国内外发展现状.....	3
1.2.1 传统漏洞与逻辑漏洞的定义及分类.....	3
1.2.2 漏洞挖掘所需信息的提取与表征技术	5
1.2.3 适用于云平台通用组件的传统漏洞与逻辑漏洞挖掘方法	8
1.3 论文的研究内容与创新点	11
1.4 论文的组织结构.....	13
第二章 相关技术	15
2.1 本文所关注的逻辑漏洞的定义及分类	15
2.2 云平台通用组件逻辑漏洞挖掘所需信息	17
2.2.1 基于图形化代码仓库表征程序静态逻辑信息	18
2.2.2 基于 REST API 规范文档分析文本蕴含的逻辑信息	19
2.2.3 基于代码插桩技术获取运行时调用栈逻辑信息	20
2.3 云平台通用组件的逻辑漏洞挖掘与检测方法	20
2.3.1 基于静态信息分析漏洞调用逻辑.....	20
2.3.2 基于静态信息指导动态逻辑测试样本生成	21
2.3.3 基于动态信息分析漏洞行为逻辑特征	21
2.4 本章小结.....	21
第三章 逻辑漏洞语言无关型特征提取与表征技术	23
3.1 问题描述.....	24
3.1.1 概念定义.....	24
3.1.2 设计动机.....	24
3.1.3 设计目标.....	25
3.2 面向程序纵向层级架构的语言无关型逻辑特征提取方法	26
3.2.1 从属关系的提取方法.....	26
3.2.2 调用关系的提取方法.....	27
3.3 面向程序横向执行语句的语言无关型逻辑特征提取方法	30
3.4 实验评估.....	31
3.4.1 对于逻辑漏洞的分析及复现能力.....	32
3.4.2 对于逻辑漏洞模型的总结与提取能力	34
3.4.3 与业内代表性工作的对比.....	37
3.5 本章小结.....	38
第四章 基于静态程序逻辑理解的代码层逻辑漏洞挖掘	39

4.1 问题描述.....	39
4.1.1 概念定义.....	40
4.1.2 威胁模型.....	41
4.1.3 总体框架.....	43
4.2 基于递归结构安全策略违背的算法逻辑漏洞模型构造	43
4.3 基于调用链及代码仓库搜索的算法逻辑漏洞分析方法	44
4.3.1 覆盖类方法递归的复杂递归结构分析方法	44
4.3.2 预期外的不安全递归终止条件分析方法	45
4.3.3 未受控的异常和错误处理分析方法.....	47
4.4 基于递归结构逻辑理解的漏洞利用载荷构造方法	48
4.5 实验评估.....	49
4.5.1 对于知名开源项目的逻辑漏洞挖掘能力	49
4.5.2 与业界先进研究的功能及性能对比.....	51
4.5.3 对于漏洞验证与利用的辅助能力.....	52
4.6 本章小结.....	54
第五章 基于动态程序逻辑推演的业务层逻辑漏洞挖掘	55
5.1 问题描述.....	56
5.1.1 概念定义.....	56
5.1.2 威胁模型.....	57
5.1.3 设计目标.....	58
5.1.4 总体框架.....	59
5.2 云平台通用组件 REST API 相关逻辑信息的获取与处理方法	60
5.2.1 基于文档信息的 API 归并与测试组生成方法	60
5.2.2 基于文档信息与 API 代码实现的不一致问题检测方法	64
5.2.3 基于装饰器插桩的 API 运行时逻辑信息获取方法	65
5.3 基于模型推演与参数推断增强的测试样本构造方法	67
5.3.1 基于资源生命周期的 IDOR 漏洞检测模型定义.....	67
5.3.2 基于模型推演与参数推断的测试样本构造方法	68
5.4 基于 API 调用栈辅助的测试样本执行结果分析方法	69
5.5 实验评估.....	71
5.5.1 代码与文档不一致风险的分析与检测	71
5.5.2 测试样本的测试覆盖率和有效性.....	73
5.5.3 对热门公有云平台业务组件的逻辑漏洞挖掘效果	75
5.6 本章小结.....	76
第六章 基于程序运行时逻辑特征的应用层逻辑漏洞挖掘	79
6.1 问题描述.....	80
6.1.1 概念定义.....	80
6.1.2 威胁模型.....	81

6.1.3 设计目标.....	81
6.2 机器学习模型应用组件后门逻辑漏洞的植入与利用方法	82
6.2.1 模型保存阶段的直接注入方法.....	82
6.2.2 模型保存阶段的隐写注入方法.....	83
6.3 基于运行时逻辑特征的机器学习模型后门逻辑漏洞检测方法	84
6.3.1 基于宿主机运行调试器的部署时运行信息追踪方法	85
6.3.2 基于运行时信息的函数调用逻辑特征归一表示方法	86
6.3.3 基于部署时运行信息的模型异常调用逻辑检测方法	87
6.4 实验评估.....	89
6.4.1 后门逻辑漏洞植入方式的效果评估.....	90
6.4.2 部署时运行信息追踪的效果及效率.....	92
6.4.3 对开源模型后门逻辑漏洞及实网攻击的检测结果	95
6.5 本章小结.....	97
第七章 总结与展望	99
7.1 工作总结.....	99
7.2 未来展望.....	101
参考文献.....	103

第一章 绪论

1.1 选题背景与研究意义

近年来，云计算^[1]等新一代信息技术飞速发展并成为国家级战略举措，美国、英国、欧盟等国家及政府部门分别发布一系列计划与准则，推进安全可信云计算平台的发展建设与部署应用^[2]。美国政府提出的“云优先^[3]”以及“云敏捷”战略，推进联邦政府等机构采用现代化云解决方案，以促进其在经济、军事、科技等方面的优势；欧盟于 2016 年首次提出“欧盟云”的概念^[4]，并于 2023 年批准一项价值 12 亿欧元的国家援助计划，以推进下一代云计算基础设施和服务建设^[5]；英国国防部于 2023 年发布《国防云战略路线图》^[6]，推进云计算在英国军事国防现代化转型中的应用；而我国也在“十四五”规划和 2035 年远景目标纲要中将云计算列为重点产业。与此同时，我国云计算产业市场发展迅速，如图 1-1 所示，预计 2025 年我国云计算整体市场规模将突破万亿元。

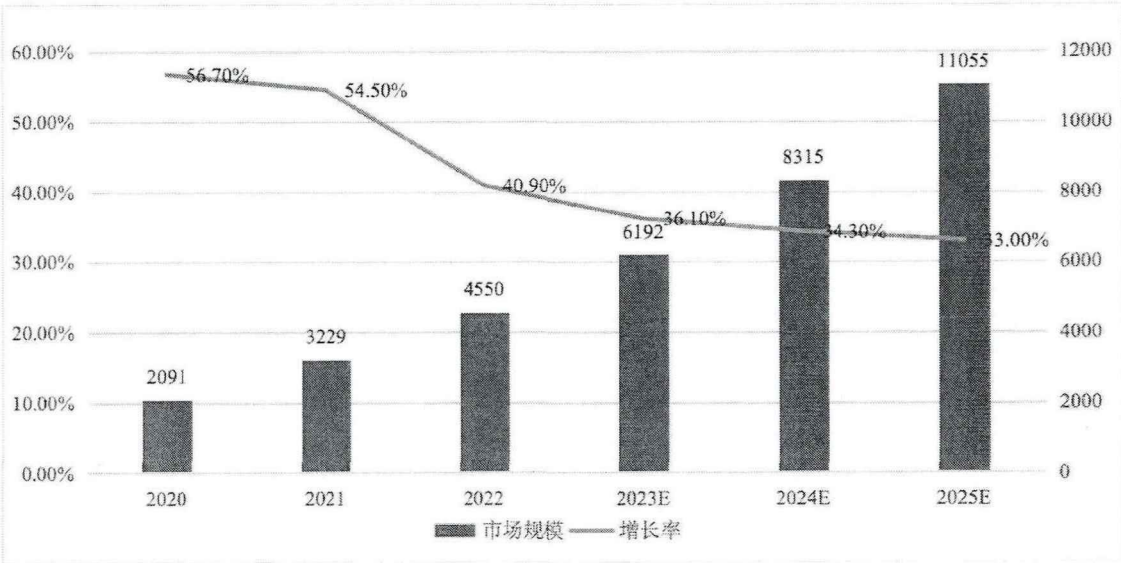


图 1-1 我国云计算整体市场规模及增长率

数据来源：中国信息通信研究院《云计算白皮书（2023 年）》^[2]。

随着云计算等新一代信息技术的发展，程序逐渐由单体应用向云计算平台等大型分布式应用转变，软件构建方式与开发方式也发生了巨大变化：一是主流编程语言由内存不安全的 C/C++ 逐步转变为 Java、Python 等内存安全的语言^[7]；二是程序的规模日益庞大，代码逻辑复杂度呈现指数级增长，单个开发人员已经无法掌握程序逻辑的全貌；三是软件供应链日趋复杂，上游软件出现的漏洞往往波及下游软件^[8]。这些变化客观上导致了逻辑漏洞的产生，并扩大了逻辑漏洞的影

响范围。逻辑漏洞作为一类典型的高威胁复杂性漏洞，近年来已成为一个新兴的研究热点。逻辑漏洞是一类由错误的程序或业务逻辑引起的安全缺陷，典型的逻辑漏洞包括算法逻辑漏洞、越权型逻辑漏洞、条件竞争型逻辑漏洞等，因为它们与系统的具体功能密切相关，所以长期以来一直被认为难以自动化检测^[9]。

云平台通用组件，即在云计算虚拟化层、应用层被广泛使用的软件组件，这些组件通常提供了标准化、可重用的功能，可被集成到各种云应用和服务中，也同样可被应用于构建传统软件。在新软件形态的背景下，大量开源软件供应链所提供的通用软件组件被开发者广泛引用，所产生的安全问题产生重大影响。2020年，开源云计算平台 OpenStack 的 keystone 组件发生身份鉴权逻辑漏洞，用户可伪造他人身份，通过 API (Application Programming Interface) 发送特定请求以冒充其他用户，甚至以 admin 权限执行命令^[10]；2021 年 log4j 核弹级反序列化代码执行逻辑漏洞，影响下游 35,863 个可用软件包，攻击者可以通过恶意的日志请求触发该漏洞，然后在受影响的应用程序中执行任意代码^[11]；2022 年，随着 ChatGPT 的发布^[12]，机器学习预训练模型的安全问题同样引起大众的安全关切，如 Hugging Face、TensorFlow Hub 等机器学习模型仓库所提供的预训练模型可能被植入恶意代码，在云平台的迁移部署过程中可造成针对云平台的远程命令执行^[13]。可以发现，云平台通用组件，包括代码层、业务层及应用层的通用支撑性组件、通用业务组件、通用功能性组件都被逻辑漏洞严重困扰^[14]。

然而，现今逻辑漏洞挖掘与分析技术主要面临以下两大困境，制约了云平台通用组件及云平台自身的安全性：其一，缺乏对复杂程序逻辑与逻辑漏洞的表征能力，现有的程序分析技术中，静态分析技术需要较高的开发与使用门槛，其效果受到开发与使用者水平的限制，对于复杂程序分析可能发生路径爆炸等问题，但对于逻辑特征的表征能力又稍显不足，而动态程序分析技术又存在开销大、覆盖率差等问题。其二，缺乏对逻辑漏洞的建模、分析与挖掘能力，现有的动态漏洞挖掘技术以模糊测试 (Fuzzing Test) 技术为代表，对于挖掘传统内存型漏洞的能力强，但难以发现复杂程序逻辑漏洞，而静态漏洞挖掘技术又受限于静态程序分析技术的发展水平，难以以较低的误报率和较高的检测率发现逻辑漏洞；同时这些方法缺乏对逻辑漏洞的分析能力，难以从庞杂的程序特征或测试结果中筛选出真正有问题的逻辑片段，因而难以高效的检测与分析漏洞。

因此，本文主要研究未来典型软件形态背景下，云平台通用组件的逻辑漏洞挖掘与分析关键技术，具体目标如下。针对困境一，研究：1) 逻辑漏洞语言无关型特征提取与表征技术，基于已有先验知识，对逻辑漏洞的与逻辑相关、与语言无关的特征进行提取与表征，支撑逻辑漏洞分析。针对困境二，研究：2) 基于静态程序逻辑理解的代码层逻辑漏洞挖掘技术，对云平台代码层通用组件进行

程序逻辑表征,提取与漏洞相关的关键逻辑要素,对程序进行静态漏洞检测;3)基于动态程序逻辑推演的业务层逻辑漏洞挖掘技术,针对云平台通用组件的业务层逻辑进行理解,从云平台通用组件的对外服务接口,如 REST (REpresentational State Transfer) API 接口入手,自动化进行针对云平台业务逻辑的动态测试,挖掘云平台业务层逻辑漏洞;4)基于程序运行时逻辑特征的应用层逻辑漏洞挖掘技术,提取云平台应用层通用组件的部署时特征,构造安全组件的部署时特征模型,提出程序运行时异常逻辑特征检测方法以挖掘逻辑漏洞。

通过研究以上内容,本文将达成以下有益效果:1)实现针对云平台通用组件相关逻辑及逻辑漏洞的深入理解分析,包括逻辑漏洞的分类及成因、逻辑漏洞的发生本质及外在表现,进一步支撑逻辑漏洞挖掘方法体系的设计,支撑逻辑漏洞的建模及挖掘;2)形成针对云平台通用组件的代码层逻辑漏洞挖掘方法,基于所提出的逻辑漏洞语言无关特征的提取与表征方法,实现代码层逻辑漏洞的特征提取与建模,提出代码层逻辑漏洞利用链挖掘模型;3)形成对云平台通用组件的业务层逻辑漏洞挖掘方法,基于对 API 接口逻辑的理解,构造与业务逻辑关联的测试样本,提升业务逻辑的测试深度,并获取测试样本所触发的业务层运行逻辑,实现业务层逻辑漏洞的检测与分析;4)形成对云平台通用组件的应用层逻辑漏洞挖掘方法,基于调试插桩技术,追踪通用应用的部署时调用逻辑,构造通用应用组件的逻辑漏洞特征检测模型,发现隐蔽嵌入的应用层逻辑漏洞。最终,本文实现针对云平台通用组件的逻辑漏洞挖掘与分析方法,所提出的方法适用于多种逻辑漏洞分析与挖掘场景,有效增强逻辑漏洞的挖掘与分析能力,提升云平台及云平台通用组件的安全水平。

1.2 国内外发展现状

漏洞挖掘相关技术的研究早已起步^[15, 16],但针对逻辑漏洞挖掘的相关研究仍未发展成完整的技术体系,甚至对于逻辑漏洞的定义仍未有明确且统一的描述。漏洞的挖掘与分析是一项复杂且流程化的工作,需要分析人员对于分析目标、目标面临的威胁类型、可能对应的漏洞种类、相应的漏洞分析与挖掘方法具有全面的掌握。这涉及多方面的先验知识,包括漏洞的定义及分类方法、漏洞挖掘所需信息的获取方法、基于所获信息进行漏洞分析的方法。因此本节将遵循漏洞挖掘的一般工作流程,从传统漏洞与逻辑漏洞的定义及分类、漏洞挖掘所需信息的提取与表征技术、漏洞挖掘方法三个方面对当前领域内的发展现状进行讨论。

1.2.1 传统漏洞与逻辑漏洞的定义及分类

漏洞的定义和分类不仅提供共同的语言框架,使得安全研究人员、开发人员等相关人员能够以标准化的方式讨论和处理安全问题,并且为漏洞的检测、分析

和修复工作提供了一种结构化的方法：通过精确的漏洞定义，能够帮助分析人员明确区分各种类型的安全威胁和漏洞，更有效地进行漏洞管理，针对性地制定检测策略和缓解措施；而对漏洞进行分类管理有助于制定针对性的检测、验证和防御措施，并促进安全社区之间的知识共享，促进安全最佳实践和经验的传播，实现安全威胁的预防或迅速响应^[17]。

1.2.1.1 国内外漏洞定义方式

现有逻辑漏洞的定义大多围绕业务逻辑进行。在本论文的开题阶段，笔者注意到现有对“逻辑漏洞”的定义往往局限于业务逻辑层面，尤其关注 Web 应用程序的安全问题^[18]，包括但不限于认证绕过、订单金额的任意修改、会话劫持、验证码绕过等。诚然，上述漏洞是逻辑型漏洞的典型代表，即其不是由传统的编程错误（如类型错误或语法错误）造成的，而是由于应用程序业务逻辑的设计或实现上存在逻辑缺陷而导致。然而，该定义方式仍然过于狭隘，未能涵盖逻辑漏洞的全貌。

近期，OWASP（Open Worldwide Application Security Project）组织针对业务逻辑漏洞的定义进行了进一步拓展，指出逻辑漏洞是一种“利用应用程序的合法处理流程以产生对目标有负面影响的方式”^[19]。换言之，业务逻辑漏洞是一种应用程序的设计或实现中的缺陷，攻击者可能利用该缺陷操纵合法功能来实现恶意目标。如前文所述，此类漏洞通常不是由于技术缺陷（如编程错误）所引起，而是源于业务流程设计或实现的逻辑错误。业务逻辑漏洞通常允许攻击者利用应用程序的正常功能以意想不到的方式实现恶意目的。

综上所述，逻辑漏洞的实质不仅仅体现在 Web 应用程序的安全问题上，而是涵盖了更广泛的范畴。逻辑漏洞可以出现在任何类型的软件系统中，包括但不限于桌面应用程序、移动应用程序、云应用等。它们可能与业务流程、数据处理逻辑、用户接口交互、系统配置管理等多个方面相关。此类漏洞发生的共通点在于它们通常源于软件系统整体逻辑的错误设计或实现，而非仅仅代码层面的局部编码缺陷。例如，一个财务系统可能由于复杂的业务规则处理不当，导致特定情况下用户能够绕过正常的审批流程；或者代码层算法逻辑问题导致的异常资源耗尽。上述情况虽然在表现形式上各不相同，但它们都源于系统逻辑的错误设计或错误实现，而不是传统意义上的代码漏洞。

1.2.1.2 国内外漏洞分类标准

网络安全漏洞分类国家标准，即《GB / T 30279 -2020 网络安全漏洞分类分级指南》^[20]。该标准主要基于漏洞的产生或触发原因对漏洞进行分类，将漏洞统一划分为代码问题、配置错误、环境问题及其他等。进一步地，代码问题包括资源管理错误、输入验证错误、数字错误、竞争条件问题、处理逻辑错误、加密问

题、授权问题数据转换问题、未声明功能等；而环境问题又包括信息泄漏和故障注入等。

国家信息安全漏洞库（China National Vulnerability Database of Information Security, CNNVD）漏洞分类分级标准^[21]。CNNVD 将信息安全漏洞划分为 26 种类型，包括配置错误、代码问题、资源管理错误、数字错误、信息泄漏、竞争条件等。国家信息安全漏洞共享平台（China National Vulnerability Database, CNVD）则根据漏洞的产生原因，将漏洞划分为 11 种类型，包括输入验证错误、访问验证错误、边界条件错误、缓冲区错误等。

通用缺陷枚举（Common Weakness Enumeration, CWE）标准^[22]。CWE 标准是由美国 MITRE 公司开发的一个描述在软件架构、设计以及编码等环节中存在的安全缺陷与漏洞的通用规范。2023 年 10 月 26 日发布的最新版 CWE 标准共包含 1421 个条目，其中包括缺陷与漏洞 934 项。同时该标准提供三个视图帮助理解，包括软件开发视图、硬件设计视图、研究概念视图。该分类标准目前被国内外众多机构广泛采用。

综上所述，已有漏洞分类标准大多基漏洞产生或触发的原因对漏洞进行分类，此分类方式可能存在的不足包括：部分分类标准如 CNNVD 和 CNVD 等分类粒度较粗，更多的是基于漏洞原因进行分类，而缺乏对漏洞外在表现的描述，使其更适合应用于漏洞数据库的标准化，而非指导漏洞的分析及挖掘；而 CWE 标准分类粒度则过细，部分漏洞分类条目映射为同一类型漏洞，对数据库的维护造成了一定阻碍。

1.2.2 漏洞挖掘所需信息的提取与表征技术

通常漏洞挖掘所需要提取的信息包括静态信息与动态信息两大类，对应静态分析与动态分析两种漏洞挖掘方法。静态信息通常涉及程序的代码、配置文件、文档等在程序运行之前就可以获取的数据。动态信息包括程序在运行时的行为特征，如内存使用情况、执行路径、I/O（输入/输出）操作等。进一步地，对于静态分析而言，静态分析提取的对象包括代码信息，如数据流、控制流等信息；包括文档信息，如文档所包含的 API 调用接口、自然语言描述的程序功能等。围绕上述信息，现有漏洞挖掘所需信息的提取与表征技术主要包括两大方向：其一是静态程序理解及分析技术，其二是动态程序执行信息获取及状态监视技术。

1.2.2.1 静态程序理解及分析技术

程序理解是一个从计算机程序中获取知识信息的过程^[23-25]。程序理解是软件工程领域的一个重要部分，通常被应用于软件维护、软件升级以及软件再工程。程序理解的工作一般包括：确定程序理解的目标、掌握目标程序的结构信息、分

析数据流与控制流信息等。程序理解的方法一般包括：基于文本信息的程序理解，以及基于代码信息的程序理解。

（1）基于文本信息的程序理解

文本信息常被用于描述程序的功能、变更历史、甚至代码生成。Shahbazi 等人^[26]利用神经网络模型，基于预定义的 API 文档进行代码注释生成，并引入排名机制以过滤不相关的 API；Godefroid 等人^[27]提出针对 REST API 变更情况的回归测试方法，从文档的角度识别 API 文档规范的变更，进一步指导对服务本身的回归测试；随着大模型时代到来，ChatGPT、CodeGeeX^[28]等工具根据用户自定义的功能描述实现自动化代码生成。

近些年，文本信息同样被用于进行漏洞挖掘。Chen 等人^[29]基于文本信息对在线支付系统进行逻辑漏洞预测，发现 Alipay、WeChat Pay 等在线支付系统的 5 个安全缺陷；Gorski 等人^[30]对 API 文档进行分析，以发现功能描述中的安全相关信息，探索应用系统可能存在的安全问题，并发现安全相关信息对内容安全策略实施的影响；Haryono 等人^[31]基于漏洞描述的文本信息进行未受指明的受影响库的识别，以解决软件成分分析过程中的依赖项更新缺失等问题。

综上所述，基于文本信息的理解促进了代码质量的维护以及用户对程序的理解，便捷了代码与文本信息的互转与生成，同时也对安全问题的检测与发现提供了有益帮助。但现有的相关研究大多局限于对文本中单一信息的分析，即分析单独 API 以生成代码注释、发现其历史变更或预测安全问题，而较少关注 API 之间及背后蕴含的复杂逻辑信息，进而无法发现复杂 API 关联的逻辑漏洞。

（2）基于代码信息的程序理解

基于代码信息的程序理解技术大多以程序分析技术^[32]为基础，对程序的结构信息、数据流控制流等信息进行理解。以各类 IDE（Integrated Development Environment）为代表^[33]，它们集成了各类工具及插件，支持开发者理解及操作代码；以及知名程序分析框架如 Soot^[34, 35]、Tai-e^[36]等，这些工具提供了基本的程序分析框架底座，支持开发者深入分析及优化代码。基于此类程序的静态信息，分析人员可追踪敏感的数据流及控制流信息，进而发现程序的安全漏洞。

此外，基于代码的程序理解还被用于进行程序切片、代码克隆检测、程序变异分析等。Weiser^[37]著作的经典论文介绍了程序切片技术，至今仍被广泛应用于程序的调试、维护及理解中；Tao 等人^[38]提出 C4 工具，基于 CodeBERT 模型^[39]，将不同编程语言转换为统一表示形式，进而检测跨语言的代码克隆，缓解大批量代码重用带给软件维护和演化的不利影响。

综上所述，目前基于代码信息的程序理解技术发展迅速，并被广泛地应用于代码优化及安全检测。然而这些技术在应用于逻辑漏洞挖掘时，仍然面临着一定

困境,即所提供的基础程序分析信息更偏向于代码底层,与编程语言特性紧耦合,难以提取与程序逻辑相关、与编程语言特性无关的程序逻辑特性,对逻辑漏洞挖掘缺少高维度视角支持,导致后续逻辑漏洞分析仍然需要大量经验性质操作;缺乏分析者友好型支持,需要分析人员对程序具有深入了解才能高效分析安全漏洞。

1.2.2.2 动态程序执行信息获取及状态监视技术

程序执行信息获取及状态监视,即从两个角度获取程序的运行时信息,一是从内在角度追踪程序运行时的语句执行及函数调用,以追踪程序运行时的内在逻辑,所依据的技术途径主要包括插桩技术(Code Instrumentation)等^[40];二是从外在角度监视程序运行时的状态,包括程序运行时的反馈输出、对于宿主机及操作系统的资源消耗情况。下面将从上述两个方向对研究现状进行介绍。

(1) 代码插桩技术

从插桩方式进行分类,代码插桩技术主要包括静态插桩技术^[41, 42]和动态插桩技术^[43, 44]:静态插桩技术即在源代码或编译后的代码中加入额外的监控或控制代码;动态插桩技术即在程序运行时进行插桩,通常通过特定的工具或库在运行时动态修改程序内存以实现插桩的目的。从插桩对象进行分类,代码插桩技术主要包括源代码插桩技术和二进制插桩技术等:源代码插桩技术直接在源代码中添加额外代码,用于实现信息记录、状态检查或执行特定检测;二进制插桩技术则在编译后的二进制代码中添加插桩代码,无需修改源代码。

(2) 状态监视技术

基于虚拟机的状态监视技术。随着云计算和虚拟化技术的快速发展,虚拟机监视在云计算领域变得尤为重要。虚拟机监视主要通过关注虚拟机的资源使用情况,包括CPU(Central Processing Unit)使用率、内存使用、虚拟磁盘I/O以及网络带宽使用等,实现对运行于虚拟机中的程序性能的评估,并及时发现可能存在的安全问题^[45, 46]。例如,通过分析虚拟机的资源使用变化情况,发现运行于其中的算法应用的资源消耗异常;通过对虚拟机内部程序运行时行为的监控,发现可能存在的代码执行等问题。

基于运行时全量日志信息的状态监视技术。此类监视方式主要通过收集、存储和分析系统运行时产生的日志信息以了解系统的运行状态^[47, 48]。日志信息通常包括系统日志、应用日志以及安全日志等,通过上述信息可以了解到系统或应用的工作情况,如资源使用情况、处理的请求以及可能的错误信息等。通过实时或定期对日志信息进行分析,可以及时发现系统可能存在的问题,并且可以根据日志信息进行故障定位,发现安全问题并缩短恢复服务的时间。在云服务场景下,分布式跟踪成为此类方法的新技术形态并广受欢迎^[49]。

基于程序运行反馈信息的状态监视技术。在程序运行过程中,应用系统往往会产生大量的反馈信息,监视此类信息可以帮助安全测试与分析人员了解系统的运行情况。典型的程序运行反馈信息包括 HTTP (Hypertext Transfer Protocol, 超文本传输协议) 响应等,这些反馈信息可能包括程序接收到的用户请求、程序的运行结果等^[50, 51]。通过分析和处理这些反馈信息,安全分析人员可以发现潜在的安全问题,并为后续测试工作提供指导。

综上所述,当前动态程序执行信息获取及状态监视技术主要分为侵入式及无侵入式两类,其中:侵入式方式通过向程序源代码或编译后代码插入数据采集探针以实现程序的动态执行信息获取,其面临的主要问题是难以实现执行信息获取的效率和效果的均衡;无侵入方式主要通过收集程序的反馈输出或以沙箱形式监视程序的资源消耗等行为以分析程序的运行时特性,其主要面临所获取信息全面性的挑战。与此同时,现有动态程序执行信息分析技术难以提取程序执行的关键逻辑,进而难以构建逻辑漏洞的特征模型,制约逻辑漏洞的建模及分析效率。

1.2.3 适用于云平台通用组件的传统漏洞与逻辑漏洞挖掘方法

传统漏洞挖掘方法主要分为几个方向,包括基于静态程序分析的静态漏洞挖掘方法^[52]、基于动态分析的动态漏洞挖掘方法^[53],以及近些年引起广泛关注的基于机器学习及深度学习的漏洞挖掘方法^[54-56]。其中静态分析重点关注程序的语义信息、数据流控制流等信息,通过分析敏感函数、漏洞模式等,无需实际运行程序即可挖掘漏洞;动态分析方法则重点关注程序运行过程中的调用特征、对资源的占用消耗情况、运行时错误等信息以发现漏洞;基于机器学习及深度学习的漏洞挖掘方法需要依据专家人员的先验知识,通过训练大量含有和不含漏洞的代码样本、代码图等信息,自动构建漏洞的分类模型,从而检测漏洞。

对于逻辑漏洞的挖掘方法,业内尚未进行完善的梳理与分类,因此本文按照其与云平台通用组件的关联性、适用能力进行分类。云平台软件应用运行于云平台基础设施之上,云平台应用封装架构自下而上主要包含:代码层组件,即基础的代码构建块,包括库、框架和其他可重用代码模块,为构建更高级的业务逻辑提供基础;业务层组件,由代码层组件构造而成,实现具体的业务逻辑,是构成应用的中间层,负责处理资源管理等任务;应用层组件,直接运行在云平台上的通用软件,为最终用户提供完整的应用服务。针对此三类软件应用,相关研究者同样遵循静态分析、动态分析等基本思路,对逻辑漏洞挖掘技术展开研究。

1.2.3.1 适用于云平台代码层组件的漏洞挖掘方法

云平台代码层通用组件主要包括库、框架和其他可以重用的代码模块,如 GitHub、Maven Repository、PyPI 等开源代码仓库所提供的各类供应链第三方组件。这些组件的软件形态包括源代码、字节码或二进制代码等,针对此类组件的

漏洞挖掘方法主要包括静态分析方法和动态分析方法,以及基于机器学习或深度学习的漏洞挖掘方法。针对其中最为典型的云平台代码层组件的逻辑漏洞,如算法逻辑漏洞,相关研究者提出漏洞挖掘方法包括:

基于静态方法的算法逻辑漏洞挖掘方法。Awadhutkar 等人^[57, 58]对程序的算法逻辑进行抽象,生成循环终止条件依赖图以及循环投影控制图,以提供循环视图抽象,并提出 DISCOVER 工具,用于辅助安全分析人员进行交互式审计;与此同时,作者认为完全自动化的算法逻辑漏洞检测是几乎不可能实现的,因此该工具需要分析人员以此为基础进行进一步深入分析。

基于动态方法的算法逻辑漏洞挖掘方法。Blair 等人^[59]提出的 HotFuzz 工具、Wei 等人^[60]提出的 Singularity 工具等,均采用模糊测试的方式,通过随机生成初始样本并利用遗传算法生成变异样本,实现对算法输入的变异测试,并通过虚拟机监视等手段监控算法运行时资源消耗,以发现算法的极端复杂度,从而发现算法逻辑问题。

基于混合方法的算法逻辑漏洞挖掘方法。由于传统符号执行技术存在所发现的敏感函数路径误报率高、可能发生路径爆炸等问题,Liu 等人^[61]、Noller 等人^[62]、Liu 等人^[63]提出混合方法,将动态方法与静态方法结合,利用动态方法验证符号执行技术初筛得到的疑似存在漏洞的调用路径,并构造真正导致程序逻辑异常的样本,在发现算法逻辑漏洞的效率上得到了大幅度提升。

综上所述,现有适用于云平台代码层组件的漏洞挖掘方法大多面临如下问题:对于静态方法,仍然存在误报率高、状态爆炸等问题,同时缺乏高效可靠的逻辑漏洞分析模型作为支持;而动态方法尽管能够准确发现漏洞,但受限于所构造样本的有效性,存在大量漏报情况,同时无法准确地揭示漏洞的本质成因,尽管其在漏洞验证方面实现效率提升,但整体漏洞挖掘与分析流程并未得到显著优化。

1.2.3.2 适用于云平台业务层组件的漏洞挖掘方法

云平台业务层通用组件主要包括各类网络、存储、计算等资源的管理组件,如 OpenStack 开源云计算平台、AWS Cloud 及 Microsoft Azure 的资源管理平台等,这类组件通常对外提供 Web 接口,充当用户和云资源之间的中介,负责解析用户的请求并转化为对具体资源的操作。此类组件通常通过 REST API 对外提供资源管理接口,以实现用户与底层资源的交互。适用于此类软件的漏洞挖掘方法主要包括针对 API 的黑盒模糊测试方法和白盒模糊测试方法。

黑盒模糊测试方法主要基于 API 文档信息进行 REST API 的动态测试^[64-67]。微软提出的 RESTler 工具^[68]实现了一种有状态的云服务测试方法,基于 API 文档分析并构造测试样本,并提出若干主动检查器以发现安全问题^[69];Deepa 等人^[70]提出 DetLogic 工具,开发三类 Web 应用程序逻辑漏洞检测模型,用于检测参

数操控、访问控制和工作流绕过漏洞；Liu 等人^[71]提出一种基于模型的测试方法，构造 REST 服务属性图，进而对漏洞进行建模并指导测试样本序列生成。

白盒模糊测试方法以 EvoMaster 的白盒运行模式为代表，该工具首先在黑盒状态下基于 API 接口文档描述信息进行测试用例生成，并在白盒状态下使用插桩技术来监控应用程序的内部行为，例如记录方法调用和变量值，进而指导基于路径覆盖或分支覆盖的测试用例生成。因此白盒模式下的 EvoMaster 能够执行更精细、更高质量的测试用例生成，因为它能够根据插桩技术所获得的应用程序的实际代码结构和行为来指导测试用例的生成^[72-76]。

综上所述，现有适用于云平台业务层组件的漏洞挖掘方法以动态测试方法为代表，然而所关注的安全问题大多为可靠性问题，即以发现服务端的处理异常为主要目的，因此现有方法所构造的测试样本大多基于字符串随机变异，难以发现 API 之间的逻辑问题；与此同时，现有方法所关注的覆盖率指标大多为代码分支覆盖率，而对逻辑的覆盖、对深度的覆盖则鲜有考虑。因此业务层通用组件的逻辑漏洞挖掘技术仍有广阔的发展空间。

1.2.3.3 适用于云平台应用层组件的漏洞挖掘方法

云平台应用层通用组件主要包括各类可通过迁移部署直接向用户提供服务的组件，如各类 Web 应用、数据库服务、分析工具，以及机器学习预训练模型等。此类应用往往被云平台运营者预先部署或被用户自定义部署，并通过接口向用户提供服务。此类应用一旦被部署，便在服务器上持续运行，其内部植入的逻辑漏洞将随之对云平台及用户持续造成威胁。适用于此类通用组件的漏洞检测方式主要包括运行前漏洞检测以及运行时漏洞检测。

运行前漏洞检测以静态代码检测技术为代表，大多以程序或样本文件本身内容为入手点进行分析。传统的基于模式匹配的防火墙或入侵检测系统已被证明对于恶意漏洞的直接植入起到较好的检测效果；Liu 等人^[77]及 Kolter 等人^[78]提出基于文件中提取得到的文本型特征对恶意代码进行检测的方法，分别利用文件头特征及字节码特征进行恶意代码检测；Nataraj 等人^[79]提出将恶意代码转化为灰度图，进而使用支持向量机对样本进行分类的方法，引领了相关技术的发展。

运行时漏洞检测以动态代码检测技术为代表，包括 Liu 等人^[80]提出基于行为关系网络的恶意代码检测方法，通过沙箱模式记录 API 调用、文件读写等操作，进而利用支持向量机进行分类，达到 95.5% 的分类准确率；Wang 等人^[81]提出一种结合图神经网络和图匹配技术的行为检测技术，以识别和区分未知恶意软件与已知恶意软件或良性软件之间的差异；Liu 等人^[82]针对云环境下的 webshell 攻击提出基于深度学习的检测方法，通过代码静态信息和动态行为共同检测恶意代码。

此类应用的典型实例——机器学习预训练模型同样存在漏洞隐患。当前针对此类应用的安全问题研究主要集中在机器模型的对抗方面,而对于模型本身部署阶段可能造成的漏洞隐患则研究较少。Wang 等人^[13]提出在神经网络中嵌入恶意软件的方法,对模型造成的精度损失控制在 1%范围之内;Hua 等人^[83]进一步优化,实现将恶意有效负载隐蔽嵌入在深度学习移动应用程序中,将精度损失控制在 0.4%;但业界对于此类问题的检测方式研究相对较少。

综上所述,现有适合于云平台应用层组件的漏洞挖掘方法以运行前检测以及运行时检测为主,其中运行时检测因为其能够对应用的运行时行为进行建模,因此相对更为有效。但现有相关方法更多关注传统的恶意代码检测,其中静态方法仍然以基于文本信息的分析方法为主,难以发现隐蔽植入的逻辑漏洞;动态方法更多关注传统程序行为模式,不适用于部分新型逻辑漏洞检测;所提出的优化方法更多的关注机器学习或深度学习模型的优化改进,缺乏对于漏洞的解释能力。

1.3 论文的研究内容与创新点

基于本文的选题背景与研究意义,以及当前国内外的发展现状,考虑到当前针对云平台通用组件逻辑漏洞分析与挖掘能力的不足,本文针对四个方面进行研究,涵盖逻辑漏洞的分析与挖掘等方面,具体包括:逻辑漏洞语言无关特征提取与表征技术,基于静态程序逻辑理解的代码层逻辑漏洞挖掘,基于动态程序逻辑推演的业务层逻辑漏洞挖掘,基于程序运行时逻辑特征的应用层逻辑漏洞挖掘。论文的整体研究框架及各研究内容间的逻辑关系如图 1-2 所示:

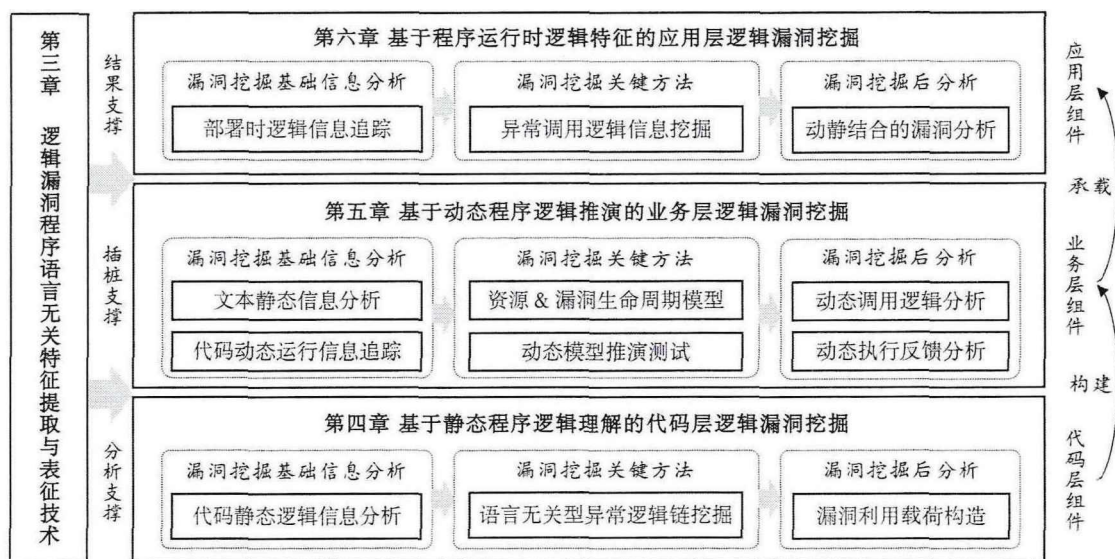


图 1-2 本论文研究内容间逻辑关系

其中,逻辑漏洞语言无关特征提取与表征技术为逻辑漏洞的分析提供图结构表示的基础信息底座支持,可通过该分析底座复现逻辑漏洞,提炼逻辑漏洞的静

态特征结构，构建逻辑漏洞的挖掘模型；基于该信息底座，可进一步支持基于静态程序逻辑理解的代码层逻辑漏洞挖掘、支持基于动态程序逻辑推演的业务层逻辑漏洞挖掘、支持基于程序运行时逻辑特征的应用层逻辑漏洞挖掘。具体而言：

（1）逻辑漏洞程序语言无关特征提取与表征技术

本部分研究内容的设计依据在于，逻辑漏洞通常具有语言无关特性，如特殊的循环递归结构等，这些结构本质上与编程语言特性无关，而与具体的算法设计逻辑相关。因此本部分聚焦于逻辑漏洞的本质特征，以期从纵向层级架构视角对从属关系、调用关系等语言无关型特征进行提取与表示，从横向执行语句视角对逻辑特性进行抽象；进一步构建图形化的信息底座，支持领域特定语言（Domain Specific Language, DSL）风格的逻辑结构检索分析。实验结果表明，本技术能够支持多种类型逻辑漏洞的快速复现及分析，能够支持对目标程序逻辑结构的快速直观筛选，并能够有效支撑逻辑漏洞的挖掘及验证，最终形成原型工具 Luger，支持挖掘代码层、业务层、应用层通用组件逻辑漏洞多项。

（2）基于静态程序逻辑理解的代码层逻辑漏洞挖掘

本部分研究内容的设计依据在于，当前的漏洞挖掘工具大多与编程语言绑定，与语言特性强相关，而逻辑漏洞实际上是一种与具体代码的流程逻辑相关的漏洞，不具有显式的“点式特征”，更多的是隐式的“链式特征”。因此本部分聚焦于代码层逻辑漏洞的链式特征，提出代码层逻辑漏洞挖掘的一般性方法论与 workflow，首先对逻辑漏洞的语言无关型特征进行分析，构建基于算法结构安全策略违背的新型逻辑漏洞分析模型；进一步结合编程语言特性进行漏洞模型实例化，挖掘特定语言下的逻辑漏洞；最后基于静态程序理解构造漏洞利用载荷。实验结果表明，本技术能够实现算法逻辑漏洞的快速准确挖掘，提升漏洞挖掘效率及准确性，最终形成原型工具 ACVFind 并挖掘未公开逻辑漏洞多项。

（3）基于动态程序逻辑推演的业务层逻辑漏洞挖掘

本部分研究内容的设计依据在于，当前针对云平台业务层的安全检测方法大多以 API 模糊测试为主，仍然聚焦于安全问题的触发点特征，所发现的问题更多偏向于业务处理异常而并非逻辑漏洞。因此本部分研究聚焦于业务层组件的 API 设计范式，提出从文本信息及动态执行信息理解业务层逻辑的方法；构建资源生命周期模型，并关联业务逻辑漏洞挖掘模型，提出基于模型推演的样本构造方法，并提出基于参数推断的代码与文本不一致性的检测与缓解方法，有效提升针对业务逻辑的测试深度与广度；最后基于稀疏匹配的方法分析业务运行时特征模式以发现逻辑漏洞。实验结果表明，本技术能够针对云平台业务层通用组件进行逻辑漏洞挖掘，最终实现原型工具 RESTlogic 并发现未公开逻辑漏洞多项。

（4）基于程序运行时逻辑特征的应用层逻辑漏洞挖掘

本部分研究内容的设计依据在于,随着云平台应用服务能力的发展,云平台应用开发人员及个人开发者可能引入其他开发者提供的应用程序,这些应用程序在部署的过程中可能产生安全问题,而现今技术对此类应用逻辑漏洞的检出能力不足。因此本部分聚焦于此类部署型应用,以当前热门机器学习预训练模型为例,提出针对此类应用部署阶段运行时信息的无侵入式追踪方法,构建运行时逻辑特征的表示模型;构建异常运行时行为逻辑的检测模型。实验结果表明,本技术能够对此类通用部署类应用组件进行运行时行为逻辑的追踪与表示,进一步支撑逻辑漏洞的挖掘,提升对潜藏在正常业务背后的后门逻辑漏洞的检测能力,最终实现原型工具 DPDetect 并发现现实网环境开源代码仓库中的恶意模型多项。

综上所述,本文围绕云平台通用组件开展逻辑漏洞挖掘与分析关键技术研究,各研究内容间存在架构关联关系、技术关联及递进关系。其中,研究内容 1 提出普适性的逻辑漏洞语言无关型特征提取与表征技术,对逻辑漏洞的语言无关型成因本质进行抽象及建模,支撑逻辑漏洞复现分析及模型提取,为研究内容 2、3、4 的开展提供信息底座支撑。进一步地,研究内容 2、3、4 从云平台通用组件的组织架构入手,分别针对代码层组件、业务层组件、应用层组件进行漏洞挖掘方法的设计,并提出相应的漏洞挖掘所需逻辑信息的获取及处理方法、逻辑漏洞挖掘模型、逻辑漏洞的检出方法。本文所提出的逻辑漏洞挖掘与分析方法所分析的基础逻辑信息包括代码类信息、非代码类信息,以及静态代码信息、动态程序运行时信息,所提出的逻辑漏洞挖掘方法包括静态方法与动态方法,覆盖云平台通用组件的多种软件形态,覆盖逻辑漏洞挖掘的工作流程环节。

1.4 论文的组织结构

本文主要研究了云平台通用组件逻辑漏洞挖掘与分析关键技术,组织结构如下:

第一章,详细阐述本文的选题背景与研究意义,总结当前国内外针对云平台通用组件漏洞挖掘、特别是逻辑漏洞挖掘的研究现状,分析现有研究的不足之处,阐明本文的研究内容和主要贡献。

第二章,详细阐述本文研究的相关预备知识,包括本文所主要关注的云平台通用组件逻辑漏洞类型、进行逻辑漏洞挖掘所需的相关信息、逻辑漏洞挖掘方法等,作为后续研究内容的预备知识补充。

第三章,针对当前漏洞挖掘方法难以分析逻辑漏洞的发生本质、缺乏对逻辑漏洞挖掘所需通用语言无关型逻辑信息进行提取与表征能力的问题,研究面向逻辑漏洞挖掘的程序语言无关型特征提取与表征技术。

第四章，对于云平台代码层通用组件的逻辑漏洞，针对当前静态漏洞挖掘方法可扩展性不足且误报率过高、动态漏洞挖掘方法随机性较强且效率较低的问题，研究基于静态程序逻辑理解的代码层逻辑漏洞挖掘技术。

第五章，对于云平台业务层通用组件的逻辑漏洞，针对当前业务层逻辑漏洞挖掘方法未形成对目标业务的全面建模、难以实现对目标业务的深入且全面测试的困境，研究基于动态程序逻辑推演的业务层逻辑漏洞挖掘技术。

第六章，对于云平台应用层通用组件的逻辑漏洞，针对当前应用层漏洞挖掘方法缺乏对目标应用运行特征的全面理解、难以挖掘隐藏在目标应用正常行为背后逻辑问题的困境，研究基于运行时逻辑特征分析的应用层逻辑漏洞挖掘技术。

第七章，对本文的研究成果进行总结，并提出未来的研究计划。

第二章 相关技术

2.1 本文所关注的逻辑漏洞的定义及分类

逻辑漏洞分类体系是对逻辑漏洞进行分类和归纳的体系,它主要用于对不同类型的逻辑漏洞进行整理和分类。现存漏洞分类体系如 CNNVD、CVE (Common Vulnerability & Exposure) [84]、CWE[22]、OWASP Top 10[85]等,在帮助安全专家和开发人员理解并防范漏洞、社区知识共享方面提供了很大的帮助,但它们也存在一些缺点和局限性,包括:不完整性,即,分类体系可能无法及时涵盖所有新出现的漏洞类型;复杂性,有些漏洞具有复杂的原因和影响,可能涉及多个不同的漏洞类型,特定漏洞难以实现准确分类。因此,现有的漏洞分类技术体系难以从本质上揭示逻辑漏洞成因(如部分具有不同 CWE 分类编号的漏洞实际上具有相同的本质成因),难以有效指导逻辑漏洞挖掘。

具体而言,本文提出一种针对逻辑漏洞的定义和描述方法,以帮助分析人员快速理解逻辑漏洞的基本原理,描述语句以五元组构成:

{Target, Cause, Entity, Feature, Impact}

其中,Target 为目标应用类型,可理解为发生层次,对于云平台,可包括代码层、业务层、应用层等;Cause 用于描述漏洞的本质成因,如业务流程设计异常、算法逻辑设计异常等;Entity 用于描述漏洞的参与或利用实体,对于算法逻辑漏洞,参与实体为单一攻击者构造的恶意样本,而越权型漏洞的参与实体则包括创建资源的实体以及发生越权的实体;Feature 用于描述逻辑漏洞的完整链式特征,以及最终的触发点特征;Impact 用于描述漏洞的最终影响效果。

具体而言,对于漏洞的发生层次(Target),包括:

- 1) 业务逻辑漏洞,主要发生在业务层面,涉及业务流程和规则的设计与实现,具体包括:业务的设计不规范导致的逻辑漏洞,如业务规则、流程或策略的设计不符合实际需求或存在安全隐患,导致开发所得的代码存在逻辑漏洞;业务的设计与实现不一致导致的逻辑漏洞,如业务规则的设计与实际代码实现之间存在不一致性导致的逻辑处理异常。
- 2) 代码逻辑漏洞,主要发生在应用程序的代码实现过程中,由于算法逻辑的错误或安全实现上的缺陷导致的漏洞,具体包括:代码算法实现的逻辑漏洞,如算法循环、递归、迭代逻辑设计有误,导致程序在特定条件下执行错误操作;代码的权限漏洞,如代码对于用户权限的控制不足,

导致未经授权的用户可以执行敏感操作，如危险函数的使用及危险函数的不当控制。

具体而言，对于漏洞的特征（Feature），包括：

- 1) 漏洞利用链特征，包括非跨文件、组件的利用链特征，即漏洞利用发生在同一个文件或组件内部，具体包括：循环、迭代、递归相关漏洞，即通过恶意输入数据反复执行某一逻辑，导致漏洞的利用和攻击；验证及鉴权不当相关漏洞，即验证机制不当的情况下，应用程序可能无法正确地验证用户输入或用户权限，导致攻击者绕过访问控制并执行未授权的操作。以及跨文件、跨组件的利用链特征，如供应链逻辑漏洞组件引用，以及不同编程语言、不同组件安全约束不一致导致的漏洞。
- 2) 漏洞利用点特征，即最终引起逻辑漏洞生效的关键漏洞利用点，具体包括：鉴权型利用点，如典型的鉴权语句，可能关联越权型逻辑漏洞；代码执行利用点，如典型的代码执行语句，可能关联代码执行型逻辑漏洞；内存占用利用点，如典型的内存占用与释放语句，可能关联内存破坏型逻辑漏洞；时间消耗利用点，如典型的算法递归调用语句，可能关联拒绝服务型逻辑漏洞；打印输出利用点，如典型的命令行输出语句，可能关联信息泄露型逻辑漏洞。

具体而言，对于漏洞的影响效果（Impact），包括但不限于：

- 1) 具备拒绝服务能力的逻辑漏洞：攻击者可以利用该漏洞使目标系统不正常地停止响应服务请求，进而无法正常运行或提供服务。攻击者可能通过输入恶意数据或触发特定逻辑条件以利用漏洞，导致系统崩溃或进入死循环，造成目标系统拒绝服务（Denial of Service, DoS）。
- 2) 具备代码执行能力的逻辑漏洞：攻击者可以利用该漏洞在目标系统上执行任意代码，并实施恶意操作。
- 3) 具备权限提升能力的逻辑漏洞：攻击者可以利用该漏洞在目标系统上获取预期外的操作权限，甚至管理员权限。

基于上述漏洞定义及描述方法，本文所关注的主要漏洞类型为：

- 1) 发生在代码层，由于算法结构设计与安全控制不当，单一攻击者可构造恶意攻击样本，触发无法终止的算法递归调用，进而造成目标系统拒绝服务的逻辑漏洞；
- 2) 发生在业务层，由于业务权限设计与控制不当，单一攻击者可通过构造攻击样本对其他用户造成影响，实现调用链中鉴权逻辑绕过，进而造成权限提升的逻辑漏洞；

- 3) 发生在应用层，由于代码或参数文件保护不当，单一攻击者可嵌入恶意后门逻辑漏洞，在应用部署运行阶段触发反序列化调用链路，进而造成任意代码执行的逻辑漏洞。

2.2 云平台通用组件逻辑漏洞挖掘所需信息

在对逻辑漏洞进行分类描述后，想要针对云平台通用组件进行逻辑漏洞挖掘，首先需要获取逻辑漏洞挖掘所需信息。逻辑漏洞挖掘所需信息包括各类代码类信息、文本类信息。而进一步划分，以代码单元为视角，代码类信息包括静态的源代码信息、字节码信息、以及动态的代码输入与输出，文本信息则包括代码的功能描述文本、代码注释等信息；而以代码间关系为视角，代码类信息包括静态的数据流控制流信息、代码间调用接口的实现等信息、以及动态的函数调用栈等信息，文本信息则包括 API 之间的关联关系等。值得一提的是，上述部分信息可能并非直观地呈现在分析人员面前，需要对各类基础信息进行一定的分析工作以进行获取。

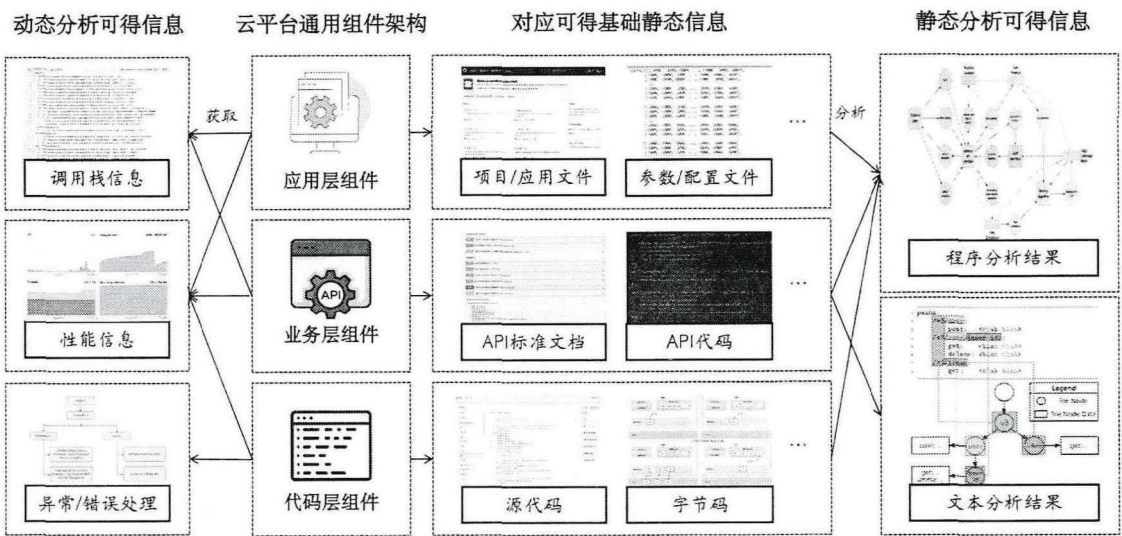


图 2-1 本文所提出的逻辑漏洞挖掘方法所需信息示例

如图 2-1 所示，为本文针对云平台通用组件进行逻辑漏洞挖掘与分析所获取的基础信息示例，其中静态信息包括但不限于：

- 1) 针对云平台代码层通用组件，此类应用所提供的文本信息有限，如 Maven Repository、PyPI 等开源代码仓库仅提供基本描述信息或非形式化功能描述信息，因此主要从代码信息中提取逻辑漏洞挖掘所需逻辑信息。可直接获得的代码信息包括：源代码、字节码等。
- 2) 针对云平台业务层通用组件，此类应用作为用户与资源间的访问中台，为便捷用户使用，通常提供 API 文档以指导用户操作，同时部分组件以

开源形式存在, 因此可直接获得的信息包括代码信息, 即源代码信息, 以及文本信息, 即 API 文档信息。

- 3) 针对云平台应用层通用组件, 此类应用为方便用户部署, 以源代码或封装后代码形式提供给云平台运营者或用户, 以实现应用的快速部署。针对此类应用, 可获得的文本信息有限, 大多应用仅提供基本的非结构化功能描述, 因此可直接获得的信息主要包括代码信息。

而对于本文研究方案中所涉及的动态信息, 则包括:

- 1) 调用栈信息。在宏观角度下, 调用栈信息可包括 API 级别的调用信息; 在微观角度下, 调用栈信息可包括函数调用信息等。调用栈显示了程序运行时的 API 及函数调用顺序、调用层次结构, 甚至可以记录变量值的修改与变化, 反映程序的执行逻辑。
- 2) 性能信息。性能信息包括 CPU 使用率、内存使用情况、响应时间等。性能信息可以反映算法的复杂度情况, 可以反映算法执行期间的资源消耗, 包括时间复杂度, 即算法的执行时间; 以及空间复杂度, 即对于内存的消耗。通过监视性能信息, 可以发现拒绝服务型逻辑漏洞。
- 3) 异常和错误信息。异常和错误处理机制用于捕获程序运行时异常, 以发现并理解程序的异常处理逻辑和潜在的稳定性问题。异常和错误处理机制的使用可以反映代码开发者对于安全问题的预期, 相关的处理机制的滥用、缺失可能反映程序逻辑的设计或实现异常。

针对上述信息, 以其中几类关键信息为例, 相关信息分析获取与表示方法如下所述。

2.2.1 基于图形化代码仓库表征程序静态逻辑信息

如图 2-2 所示, 利用图形化代码仓库 (Graph Codebase), 可以有效地表征程序的静态逻辑信息。通过将代码或代码分析结果转换成图形结构, 例如可将抽象语法树 (Abstract Syntax Tree, AST)^[86, 87]或控制流图 (Control Flow Graph, CFG) 利用图结构进行表示, 研究人员可以直观地理解程序的结构和逻辑流程。其中, 抽象语法树以树状的形式表现编程语言的语法规则, 用树中的每个节点代表源代码的结构如循环、条件判断、函数声明、编程语句等; 控制流图使用节点表示程序中的基本块, 使用边表示程序执行过程中可能的控制流跳转, 如循环、条件分支等。利用此方法, 在漏洞的复现过程中, 研究人员可以直观地识别逻辑漏洞的静态逻辑特征, 如不恰当的循环逻辑或条件判断, 以及潜在的代码执行路径, 为后续的漏洞模型构建以及深入分析提供基础。

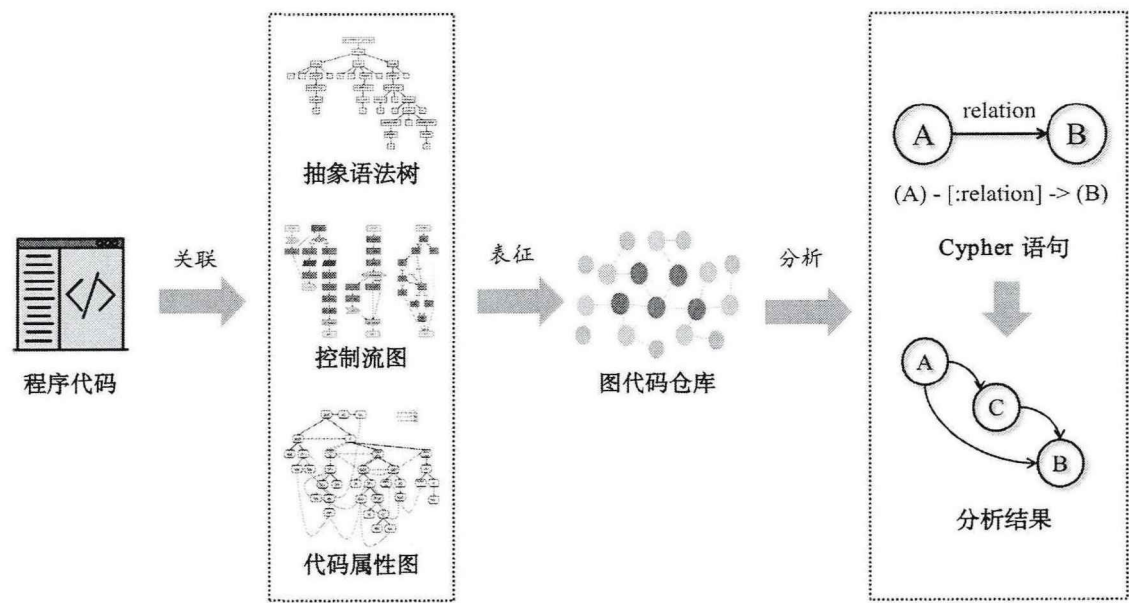


图 2-2 基于图形化代码仓库表征并分析程序静态信息的技术路线原理

2.2.2 基于 REST API 规范文档分析文本蕴含的逻辑信息

REST API 规范文档是云平台业务组件相关逻辑信息的重要来源，通过分析这些文档，可以获取到服务的接口定义、参数说明、以及预期的行为逻辑。REST API 文档大多遵循 OpenAPI 规范，即定义 API 的基本信息、可用路径(Endpoints)、操作、参数以及响应格式等。如图 2-3 所示，通过分析 API 文档，可以提取出关键信息包括接口调用条件、输出格式、可能的错误码、API 组间关联信息等，此类信息对于理解服务的逻辑极为关键，并可被用于指导后续基于动态程序逻辑推演的逻辑漏洞挖掘工作。

提取

提取得到的 DELETE /pet/{petID} 方法

```
deletes:
  tags:
    - pet
  summary: Deletes a pet
  description: ''
  operationId: deletePet
  parameters:
    - name: api_key
      in: header
      description: ''
      required: false
      schema:
        type: string
    - name: petId
      in: path
      description: Pet id to delete
      required: true
      schema:
        type: integer
        format: int64
  responses:
    '400':
      description: Invalid pet value
  security:
    - petstore_auth:
      - write:pets
      - read:pets
```

分析

请求方法: DELETE
Endpoint : /pet/{petID}
参数名称: petID
参数类型: int64
预期响应: 400 Invalid

符合OpenAPI规范的API文档可视化

DELETE /pet/{petID} API声明源代码

图 2-3 基于 REST API 规范文档分析可得文本蕴含的基础逻辑信息示例

2.2.3 基于代码插桩技术获取运行时调用栈逻辑信息

如图 2-4 所示，利用代码插桩技术，即运行时信息获取技术，通过在程序的关键位置插入监控代码或进行程序运行时状态监视，可以在程序执行时获取详细的运行时信息，如调用栈、参数值和返回值。基于此信息可理解程序的运行时行为，并识别潜在的运行时逻辑漏洞。例如，通过分析调用栈信息，可以识别出递归调用过深、异常处理不当等情况，也可识别越权操作、代码执行等不恰当行为，为漏洞分析与挖掘提供依据。

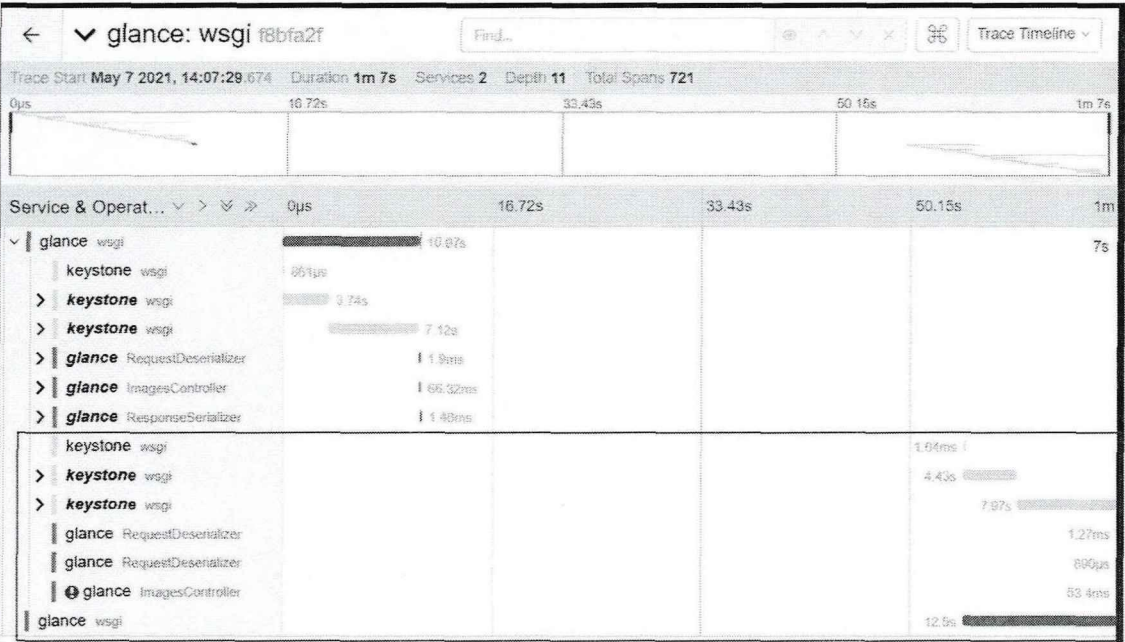


图 2-4 基于装饰器插桩技术对 OpenStack 软件进行插桩并获取运行时调用栈信息示例

2.3 云平台通用组件的逻辑漏洞挖掘与检测方法

在对云平台通用组件的逻辑漏洞类型进行定义和对逻辑漏洞挖掘所需信息进行获取之后，可以进一步提出针对不同类型组件的逻辑漏洞挖掘与分析方法。由于云平台应用架构的多样性，云平台上的通用组件可以分为不同的层级，如代码层、业务层和应用层，每一层的组件特性和交互模式都有所不同，可获得的初始逻辑相关信息不同，逻辑信息的处理方式不同，因此需要采用不同的逻辑漏洞挖掘与分析方法，以适配不同层次通用组件的逻辑漏洞挖掘。

2.3.1 基于静态信息分析漏洞调用逻辑

基于静态信息分析漏洞调用逻辑遵循污点分析技术理念，即通过追踪数据的流向和传播过程来检测数据在程序中潜在的不安全传播，无需实际执行程序便可识别出程序中的安全漏洞。分析的关键环节包括：定义逻辑起始点，近似于污点源，即定义哪些逻辑执行的入口是外部可控的；定义逻辑传播规则，近似于污点传播规则，即定义传递函数与无害化处理函数；定义逻辑利用点，近似于汇聚点，

即定义关键逻辑漏洞敏感函数。基于此理念，以图形化方式复现、建模及查询漏洞利用链，提高漏洞调用链分析效率。

2.3.2 基于静态信息指导动态逻辑测试样本生成

基于静态信息指导动态逻辑测试样本生成遵循动静结合的模糊测试技术理念，即通过静态信息辅助模糊测试阶段的样本构造，以提高发现漏洞的效率。基于文本及代码等静态信息，分析 API 之间的关联关系，构造 API 的生命周期模型，构造符合 API 生命周期模型的漏洞检测模型，构造符合时序约束、关联关系约束的测试样本序列，从而实现对关键业务逻辑的动态测试，最后基于所提出的测试预言，判断是否存在因逻辑漏洞而导致的异常调用。

2.3.3 基于动态信息分析漏洞行为逻辑特征

基于动态信息分析漏洞行为逻辑特征遵循异常检测技术理念，即通过对程序的正常行为模式进行建模，区分正常与异常的行为模式，以发现预期之外的调用逻辑，进而发现逻辑漏洞。基于插桩等技术获取程序调用栈信息（包括 API 调用栈或函数调用栈）之后，对异常逻辑点式结构、异常逻辑链式结构进行检测与识别，能够实现对于逻辑漏洞风险的检测，例如发现因业务逻辑设计不当导致的用户可越权操作其他用户资源或反序列化代码执行等逻辑漏洞。

2.4 本章小结

本章特别针对本文所提出的云平台通用组件的逻辑漏洞挖掘与分析技术所涉及的逻辑漏洞类型、逻辑漏洞挖掘所需信息、基于所获信息挖掘逻辑漏洞的方法进行详细的阐述，为后续针对云平台代码层、业务层、应用层通用组件的逻辑漏洞挖掘方法设计提供详实的依据，为后续逻辑漏洞语言无关型特征提取与表征技术、基于静态程序逻辑理解的代码层逻辑漏洞挖掘技术、基于动态程序逻辑推演的业务层逻辑漏洞挖掘技术、基于程序运行时逻辑特征的应用层逻辑漏洞挖掘技术的研究奠定基础。

第三章 逻辑漏洞语言无关型特征提取与表征技术

随着云环境下计算机软件开发重心逐渐向以云平台软件为首的新型通用软件迁移,软件的主要漏洞类型也逐渐地由传统的内存型漏洞转变为逻辑型漏洞。身份验证漏洞、权限越界漏洞、条件竞争漏洞等逻辑漏洞对新型软件的影响日益显著,此类漏洞与具体的编程语言特性相关性较弱,而与具体的程序设计逻辑相关性较强,因此此类漏洞的成因及外在表现更加隐蔽,业界缺乏行之有效的手段对其进行分析。加之现有针对逻辑漏洞,乃至所有漏洞的静态分析方法,包括数据流分析、控制流分析等通常需要分析人员具备更加深入的领域内知识,一定程度上也延缓了逻辑漏洞分析及挖掘的进程。

在此背景下,相关研究者提出一系列分析友好的程序分析及漏洞分析工具,包括 soot、doop 等静态分析框架工具、基于图结构的代码分析工具^[88, 89]、以及 GitHub 官方提供的代码仓库工具 CodeQL 等^[90]。其中,CodeQL 工具通过将代码转换成数据库,即代码仓库,并构建基于该代码仓库的分析引擎,为分析人员提供发现代码安全问题的能力^[91, 92]。在代码仓库中,代码被视为数据,安全漏洞、异常和其他错误被建模为可从代码仓库中经由查询语句获得的分析结果。然而当前领域内工具大多聚焦于传统漏洞,所需编写的分析语句与编程语言具有较强的相关性,对漏洞复现过程的支撑能力差、对逻辑漏洞的建模能力差,仍然需要分析人员具有漏洞模型的大量先验知识才能高效的分析漏洞。

针对以上问题,为提取与表征逻辑漏洞的语言无关特征,本章节提出一种基于程序语言无关型中间表示的图形化代码仓库技术,通过对逻辑漏洞的语言无关型特征的提取与表征,支持对逻辑漏洞的复现及模型构建。首先,提出面向程序纵向层级架构的语言无关型逻辑特征的提取方法,提取对象包括调用及从属逻辑关系;其次,提出面向程序横向执行语句的语言无关型逻辑特征的提取方法,消除编程语言特性及语法规则差异;最后,构建逻辑关系的图形化表示,并提供 DSL 风格的逻辑关系查询方式,支持逻辑漏洞的快速图形化分析;最后,实现原型工具 Luger,并在 Java 语言和 Python 语言环境下对本方法进行验证,证明本方法在漏洞复现、建模、挖掘等方向的应用前景。

本章组织架构如下:3.1 节对概念定义、研究动机、设计目标进行阐述;3.2、3.3 章节对核心方法进行阐述,包括程序层级架构视角以及执行语句视角的关键逻辑要素的提取与表征方法;3.4 章节对方案进行漏洞复现及分析的仿真验证,并与业内先进研究进行对比,证明本方案的有效性;3.5 节对本章工作进行总结,并提出研究展望。

3.1 问题描述

本章节主要介绍针对逻辑漏洞语言无关型特征提取与表征技术的有关概念定义、设计动机以及设计目标。

3.1.1 概念定义

编程语言特性。编程语言的特性因其设计和目的的不同而不同，不同编程语言具有不同的代码结构和组织方式。Java 语言是一种面向对象的编程语言，通过使用类（Classes）和对象（Objects）在代码中封装状态和行为。Python 语言是一种多范式的编程语言，支持面向对象编程，即通过类和对象的概念提供完整的面向对象编程支持；支持过程式编程，可以编写作用于数据的函数或过程，然后按照特定顺序调用；支持函数式编程，可以创建无状态的函数，并通过高阶函数（接收其他函数为参数）进行操作。总而言之，不同编程语言特性决定了不同的参数函数定义及使用方式，一定程度上加大了程序逻辑漏洞的分析难度。

语言无关中间表示。为了能够兼容多种编程风格范式，需要一种能够有效处理各种不同的代码组织和表示方式中间表示（Intermediate Representation, IR）^[93]模型。具体而言，该 IR 模型应该能够容纳不同语言下，面向对象、面向过程以及函数式编程等不同编程范式中的不同函数和方法定义方式，确保在各种编程环境下都能正确识别和处理函数调用和定义。基于此，有必要在对目标语言进行语义分析之后，构建一套适用不同编程语言、编程范式、编程表达方式的语言无关中间表达形式，以抽取代码中的本质逻辑关系，为逻辑漏洞的成因分析和漏洞挖掘方法研究提供依据。

图形化代码仓库。图形化代码仓库的概念结合了代码仓库（Codebase）的概念和图数据库（Graph Database）的概念^[94]，是一种图形化的代码仓库表达。它结合了传统代码仓库的功能与图数据库的高效数据结构和查询能力^[95]，从而能够帮助分析人员更直观、更高效地管理和理解复杂的代码库。在图形化代码仓库中，代码的结构、依赖关系和调用关系被映射为一个图形网络，其中包含节点（代表如函数、类、模块等代码元素）和边（表示这些元素之间的关系，如调用关系、从属关系等）。在本文中，构建图形化代码仓库的目的在于支撑逻辑漏洞的特征提取与表征，进一步支撑逻辑漏洞的分析与挖掘。

3.1.2 设计动机

如图 3-1 所示，从逻辑漏洞的利用链分析视角进行分析，作为逻辑漏洞的典型代表，Java 语言下的反序列化漏洞利用链通常具有如下相似性特征：起始点为用户可控输入点，如 `FileInputStream`；中间调用链，一系列未经过无害化处理（sanitize）的中间调用，导致用户精心构造的恶意输入能够被 `readObject` 反序列

化方法读取，最终执行如 `Runtime.getRuntime().exec()` 等恶意命令。而在 Python 语言下，官方库 `pickle` 在进行反序列化操作 `pickle.load(bytes)` 时，会调用 `__setstate__` 方法恢复自定义对象 `bytes`，并最终调用 `eval()` 或 `exec()` 等函数，以实现代码执行。综上所述，反序列化逻辑漏洞在不同编程语言中均广泛存在，其具有相似的发生原理与调用特征；然而由于编程语言的特殊性质，不同语言中相同类型漏洞具有不同的触发实例，因此为深入分析逻辑漏洞的成因与特性，有必要构造一种统一化的语言无关表示，揭示逻辑漏洞的本质成因。

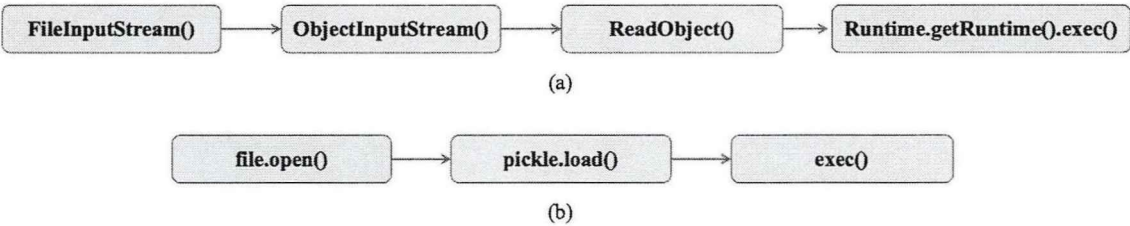


图 3-1 不同编程语言下的简化反序列化漏洞利用链示例

进一步地，从逻辑漏洞的利用点视角进行分析，不同编程语言下程序语句的编写方式存在显著差别，而在相同编程语言下也同样存在一定程度差异。例如对于数组的定义而言，不同编程语言均提供了基本数据类型定义方式、标准模板库定义方式、用户自定义数据类型定义方式。如表 3-1 所示，在 Java 语言下可通过基于内设基本数据类型进行定义 `String[] str_array = new String[10]`，也可通过标准模版库提供的集合框架来进行定义 `List<Array> str_array = new ArrayList<>()`，甚至同样可通过用户自定义代码来进行定义 `org.bupt.MyStrArray str_array = new org.bupt.MyStrArray()`。这些定义方式尽管通过不同的编程语言表达式进行实现，但其在漏洞利用过程中扮演的角色相似、起到的作用相似，因此有必要在分析阶段消除此类差异，以减轻分析负担。

表 3-1 不同编程语言下对统一数据类型的不同定义方式

编程表达方式	类型定义来源	Java 语言表示形式	Python 语言表示形式
基本数据类型	基于内设	<code>String[] str_array = new String[10];</code>	<code>str_array = ["", "..."]</code>
标准模板库类型	基于系统库	<code>List<String> str_array = new ArrayList<>();</code>	<code>str_array = np.array(["", "..."]);</code>
自定义数据类型	用户自定义代码	<code>org.bupt.MyStrArray str_array = new org.bupt.MyStrArray();</code>	<code>str_array = MyStrArray()</code>

3.1.3 设计目标

- 1) 程序层级架构的逻辑信息语言无关化提取，旨在将大型复杂程序中文件、类、函数、语句等不同层级架构的逻辑信息实现编程语言无关化提取，支持逻辑信息的语言无关化表示，进而支撑复杂程序的逻辑漏洞分析。

- 2) 程序执行语句的逻辑信息语言无关化提取，旨在将程序中的执行语句与特定编程语言无关化，以便更容易进行逻辑漏洞的分析和理解，进一步支撑逻辑漏洞的成因本质分析。

3.2 面向程序纵向层级架构的语言无关型逻辑特征提取方法

尽管源代码能够直接反映出编程者对业务逻辑的理解，但在云计算环境中开发过程通常以项目组件为基本单元进行。在一个典型的业务项目中，可能包含数十个组件，每个组件可能又包含数以万计的代码。如果在代码分析时采用过大的粒度，可能会导致分析效率低下和结果覆盖不全，进而影响分析结果的有效性。针对以上挑战，本节提出一种基于纵向层级架构划分的程序逻辑提取和分析方法，旨在解决代码分析效率低下和复杂软件形态下的程序逻辑提取与分析难题，根据不同的分析需求，从代码文件、类、函数、语句四个不同层次进行逐级深入的分析，以适应不同粒度的理解需求。总体而言，在层级架构视角下，相关逻辑关系可被抽象为从属关系及调用关系。

3.2.1 从属关系的提取方法

从属关系，指程序中的一个元素对另一个元素的依赖。由于从层级架构视角可将大型程序划分为文件、类、函数、语句四个层次，因此本节所描述的从属关系同样可从该四个角度进行分析与提取。这种从属关系不仅限于直接的代码调用，还包括数据依赖、资源依赖、环境依赖等，同时存在于软件的任何层次之间，包括文件从属关系、类从属关系、函数从属关系、语句从属关系，以及不同层次之间的从属关系等。从属关系反映了软件系统中各个部分之间的相互依赖性，是理解程序逻辑的关键。具体而言，从属关系如表 3-2 所示，包括但不限于：

表 3-2 从属关系示例

从属关系类型	表达方式	含义
类从属关系	$f_C: C \rightarrow F$	类 C 在文件 F 中被定义
函数从属关系	$f_{Func}: Func \rightarrow C$	函数 func 在类 C 中被定义
语句从属关系	$f_S: S \rightarrow Func$	语句 S 在函数 Func 中被声明
依赖关系	$D \subseteq Comp \times Comp$	相同层级元素之间的依赖关系，如文件引用

基于上述从属关系，本小节设计如下算法流程，对从属关系提取并存储，具体如算法 3-1 所示：

- 1) 确立父节点特性。遍历整个 AST^[96]或基于 IR 构建的 AST（后续统称为 AST），为每个节点分配其父节点。遍历时采用深度优先遍历（Depth-First

- Search, DFS) 算法, 递归地访问每个节点, 同时记录它们的父节点。在完成对某个子树的遍历后回溯到其父节点, 确保父节点特性的准确记录。
- 2) 为节点分配唯一标识。为 AST 中每个节点赋予一个独一无二的标识符, 即 `node_id`, 该过程采用与建立父节点特性相同的遍历策略。这些唯一标识符是后续的节点检索和分析的关键基础。为确保每个节点的标识符唯一性, 需要采取线程安全的措施。
 - 3) 设置节点基本属性。在定义了父节点特性和唯一标识后, 接下来的任务是为节点配置其他基础属性, 包括决定节点是否应存储于图形数据库中、节点的类型、属性名称、调用语句清单等。此步骤的目的在于确保节点属性的完整性和可查询性。
 - 4) 构建从属节点关系。再一次遍历 AST, 除记录父节点和唯一标识, 还需要抽取和设置节点的关键信息, 例如函数定义节点的名称、赋值语句的变量名等。对于一些特殊类型节点, 比如装饰器、控制流语句和异常处理节点, 需要进行额外的处理, 以确保节点间逻辑关系的准确表示。

此外, 为了提高遍历效率, 可以将设置唯一标识和构建从属关系的步骤合并并在同一个遍历过程中完成。此方法不仅减少了遍历 AST 结构的次数, 而且提高了整个算法的效率, 同时也保证了分析结果的准确性。

算法 3-1 从属关系提取算法

输入: 抽象语法树 (AST)	
输出: AST 中节点之间的从属关系 <code>dependencyGraph</code>	
1.	初始化从属关系依赖图 <code>dependencyGraph ← {}</code>
2.	对于 AST 中的每一个当前节点 <code>current_node</code> :
3.	定义元组 <code>child_parent ← {}</code> , 代表当前节点与其父节点间关系,
4.	当 <code>current_node</code> 为 root 节点时, 则
5.	<code>child_parent ← {child_node ← current_node, parent_node ← None}</code>
6.	否则
7.	<code>child ← current_node</code> 的子节点
8.	<code>child_parent ← {child_node ← child, parent_node ← current_node}</code>
9.	<code>current_node ← current_node</code> 的子节点, 将 <code>child_parent</code> 添加到 <code>dependencyGraph</code>
10.	返回 <code>dependencyGraph</code>

3.2.2 调用关系的提取方法

相对于从属关系, 调用关系是软件中更为具体的一种关系, 指的是一个函数、方法或过程直接调用另一个函数、方法或过程。调用关系是从属关系的一种特殊形式, 是实现软件功能时最常见的交互方式, 直接反映了程序的执行流程。调用关系的建立通常可选的包含: 调用者 (Caller), 即执行调用操作的函数或方法;

被调用者 (Callee)，即被调用执行的函数或方法；参数 (Parameters)，即传递给被调用者的数据或信息；返回值 (Return Value)，即被调用者执行完毕后返回给调用者的结果。具体而言，调用关系如表 3-3 所示：

表 3-3 调用关系示例

调用关系类型	表达方式	含义
直接调用	$f(x) = g(y)$	函数 g 的返回值传递给函数 f
方法调用	$ObjA.mA() = ObjB.mB()$	对象实例作为函数或方法进行调用
无返回值调用	$f(x) \rightarrow g(y)$	函数 f 调用无返回值函数 g
事件驱动调用	$Event \rightarrow Callback()$	当事件 $Event$ 发生时，调用 $Callback$

基于上述调用关系，本小节设计如下算法流程，对调用关系提取并存储：

- 1) 提取函数/方法参数列表：在分析函数或方法的 AST 节点时，首要任务是提取其参数列表。以 Python 语言为例，Python3.8 中定义的多参数类型，包括 `posonlyargs`（位置限定参数）、`args`（常规参数）、`vararg`（可变数量的位置参数）、`kwonlyargs`（关键字限定参数）和 `kwargs`（可变数量的关键字参数）。通过对不同类型参数的识别和整理，形成完整的函数参数列表，进而反映编程语言对函数参数处理方式的细节。
- 2) 提取赋值信息：尤其关注类构造函数内的赋值调用。由于类的构造函数中的赋值在对象初始化时就会执行，因此识别此类赋值语句至关重要。此过程包括定位赋值语句，并运用正则表达式来从中提取变量和被调用对象的相关信息。
- 3) 查找函数调用语句：在 AST 节点中寻找所有可能的函数调用语句是系统的核心部分，对于理解程序的执行逻辑和数据流尤为关键。包括处理多属性嵌套的调用情况，具体操作如步骤 4 所示，并采用递归方法来提取复杂的调用序列。
- 4) 调用序列的化简和节点 ID 匹配：对于包含多个属性嵌套的复杂调用序列，通过一系列判断和赋值信息的匹配来进行化简。假设存在两个类别 $A()$ 和 $B()$ ，类 $A()$ 中属性 b 为类 $B()$ 的实例，而 $B()$ 中存在一个函数 $c()$ ，则 $A()$ 的实例 a 触发的函数调用 $a.b.c()$ 可利用赋值信息 $A.b()=B()$ 化简为 $B.c()$ 。经过化简的调用序列会被映射到逻辑图中相应的节点上，并利用前文所述唯一节点标识符 `node_id` 进行标识。这一过程涉及区分系统调用、库函数调用以及源代码中定义的函数或方法调用。
- 5) 建立逻辑图中的调用关系：完成上述步骤后，系统能够构建起函数和方法节点间的调用关系，通过将这些逻辑关系反映在图结构并存储于图形

化代码仓库中，可以为程序逻辑的进一步理解奠定基础，提供关键信息支撑逻辑漏洞的复现及挖掘。

具体流程如算法 3-2 所示：

算法 3-2 调用关系提取算法

输入： 抽象语法树（AST）	
输出： AST 中节点之间的调用关系 logicGraph	
1.	初始化 logicGraph $\leftarrow \{$
2.	nodes $\leftarrow \{\}$ ，用于存储所有逻辑图中的节点
3.	callmappings $\leftarrow \{\}$ ，用于存储所有逻辑图中的调用关系
4.	$\}$
5.	初始化 mapping_infos $\leftarrow \{\}$ ，用于全局保存 node 与 node_id 间的映射关系
6.	对 AST 的每个节点 node:
7.	初始化参数列表 params $\leftarrow \{\}$
8.	对于 node 的每一个参数 param:
9.	如果 param 的类型是 posonlyargs, args, vararg, kwonlyargs, kwargs:
10.	添加参数 param 到 params
11.	节点 node 的参数 node.params $\leftarrow$ params
12.	
13.	初始化赋值关系列表 assignments $\leftarrow \{\}$
14.	如果 node 的类型是类定义 ClassDef 或函数定义 FunctionDef:
15.	定义当前节点与 node_id 的映射:
16.	mapping_info $\leftarrow \{ \text{mapping_node} \leftarrow \text{node}, \text{node_id} \leftarrow \text{node.node_id} \}$
17.	将赋值信息 mapping_info 添加到全局映射关系 mapping_infos 中
18.	对于 node 的每一个子节点 child:
19.	如果子节点 child 是赋值语句:
20.	定义赋值信息:
21.	assignment $\leftarrow \{ \text{target} \leftarrow \text{child.target}, \text{value} \leftarrow \text{child.value} \}$
22.	添加赋值信息 assignment 到 assignments
23.	节点 node 的赋值信息 node.assignments $\leftarrow$ assignments
24.	
25.	初始化调用关系 calls $\leftarrow \{\}$
26.	如果 node 的类型是函数调用 Call:
27.	定义当前调用:
28.	callee $\leftarrow$ node.func，即该节点调用的函数的名称
29.	添加调用序列 callee 到 calls
30.	否则:

续算法 3-2 调用关系提取算法

31.	对节点 <code>node</code> 的每个子节点 <code>child</code> :
32.	将 <code>child</code> 作为 <code>node</code> 进行递归
33.	节点 <code>node</code> 的调用信息 <code>node.calls</code> $\leftarrow$ <code>calls</code>
34.	
35.	对 AST 的每个节点 <code>node</code> :
36.	初始化 <code>callmappings</code> $\leftarrow$ <code>{}</code>
37.	对于 <code>node.calls</code> 中的每一个调用信息 <code>callee</code> :
38.	如果 <code>callee</code> 长度小于等于 2, 且 <code>callee</code> 为 <code>mapping_infos</code> 中的键:
39.	查找并获取 <code>callee</code> 在全局映射关系 <code>mapping_infos</code> 中对应的 <code>node_id</code>
40.	定义调用关系:
41.	<code>callmapping</code> $\leftarrow$ <code>{callerid</code> $\leftarrow$ <code>node.node_id</code> , <code>calleeid</code> $\leftarrow$ <code>node_id</code> }
42.	将调用关系 <code>callmapping</code> 添加到 <code>callmappings</code>
43.	否则:
44.	使用节点赋值信息 <code>node.assignments</code> 化简 <code>call</code>
45.	节点 <code>node</code> 的调用关系 <code>node.callmappings</code> $\leftarrow$ <code>callmappings</code>
46.	将 <code>node.callmappings</code> 添加到 <code>logicGraph.callmappings</code>
47.	将 <code>node</code> 添加到 <code>logicGraph.nodes</code>
48.	返回 <code>logicGraph</code>

3.3 面向程序横向执行语句的语言无关型逻辑特征提取方法

在以程序纵向层级架构为切入点,对逻辑链语言无关特征进行分析之后,还需进一步以横向分析为视角,对具体逻辑点的编程语言实现进行语言无关特征分析,以支撑多语言下同类型逻辑漏洞的复现分析及通用逻辑漏洞挖掘模型的提取。本节采用一种抽象方法提取与语言无关的程序执行语句的逻辑信息,将不同编程语言中的具体语句映射到更通用的逻辑表示上。此方法的核心是识别出编程语言中的基本结构和控制流元素,然后将其转化为通用的逻辑表示。具体提取过程的步骤以及相应的示例如下:

- 1) 识别基本控制流结构: 识别各种编程语言中的基本控制流结构,如条件判断、循环、函数调用等。
- 2) 定义通用逻辑表示: 为每种基本控制流结构定义一个与语言无关的通用表示。
- 3) 映射语言特定结构到通用表示: 将不同语言的具体语法结构映射到预先定义的通用逻辑表示上。

表 3-4 示例为不同编程语言下具体代码实现的通用逻辑表示，由于篇幅限制以及代码实现的多样化，本小节仅列出部分具体实现作为参考示例。

表 3-4 不同编程语言下具有相同语义的代码表示示例

通用代码要素	Python 语言	Java 语言
条件分支	if-elif-else	if-else
循环	for, while	for, while
函数定义	def	Void [#]
异常处理	try-except	try-catch
类定义	class	class
字典/哈希表	dict	HashMap, Map
元组/结构体	tuple	Custom class
数组/列表	list	Array, ArrayList
集合	set	HashSet, Set
枚举	enum.Enum	enum
执行代码	exec, eval	ScriptEngine
文件操作	open, with	FileReader, FileWriter
网络请求	requests	HttpURLConnection
数据库交互	sqlite3, pymysql	JDBC
JSON 处理	json	JSONObject, JSONArray

注：其中 Java 语言函数定义方式为：返回值类型+函数定义，void 为无返回值类型。

3.4 实验评估

本章节最终实现原型工具 Luger。针对本章节所提出方法以及所实现原型工具的实验评估主要从三方面进行：一是对于逻辑漏洞的分析与复现能力，旨在评估本工具能否准确识别并分析已知的安全漏洞，同时评估工具对不同类型逻辑漏洞本质成因的表征能力；二是对于漏洞模型的总结与提取能力，旨在评估本工具能否基于所总结的漏洞模型开展实际的漏洞挖掘，并评估工具的漏洞挖掘效果；最后，将本章成果与领域内的代表性研究进行综合对比分析。

本部分的实验内容运行于 Windows11 操作系统，图形化代码仓库依赖 Neo4j 图数据库系统，图形化所用 GPU 型号为 NVIDIA GeForce RTX 3070 Laptop，配置对应 Python、Java 语言环境，安装 neo 图形化相关工具/插件 neo4j desktop\neo4j bloom，配置 Web 控制界面 neo4j web console 的服务端与客户端。所测试目标程序包括 Java Maven 仓库的热门开源组件和 GitHub 中的 Python 语言开源项目。

3.4.1 对于逻辑漏洞的分析及复现能力

针对研究问题 1，本节针对主要的逻辑漏洞类型进行复现及分析，包括代码执行型逻辑漏洞、信息泄漏型逻辑漏洞、条件竞争型逻辑漏洞、拒绝服务型逻辑漏洞等。上述漏洞的原理定义如下所示：

代码执行型逻辑漏洞，如公式 (3-1) 所示：

$$E(T(S)) \quad (3-1)$$

其中 S 代表污点源集合，即接受用户输入的点； T 代表污点传播路径，或称之为函数调用链； E 代表最终执行代码的函数。

信息泄漏型逻辑漏洞，如公式 (3-2) 所示：

$$O(T(S)) \quad (3-2)$$

其中 S 代表敏感数据源集合，即可能泄漏的信息； T 代表污点传播路径，或称之为函数调用链； O 代表可能导致信息泄漏的输出函数。

条件竞争型逻辑漏洞，如公式 (3-3) 所示：

$$Race(T1(C1), T2(C2), R) \quad (3-3)$$

其中 R 代表资源集合， $C1$ 和 $C2$ 是对资源的操作， $T1$ 和 $T2$ 是不同的线程， $Race$ 代表对资源的竞争关系。

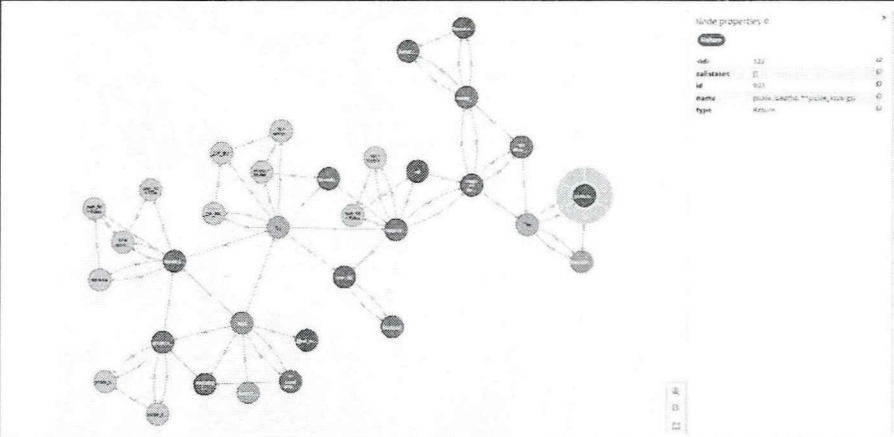
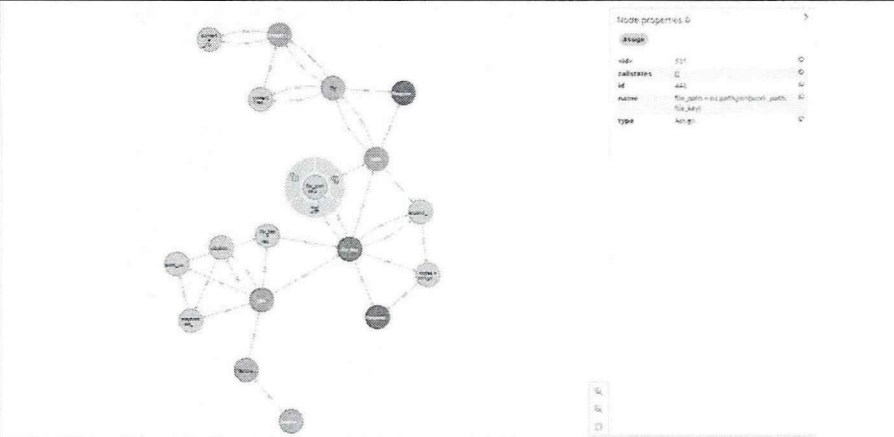
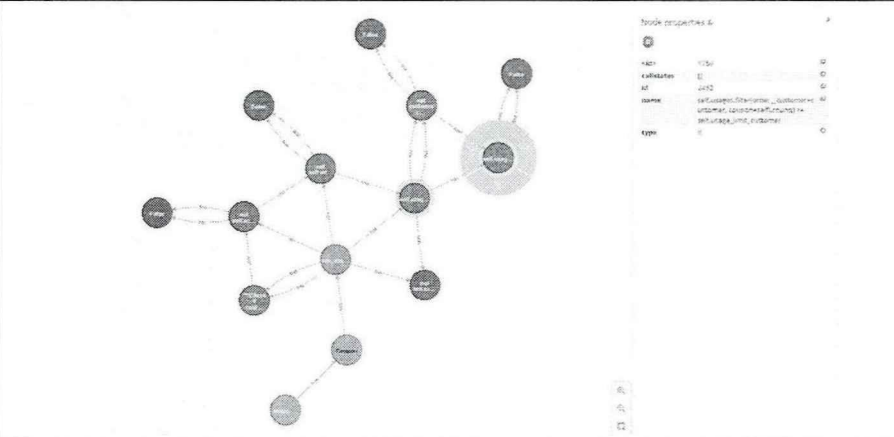
拒绝服务型逻辑漏洞，如公式 (3-4) 所示：

$$Dos(T(C), R) \quad (3-4)$$

其中 R 代表资源集合， C 代表对资源的操作， T 代表污点传播路径，或称之为函数调用链， Dos 代表可能造成拒绝服务或资源 R 耗尽的函数或算法。

基于以上漏洞原理模型，本节对已公开漏洞进行分析及验证。如表 3-5 所示为本节的实验结果，分别展示了图形化代码仓库的调用链图形化分析结果及漏洞的实际原理链路。对于拒绝服务型逻辑漏洞，由于篇幅限制及研究内容的承接关系，本文将在第四章进行详细阐述。本部分重点列举部分漏洞进行展示，包括代码执行型 CVE-2019-6446 等，信息泄漏型 CVE-2023-42819 等，条件竞争型 CVE-2023-43970 等。具体如表 3-5 所示：

表 3-5 本工具 Luger 对于不同类型逻辑漏洞的分析结果

漏洞类型	实验结果
代码执行型	漏洞编号: CVE-2019-6446
	
	原理链路: NumPy.lib.npyio.py → load() → pickle.load()
信息泄漏型	漏洞编号: CVE-2023-42819
	
	原理链路: PlaybookFileBrowserAPIView → get() → file_key → os.path.join(work_path, file_key)
条件竞争型	漏洞编号: CVE-2023-43970
	
	原理链路: Coupon → can_use_code → filter().count() → usage_limit_customer() Coupon → CouponUsage.add_usage() → cls.object.create → count()

对于代码执行型逻辑漏洞，以 CVE-2019-6446 为例，`pickle.load()`是一个典型的具有代码执行风险的调用逻辑点，基于图形化代码仓库可直观地找到该漏洞相关的完整调用链路，即 `NumPy.lib.npyio.py` $\rightarrow$ `load()` $\rightarrow$ `pickle.load()`，其中：`NumPy.lib.npyio.p` 中的 `load` 方法由于其默认参数 `allow_pickle = True`，因此可执行 Try 节点分支下的 `pickle.load()` 函数；同时链路中不存在任何无害化处理，因此可以触发造成代码执行的逻辑漏洞。

对于信息泄漏型逻辑漏洞，以 CVE-2023-42819 为例，`os.path.join()`是一个典型的具有目录穿越并读取任意文件的函数，基于图形化代码仓库可以直观地找到该逻辑漏洞相关的完整调用链路，即 `PlaybookFileBrowserAPIView` $\rightarrow$ `get()` $\rightarrow$ `file_key` $\rightarrow$ `os.path.join(work_path, file_key)`。在链路中，`filekey` 的参数是用户可控参数，且链路中不存在无害化处理函数，因此用户可以拼接路径并读取文件，进而触发造成信息泄漏的逻辑漏洞。

对于条件竞争型逻辑漏洞，以 CVE-2023-43970 为例，基于图代码仓库可以很容易地找到一个被多个操纵共享的参数，即 `count` 值。一方面创建订单过程中优惠券使用操作进行 `count` 值的“生产”工作，即 `Coupon` $\rightarrow$ `can_use_code` $\rightarrow$ `filter().count()` $\rightarrow$ `usage_limit_customer()`；另一方面后台对用户使用优惠券的校验过程进行 `count` 的“消费”工作，即 `Coupon` $\rightarrow$ `CouponUsage.add_usage()` $\rightarrow$ `cls.object.create` $\rightarrow$ `count()`。然而“生产”与“消费”并发操作的直接相关上下文中并不存在线程保护处理，因此存在对 `count` 值的竞争操作，从而产生逻辑漏洞。

3.4.2 对于逻辑漏洞模型的总结与提取能力

进一步地，需验证本方法对漏洞模型的总结与提取能力，以验证本方法能否主动发现特定的逻辑调用模式。本方案可以将最终的漏洞模型总结为代码仓库的 DSL 风格查询语句，通过该查询语句可实现漏洞调用链的搜索。

对于代码执行型逻辑漏洞，检索规则可抽象为公式 (3-5)：

$$C = \{P \mid P = [s \rightarrow F1 \rightarrow \dots \rightarrow Fn \rightarrow t], \quad (3-5)$$

$$\forall s \in \text{Source}, \forall t \in \text{RCEFunc}, \forall Fi \notin \text{SafeFunc}\}$$

对于图代码仓库的 DSL 风格查询语句，可以写为：

```
MATCH p=(s)-[*]->(t)
WHERE s IN Source AND t IN RCEFunc
AND NONE(n IN nodes(p) WHERE n IN SafeFunc)
RETURN p
```

对于信息泄漏型逻辑漏洞，检索规则可抽象为公式 (3-6)：

$$C = \{P \mid P = [s \rightarrow F1 \rightarrow \dots \rightarrow Fn \rightarrow t], \quad (3-6)$$

$$\forall s \in \text{Source}, \forall t \in \text{LeakPoints}, \forall Fi \notin \text{Safe}\}$$

对于图代码仓库的 DSL 风格查询语句，可以写为：

```
MATCH p=(s)-[*]->(t)
WHERE s IN Source AND t IN LeakPoints
AND NONE(n IN nodes(p) WHERE n IN Safe)
RETURN p
```

对于条件竞争型逻辑漏洞，检索规则可抽象为公式 (3-7)：

$$C = \{P \mid P = [T1(C1) \cap T2(C2) \rightarrow R], \quad (3-7)$$

$$\forall C1, C2 \in \text{ConcurrentOps}, R \in \text{SharedResources}$$

$$\neg \text{HasThreadProtection}(C1, R), \neg \text{HasThreadProtection}(C2, R)\}$$

对于图代码仓库的 DSL 风格查询语句，可以写为：

```
MATCH p1=(op1)-[*]->(r), p2=(op2)-[*]->(r)
WHERE op1 <> op2 AND r IN SharedResources
RETURN p1, p2, r
```

或：

```
MATCH p=(op)-[*]->(r)
WHERE r IN SharedResources
AND NOT (op)-[:HAS_THREAD_PROTECTION]->(r)
RETURN p, op, r
```

其中，前者为不同操作竞争同一资源，而后者为单一操作在无线程保护的情况下对资源进行竞争。

对于算法复杂度逻辑漏洞，检索规则可抽象为公式 (3-8)：

$$C = \{P \mid P = [s \rightarrow F1 \rightarrow \dots \rightarrow Fn], \forall s \in \text{Inputs}, \quad (3-8)$$

$$\exists Fi \in \text{RecursiveFunctions}, \text{depth}(Fi) = \infty, \neg \text{caught}(Fi)\}$$

对于图代码仓库的 DSL 风格查询语句，可以写为：

```
MATCH p=(m:Method)-[r:CALLS*]->(n:Method)
WHERE m = n AND NOT EXISTS {
    MATCH (m)-[:HAS_DEPTH_LIMIT]->()
} AND NOT EXISTS {
    MATCH (m)-[:CAUGHT_BY_EXCEPTION_HANDLING]->()
}
RETURN m, r, n
```

基于上述检索规则及查询语句，本文对实际的逻辑漏洞利用链筛选及漏洞挖掘工作进行实际操作，并与业内先进工具进行对比。

表 3-6 本工具 Luger 与业界知名工具在逻辑漏洞检测方面的对比

漏洞				Luger		Bandit	Semgrep	Snyk
测试对象	Star 数	漏洞编号	漏洞类型	结果	节点	[97]	[98]	[99]
Numpy	25.7k	CVE-2019-6446	代码执行	✓	33	×	✓	×
Pandas	41.2k	CVE-2020-13091	代码执行	✓	28	✓	✓	×
motioneye	3.7k	CVE-2021-44255	代码执行	✓	72	×	✓	×
CERT	47	CVE-2022-40238	代码执行	✓	47	×	✓	×
reportlab	-	CVE-2019-17626	权限提升	✓	8	✓	✓	×
reportlab	-	CVE-2023-33733	权限提升	✓	68	○	×	×
CKAN	4.2k	CVE-2023-32321	权限提升	✓	38	○	×	×
Jumpserver	23.1k	CVE-2023-42819	权限提升	✓	54	×	✓	✓
Jumpserver	23.1k	CVE-2023-42820	信息泄漏	✓	121	×	×	×
Jumpserver	23.1k	CVE-2023-43650	信息泄漏	✓	13	×	×	×
Shuup	2.1k	CVE-2023-43970	条件竞争	✓	19	×	×	×

如表 3-6 所示，为本章节所提出的工具 Luger 与业内知名代码安全检测工具 bandit^[97]、Semgrep^[98]、Snyk^[99]的检测结果对比，其中“✓”表示正确检出漏洞，“○”表示发生错判，而“×”表示发生漏报。对于 11 个目标待测应用，本工具检出全部 11 项漏洞并能够展示完整调用链路，其中 1 项为基于本工具挖掘的 0 day CVE 漏洞（CVE-2023-43970）；而 bandit 只能准确地检测出其中的 2 项漏洞（CVE-2020-13091 和 CVE-2019-17626），有其中两项漏洞 bandit 检测出漏洞的前置缺陷，无法发现逻辑漏洞的完整调用链，进而不能实现漏洞的准确定位及分类（具体而言，CVE-2023-33733 被错判为低危型异常处理相关漏洞，CVE-2023-32321 同样被错判为此类漏洞）；而 Semgrep 及 Snyk 工具分别检出其中 6 项以及 1 项漏洞。同时对于其他待测应用，bandit 等工具也展现出了较差的分析效果，具体在于其分析结果中产生了大量的误报、针对于一系列逻辑漏洞的漏报、以及少量漏洞的错报。经分析，造成此结果的原因在于：

- 1) 因简单规则匹配而产生的误报：bandit 等工具检测的目标漏洞类型更倾向于传统漏洞、点式漏洞，以 SQL 注入类漏洞为代表，此类漏洞最为明显的特征便是具有接受用户输入的查询语句。Bandit 等工具基于此规则对此类漏洞进行规则匹配，但因为其无法深入获取调用链上下文信息，因此无法分析是否中途存在无害化处理，因而产生大量误报。
- 2) 因上下文敏感度较低而产生的漏报：bandit 等工具所检测的目标漏洞类型为传统点式漏洞，其反馈的结果通常是认为“可能”的漏洞代码片段（默认提供 3 行代码上下文），无法反馈完整的函数调用逻辑信息；同

时逻辑漏洞的特点在于单一函数可能为正常，但若干正常函数的不当组合调用逻辑可能产生漏洞，因此 bandit 等工具对此类漏洞无能为力。

- 3) 因过度针对敏感函数检测而产生的错报：bandit 所检测的目标漏洞类型为传统点式漏洞，其对漏洞的检测方式更类似于敏感函数的检测。而在逻辑漏洞的发生过程中，敏感函数并非产生逻辑漏洞的关键点与最终落脚点，漏洞的利用方仍可能通过敏感函数/修复后的敏感函数进行进一步深入操作，以实现漏洞利用，因此 bandit 会产生大量错判。

3.4.3 与业内代表性工作的对比

业内代表性工作主要包含三个方向：其一是基础性程序分析工具，其二是代码仓库工具，其三是基于上述工具的漏洞分析及挖掘信息底座。对于逻辑漏洞的挖掘效果，由于 3.4.3 小节已进行详细阐述，因此本部分不再赘述。本小节主要从工具所依赖的程序表达方式、工具的利用方式、目标漏洞类型、分析过程的人工干预程度进行对比，其中：基础性程序分析工具包括 Soot 等，代码仓库工具主要包含 CodeQL 等，漏洞分析及挖掘底座包括 DISCOVER 等。此三大类工具在各自方向上都具有显著的代表性，且代表了各自方向的最先进的水平，因此本小节主要通过与上述三项工具进行对比，分析本工具的优势所在。

表 3-7 本工具 Luger 与业内先进代表性工具的功能对比

工具名称	表达方式	利用方式	目标漏洞	分析过程
Soot ^[35]	中间表示	自定义分析代码	代码漏洞	全人工
CodeQL ^[90]	代码仓库	自定义查询代码	代码漏洞	半自动
DISCOVER ^[57]	代码抽象	人工查询	循环算法漏洞	半自动
Luger	带抽象的代码仓库	图数据库查询	代码+业务逻辑漏洞	半自动

如表 3-7 所示，本章所提出的方案是一种程序逻辑信息的图形化代码仓库，并提供适度抽象，支撑通过图数据库的查询方式实现程序分析及漏洞复现分析工作。相较于 soot 基于 Jimple IR 的程序分析方法，本工具基于中间表示进行进一步深入分析，用图数据库表征程序逻辑关系，包括从属关系及调用关系；相较于闭源代码仓库工具 CodeQL，本工具提供图形化的代码分析方式，而无需编写大量与编程语言特性紧耦合的 QL 语句，降低学习开销及成本；相较于 DISCOVER 提供的基于图形化循环抽象的算法逻辑漏洞挖掘方法，本工具的适用范围更加广阔，不仅适用于代码层逻辑漏洞，同时能够实现对业务逻辑的表征，具有更广阔的分析范围。值得一提的是，基于本工具实现完全自动化的漏洞分析及复现是不现实的，因为漏洞挖掘本质上仍然是一种经验化工作，但仍可借助本工具实现快速、高效的漏洞复现分析及建模挖掘。

3.5 本章小结

传统的静态程序分析及漏洞挖掘方式更多关注于点式漏洞，且大多聚焦于传统漏洞，对复杂链式逻辑漏洞的复现过程支撑能力差、对逻辑漏洞模型的建模能力差、对漏洞挖掘模型的应用能力差；同时现有漏洞分析的适用范围较窄、需要人工参与过多，使用门槛较高，制约了漏洞复现分析建模的效率。本章提出一种逻辑漏洞语言无关型特征提取与表征技术，从程序的层级架构及执行语句两个角度提出程序逻辑信息的分析与提取算法，实现编程语言的适度抽象，保留程序关键逻辑信息的同时，消除编程语言特性对程序及漏洞分析造成的阻碍，同时可选地保留部分编程语言特性，从而支撑不同情况下的漏洞复现分析。实验结果表明，本章所提出的方案及方法能够对最为典型的代码逻辑漏洞及业务逻辑漏洞进行复现分析及建模工作，能够找到漏洞相关的完整逻辑链路，有助于分析漏洞的本质成因，并指导完成多项 CVE 漏洞挖掘。

与业界内先进工具相比，本工具的优势在于图形化的代码仓库表示、便捷通用的漏洞利用链查询方法，从而极大地帮助安全分析人员提升其对于逻辑漏洞的理解与分析效率。本工具的局限性在于代码的加载过程效率相对较低，由于大型应用程序的逻辑关系复杂，本工具需要进行完善的分析并用图形化代码仓库进行存储，因此前置分析工作需要较长耗时；同时目前阶段仅支持 Python 和 Java 语言的语言无关化表示，加入其他语言需要大量经验性质的分析工作；并且由于编程语言的发展，程序语法规义的复杂化，编程范式的多样化，如面向切面的编程（AOP, Aspect Oriented Programming），因此还需要在规则方面进行更进一步的总结与沉淀，以实现更好的支撑效果。未来需要继续围绕逻辑漏洞挖掘场景针对图形化代码仓库的丰富性、逻辑表征能力进行深入研究，从而更好地支撑逻辑漏洞的复现分析及挖掘工作。

第四章 基于静态程序逻辑理解的代码层逻辑漏洞挖掘

用于构建云平台各类应用的通用代码层组件通常具有复杂的程序逻辑，以支撑复杂业务功能的实现。最为典型的程序逻辑包括：循环递归结构，通过各种循环递归控制语句实现复杂算法逻辑，以便捷程序逻辑设计与代码开发，提升代码的可读性；异常处理机制，用于捕获并处理程序运行时可能产生的异常，提升代码的稳定性与可维护性。与此同时，以开源代码仓库的广泛应用为背景，云平台的开发与应用使用了大量开源第三方组件，开发者可能无意间引入第三方开发者提供的漏洞代码，加剧了此类安全问题的影响程度与影响范围。

在此背景下，相关研究者针对开源组件算法实现逻辑的安全问题进行研究，静态方法以 Awadhutkar 等人^[57]的研究为代表，通过提取循环相关算法逻辑特征并提供可视化视图，辅助安全分析人员人工分析漏洞；动态方法以 Blair 等人^[59]的研究为代表，通过结合遗传算法的导向型模糊测试方法来发现目标算法的最糟糕性能，以发现算法逻辑的安全问题。综上所述，现有研究呈现两极分化态势，其中静态分析层次较浅，需要分析者大量的人工干预；动态方法随机性较强，难以从漏洞的发生本质入手进行漏洞分析与挖掘，制约了发现漏洞的效率。

针对上述问题，本章节提出基于静态程序逻辑理解的代码层逻辑漏洞挖掘方法，针对云平台通用组件的代码层逻辑漏洞，首先提出此类逻辑漏洞分析与挖掘的一般工作流，涵盖逻辑漏洞的建模、挖掘与利用样本构造的全过程；其次，提出基于调用链逻辑特征的静态分析方法，并提出新型基于递归结构安全策略违背算法逻辑漏洞挖掘模型进行验证；最后，实现原型工具 ACVFind 并通过挖掘开源代码仓库热门组件漏洞进行验证，证明本章所提出方案的有效性。所提出的方案与模型能够更高效、更准确地挖掘算法逻辑漏洞，成功发现数十项未公开算法逻辑漏洞并获得 8 项 CVE 漏洞编号。

本章组织架构如下：4.1 节对问题的定义、威胁模型、总体框架进行阐述；4.2、4.3、4.4 节针对典型的代码层算法逻辑漏洞，提出漏洞分析、挖掘与验证的模型与方法；4.5 节对所提出方案及技术方法进行仿真验证，对开源代码仓库中的热门通用组件进行漏洞挖掘测试，并与业界先进研究进行对比；4.6 节对本章工作进行总结。

4.1 问题描述

本节主要介绍针对云平台通用组件的基于静态程序逻辑理解的代码层逻辑漏洞挖掘技术的有关概念定义、威胁模型以及总体框架设计。

4.1.1 概念定义

代码逻辑漏洞。代码逻辑漏洞指的是程序代码中由于逻辑实现不当而产生的安全弱点。传统漏洞如缓冲区溢出或格式化字符串漏洞通常由标准编程错误引起，而代码层逻辑漏洞通常不是源自标准编程错误，而是由于代码的逻辑设计或决策流程存在缺陷而产生。这些逻辑上的缺陷可能导致程序行为异常，允许攻击者绕过安全控制、访问未授权的数据、执行未授权的操作、造成目标系统的拒绝服务，或以其他方式破坏应用程序的安全性。

算法复杂度。算法复杂度 (Algorithm Complexity) 是衡量算法性能如何随着输入大小 n 的增长而变化的指标^[100,101]，通常包括时间复杂度和空间复杂度。时间复杂度指的是算法执行时间相对于输入大小的增长率，而空间复杂度关系到算法的内存需求。算法复杂度分析主要关注当 n 趋向于无穷大时的最坏情况，尽管程序的实际执行时间或内存消耗可能因处理器的速度和编译器的优化而有所不同，但通过此理论仍可帮助分析人员理解并评估算法效率。。

算法复杂度攻击。算法复杂度攻击 (Algorithm Complexity Attack) ^[102-104] 是一种利用算法最糟性能来耗尽系统资源的攻击形式，攻击者可以构造触发后端算法最复杂行为的输入，造成 CPU、内存或磁盘空间的耗尽，进而导致目标系统服务拒绝。典型的算法复杂度攻击包括 Zip 炸弹 (Zip Bomb) 或特别构造的正则表达式导致的 ReDoS (Regular Expression Denial of Service) 攻击^[105-110]。这些攻击有效性在于其无需大量的攻击资源，而是依赖技术策略来实现规模化的服务拒止。

```

1 import java.util.regex.Matcher;
2 import java.util.regex.Pattern;
3
4 public class ReDoSExample {
5     public static void main(String[] args) {
6         String input = "aaaaaaaaaaaaaaaaaaaaaaaaaaaaa!"; // Malicious input
7         System.out.println("Input matches: " + isValid(input));
8     }
9
10    public static boolean isValid(String input) {
11        // Regular expressions that may trigger catalogic backtracking
12        String regex = "(a+)+$";
13        Pattern pattern = Pattern.compile(regex);
14        Matcher matcher = pattern.matcher(input);
15        return matcher.matches();
16    }
17 }

```

图 4-1 算法复杂度漏洞示例

算法复杂度逻辑漏洞。算法复杂度漏洞 (Algorithm Complexity Vulnerability, ACV) 是一种最为典型且具代表性的代码层逻辑漏洞。此类逻辑漏洞由于程序的设计缺陷导致，可被用于算法复杂度攻击，触发时间或空间资源的异常消耗。进一步地，其根本原因在于开发者在算法选择和测试时关注平均情况性能而忽略了

最坏情况下潜在的资源耗尽问题,其中时间复杂度漏洞可能导致 CPU 资源耗尽,而空间复杂度问题可能消耗所有可用的随机访问内存 (RAM) 或磁盘空间。一种典型的算法复杂度逻辑漏洞如图 4-1 所示,其中正则表达式“(a+)+\$”在特定输入(如第 6 行的恶意输入)上性能表现不佳,甚至发生拒绝服务。

4.1.2 威胁模型

本节主要讨论两个方面,即代码层算法设计与实现面临的威胁,以及编程语言共通特性面临的威胁。

对于算法设计与实现面临的逻辑漏洞威胁,首先分析程序安全可靠设计和应用系统可靠运行的关键要素:

- 1) 适当的算法实现:软件供应链中的代码层通用组件对于相同算法逻辑可能有着不同的代码实现方式。不同开发者应采取适当的算法实现,以避免人为或无意引入安全问题。
- 2) 使用输入无害化处理 (Input Sanitization):无害化处理涉及去除用户输入中的任何潜在危险字符,以确保用户输入的数据符合预期的格式和类型^[111,112]。通过限制用户输入的范围,可以有效地减少利用漏洞的风险。
- 3) 实施硬资源限制 (Hard Resource Limits):在某些情况下,代码功能需要更灵活地接受用户输入,并且很难完全限制用户的输入空间。因此,通过严格限制用户可消耗资源的大小,有时可以产生更好的效果。

因此,当违背上述原则时,便有可能产生算法逻辑漏洞。因此,相应的算法逻辑漏洞的威胁模型包括:

- 1) 针对供应链中第三方组件的威胁:供应链中大量的第三方组件实现类似的功能或包含相似的算法片段,这些组件可能有意或无意地引入了包含算法逻辑漏洞的代码片段,进而威胁下游企业或用户。
- 2) 针对输入无害化处理的威胁:无害化处理定义了用户输入空间的格式类型并丢弃或接受某些不合规的字符。在有限的范围内,攻击者可以构建恶意样本并触发意外的算法逻辑,导致时间和空间资源的异常消耗。
- 3) 针对硬资源限制的威胁:即使用户输入空间受到限制,攻击者仍然可以构造恶意输入触发复杂的算法调用如循环、递归和分支嵌套,导致意外的资源消耗,进而触发硬资源限制的上限。

对于编程语言共通特性的威胁,主要包括两个方面:

- 1) 递归及方法递归的设计。算法设计通常会使用循环、迭代、递归等多种结构^[113],这些结构便于软件开发维护和程序理解,但同时也带来了安全性和可靠性隐患,例如循环结构可能存在未定义边界的问题^[114]。本章节的工作专注于递归引用问题,因为与其他循环结构相比,递归结构更加

简单高效，一旦出现问题将会消耗更多资源并产生更大影响，同时递归结构的安全性也常被现有研究所忽视。栈溢出概念与递归概念紧密相关，例如 Java 语言的默认方法栈大小通常只支持大约 2000 次方法递归调用，超过这个限制将触发栈溢出异常。针对递归方法，通过构造特定参数，可以实现深度递归并触发栈溢出。

- 2) 异常和错误处理机制。在程序运行时，可能会遇到各种异常(Exception)，如除以零异常、空指针异常、数组越界异常等^[114]。当这些异常发生时，程序将停止执行并抛出异常；如果异常未被处理，程序将终止执行。这种处理机制对于编程语言至关重要，因为其可以帮助开发及分析人员捕获并处理程序中的各种异常。而运行时产生的错误(Error)一般不应被捕获，但对于一些 Web 应用、安全框架和自定义异常处理器，开发者在开发某些框架或底层组件时可能需要捕获和处理可能产生的错误，以更好地确保程序的安全性和稳定性，特别是对于开源代码仓库和云平台中的组件，Apache Tomcat 和 Spring 框架是两个典型的例子。异常和错误处理机制的缺失让算法逻辑漏洞的实际发生成为可能。

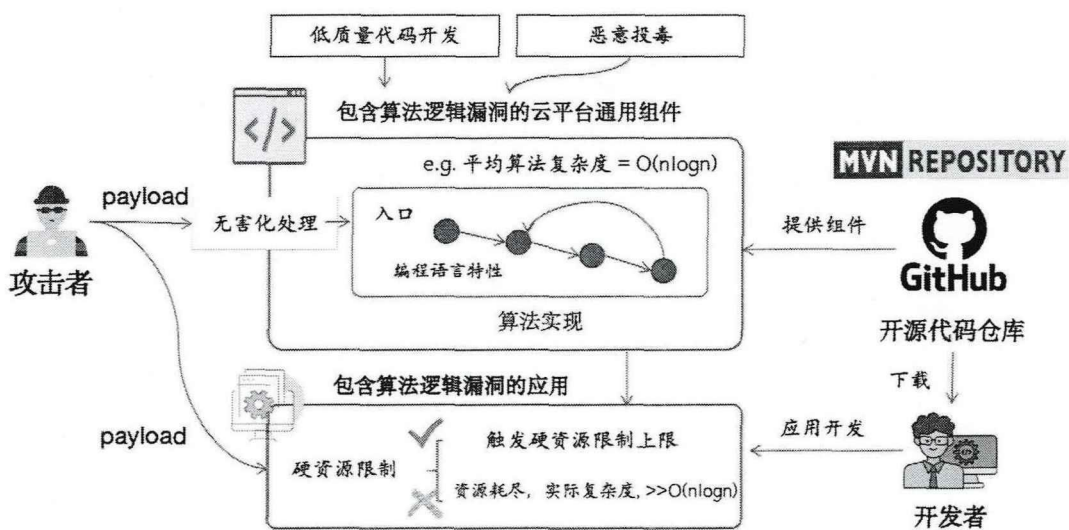


图 4-2 算法逻辑漏洞的威胁模型

如图 4-2 所示，为本章节提出的算法逻辑的总体威胁模型：开源代码仓库如 Maven Repository、GitHub 等为开发者提供第三方组件，以便下载并定制和构建应用程序。假设某个组件中某个算法的平均算法复杂度为 $O(n\log n)$ ，然而由于开发者的粗心编码或恶意篡改，该组件存在算法复杂度逻辑漏洞。攻击者构造符合输入空间要求的样本并利用编程语言共通特性来触发该漏洞，进而直接对组件或通过组件对整个应用程序发起算法复杂度攻击，导致目标系统拒绝服务。极端情况下，攻击者可能达到硬资源限制的上限或无限制地耗尽所有资源，导致实际算法复杂度远超 $O(n\log n)$ 。

4.1.3 总体框架

如图 4-3 所示, 基于所提出的概念定义和威胁模型, 本章节所提出的总体框架包括三个主要部分: 处理、分析和验证。其中处理部分对软件供应链中第三方通用代码组件进行分析, 例如 Maven Java Archive (JAR) 文件, 并得到算法的调用链特征; 分析部分基于所提出的漏洞模型来检测漏洞代码, 包括过滤分析递归终止条件和异常处理机制等算法结构。在验证阶段, 通过对漏洞调用链结构和对编程语言共通特性的理解生成符合输入空间限制的攻击载荷 (payload) 来验证和利用该漏洞。

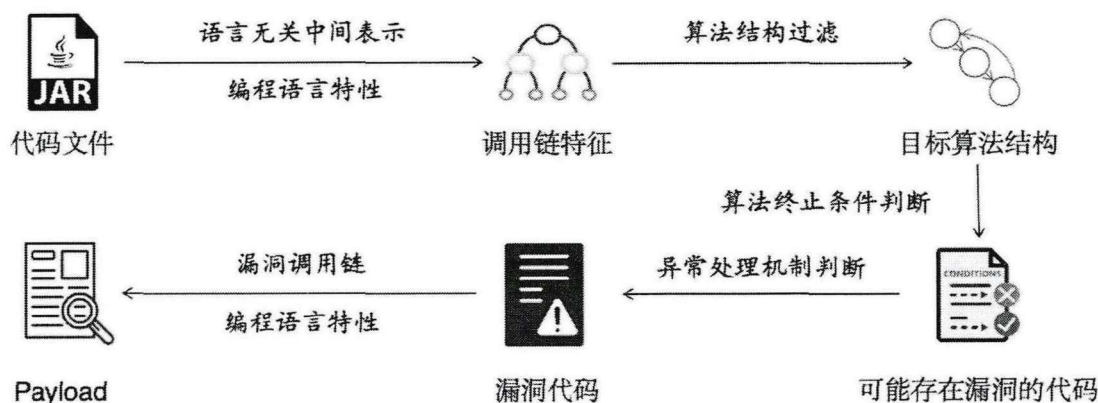


图 4-3 本章节所提出方案的总体框架

4.2 基于递归结构安全策略违背的算法逻辑漏洞模型构造

基于 CWE 漏洞类型及大量已公开漏洞, 可知算法逻辑漏洞大多与循环、迭代、递归调用结构相关。以递归调用结构为例, 该算法结构主要包含以下要素: 1) 基础情况 (Base Case), 即不需要再次递归就能解决的情况; 2) 递归调用 (Recursive Call), 即递归的核心算法流程; 3) 递归深度 (Recursion Depth), 即记录当前递归调用的深度; 4) 状态变化 (State Change), 即递归算法作用对象的变化情况; 5) 收敛条件 (Convergence Condition), 即递归条件的终止情况, 确保递归能够正常结束。

基于上述算法安全设计基本原则, 本节提出云平台通用代码组件的算法逻辑漏洞原理模型如图 4-4 所示。一个不安全的递归调用具有以下基本特征: 1) 该函数本身是一个直接递归或者间接递归函数; 2) 该递归调用缺乏递归深度限制, 如缺乏递归收敛条件的声明、缺乏递归深度的记录、缺乏状态变化或终止条件的判断等, 上述信息的缺乏可能导致递归无法终止; 3) 缺乏异常或错误处理机制对递归流程的捕获, 证明开发者在算法设计过程中未考虑处理可能存在的算法逻辑异常, 证明可能出现的算法逻辑问题发生在预期之外, 其一旦出现便可被视为逻辑漏洞。

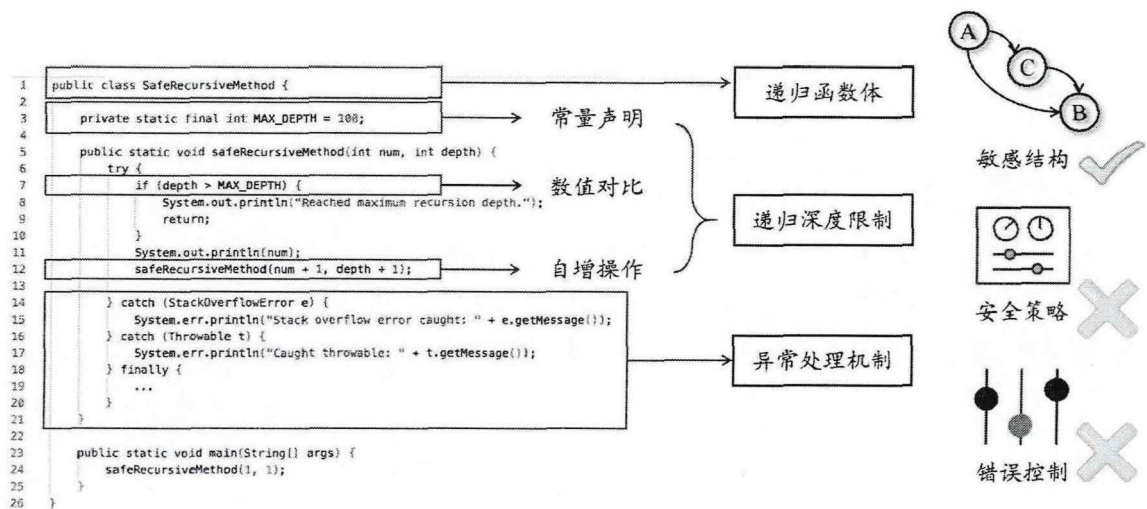


图 4-4 基于安全策略违背的算法逻辑漏洞原理模型

4.3 基于调用链及代码仓库搜索的算法逻辑漏洞分析方法

本节以上述算法逻辑漏洞模型为例，介绍针对该新型算法逻辑漏洞的检测方法。在充分理解算法逻辑漏洞发生原理和编程语言共通特性的基础上，本节提出一种新型空间型算法复杂度逻辑漏洞检测方法，主要步骤包括：1) 筛选敏感的算法结构，即针对算法复杂度逻辑漏洞的语言无关结构特性，提出类方法递归结构的分析与筛选方法；2) 判断是否存在安全策略缺失，即分析递归结构的算法实现，提出不安全终止条件的分析和判断方法，以发现可能存在算法逻辑漏洞的代码片段；3) 判断是否存在错误控制策略，即判断递归结构的调用上下文中是否存在对算法可能发生的异常和错误的捕获。

4.3.1 覆盖类方法递归的复杂递归结构分析方法

为实现代码的简洁性、运行效率和可维护性，Java 和 Python 语言均引入了类方法。类方法有递归和非递归两种设计方式，其中递归设计通过自我调用可实现用简单的代码处理大量的算法逻辑问题，然而每次递归调用都需要保存上下文信息，当递归深度过大时可能导致频繁的方法调用和栈操作，从而引发栈溢出算法逻辑漏洞。在调用链上，直接递归和间接递归具有以下特点：直接递归具有明显的自我调用特性，而间接递归可能通过复杂的间接调用实现自我调用。图 4-5 (a)显示了“methodB()”直接递归调用自身，而图 4-5 (b)显示了“methodA()”通过调用“methodB()”和“methodC()”间接递归调用自身。

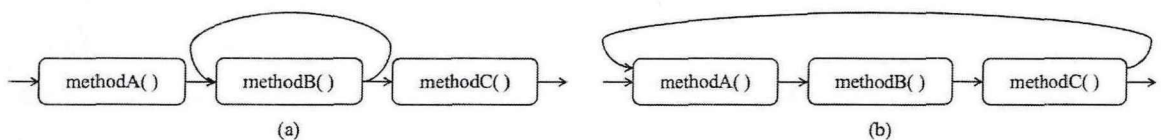


图 4-5 直接递归与间接递归结构特征，图 (a) 为直接递归，图 (b) 为间接递归

递归结构分析算法首先可以使用 Soot 框架实现，如算法 4-1 伪代码所述，首先从 JAR 文件获取所有 ActiveBody。对于 ActiveBody 中的每个单元，分析其调用类型（第 5 行），如果该调用是 InvokeStmt（调用语句）类型，或 AssignStmt（赋值语句）类型并且包含方法调用（第 6 行），则进行进一步分析：检查方法调用，如果它不在异常处理块内（第 8 行），则继续检查用于存储方法调用的栈；如果在栈中检测到递归调用（第 10 行）并且无递归深度限制（第 11 行），则记录当前调用栈为漏洞调用链。如果没有发现递归调用，则将非基本类方法和非抽象方法推入栈，并递归分析被调用的方法（第 16 行）。

算法 4-1 发现目标算法中的所有递归结构

输入： 测试目标项目的 JAR 文件	
输出： 可能存在漏洞的递归结构	
1.	利用 Soot 框架分析 JAR 文件，并得到所有 sootMethod 对象
2.	初始化 sootMethodStack 来存储 sootMethod
3.	body ← 检索 sootMethod 的所有 active body
4.	如果检索不到结果，则退出算法
5.	对于 body 中的每一个 unit:
6.	如果 unit 是 InvokeStmt 的实例或 AssignStmt 的实例且包含 InvokeExpr:
7.	calledMethod ← 提取 unit 中的被调用方法
8.	如果 calledMethod 在异常处理块中被捕获
9.	跳转至下一 unit
10.	如果 sootMethodStack 中包含 calledMethod
11.	如果 calledMethod 不包含递归深度限制
12.	记录当前的 sootMethodStack，用于之后的分析
13.	否则
14.	如果 calledMethod 既不是基本方法也不是抽象方法
15.	将 calledMethod 添加至 sootMethodStack
16.	递归地分析 calledMethod
17.	在分析完毕之后，从栈中移除该 calledMethod

4.3.2 预期外的不安全递归终止条件分析方法

其次判断递归结构是否存在不安全的递归终止条件。在筛选得到递归结构之后，紧接着对递归结构的终止条件进行分析：一旦未设置递归终止条件或递归终止条件无法触发，则算法有可能触发异常的资源消耗，进而发生拒绝服务。综上所述，递归终止条件应当具备如下性质：1) 该终止条件必须可达，即在一系列递归操作过后，递归必须终止；2) 该递归终止条件中不能进一步包含递归调用，

否则递归将可能不会终止。在软件开发实践中，通常使用两种方法来记录和判断递归深度：

- 1) 在与递归方法对应的类中定义一个类属性来记录递归深度。每次递归方法进入下一层时，增加该类属性以更新递归深度。在执行下一个递归方法之前，将该类属性与代表最大深度的常量进行比较，如果该类属性小于常量值，则递归方法可以进入下一层。
- 2) 在递归方法中设置一个变量作为参数来记录递归深度。每次方法递归到下一层时，增加该深度变量并将其传递到递归的下一层。在执行下一个递归方法之前，将该变量与代表最大深度的常量进行比较。如果该变量小于常量值，则递归方法可以进入下一层。

总而言之，安全的递归终止条件需要具备：1) 一个记录递归深度的变量，该变量随递归层级的加深同步增加；且 2) 该变量与一个常量值进行比较，只有当变量小于该常量值时，才允许递归进入下一层。因此，判断方法体是否包含递归深度限制的逻辑如下，具体如算法 4-2 所示，其中：

- 1) 确定递归方法中是否存在随递归层级增加而同步增加的变量（第 7 行）。
- 2) 判断该变量是递归方法中的形式参数还是直接或间接来自类属性（第 7 行）。
- 3) 检查该变量是否与一个常量值进行比较，并判断调用下一层递归是否在此判定逻辑的控制范围之内，即检查是否通过判定语句决定是否进入下一层递归调用（第 5 行）。

算法 4-2 判断方法体是否包含递归深度限制

输入： 用于被分析的方法的 body	
输出： 用于指示是否存在递归深度限制的 boolean 值	
1.	初始化 hasDepthLimit 值为 false
2.	对于方法 body 中的每一个 unit
3.	如果 unit 是 conditional 语句
4.	conditionalStatement ← 将 unit 转化为 conditional statement
5.	如果 conditionalStatement 是递归深度检查语句
6.	depthVar ← 提取 conditionalStatement 中的变量作为深度描述变量
7.	如果 depthVar 存在自增操作，并且是 class attribute 或参数引用
8.	hasDepthLimit ← true
9.	break
10.	返回 hasDepthLimit

4.3.3 未受控的异常和错误处理分析方法

最后,判定此可能存在逻辑漏洞隐患的代码是否在代码开发者预期范围之内。如前文所述,编程语言通过异常和错误处理机制实现对代码质量和安全性的控制,以保证系统的安全可靠运行。如对 Java 编程语言而言,如果一个递归调用没有被捕获“java.lang.throwable”、“java.lang.error”或“java.lang.StackOverflowError”等错误,则表明该算法代码可能存在未被开发者预见或处理的安全问题。因此当未捕获上述错误,特别是“StackOverflowError”时,这表明可能导致拒绝服务的算法逻辑漏洞未被开发者预期并处理,算法可能存在漏洞并可被攻击者利用。

通过利用 Soot 框架或前文所提出的逻辑漏洞语言无关型特征提取与表征技术来分析 Java 第三方库中所有 public 方法的相关调用链,可以清晰地识别程序中存在的递归路径(即,类方法直接或间接调用自身的情况)。如果这些递归方法在调用链上下文语句级别没有利用 try-catch 语句对“java.lang.throwable”、“java.lang.error”以及“java.lang.StackOverflowError”等错误进行捕获,这表明递归中的错误未被开发者预见和处理,而是在发生错误时被 JVM 抛出并运行,因此算法逻辑漏洞极有可能发生并被攻击者利用。

算法 4-3 判断递归结构是否被异常/错误处理机制捕获

输入: 方法的 body 以及用于被分析的是否存在逻辑漏洞的 sootMethod 对象	
输出: 用于指示该 sootMethod 是否在任何异常/错误处理机制 trap 中的 boolean 值	
1.	对于 body 中的每一个 trap (或称之为 block)
2.	startUnit ← 获取 trap 的初始单元, endUnit ← 获取 trap 的结束单元
3.	初始化 boolean 值 isInTrap ← false
4.	对于 body units 中的每一个 unit
5.	如果 unit = startUnit
6.	isInTrap ← true
7.	如果 isInTrap 的值为 true, 并且该 unit 包含到 sootMethod 的方法调用
8.	返回 true
9.	如果 unit = endUnit
10.	isInTrap ← false
11.	返回 false

如算法 4-3 所示,第 1 行遍历函数主体中的每个异常处理块(trap),并使用一个标志“isInTrap”来跟踪当前递归是否在任意 trap 内(第 4 行)。然后,遍历函数主体中的每个 unit(第 5 行),以检查目标疑似漏洞的方法是否被异常和错误处理块内的 unit 所调用(第 6-11 行)。此算法在 4.3.1 节提到的算法中被进一步使用。此外基于前文所提出的逻辑漏洞语言无关型特征提取与表征技术,可以对生成的图代码仓库进行查询,漏洞的原理公式如(4-1)所示:

$$C = \{P \mid P = [s \rightarrow F1 \rightarrow \dots \rightarrow Fn], \forall s \in \text{Inputs}, \quad (4-1)$$

$$\exists Fi \in \text{RecursiveFunctions}, \text{depth}(Fi) = \infty, \neg \text{caught}(Fi)\}$$

对于图代码仓库的查询语句，可以写为：

```
MATCH p=(m:Method)-[r:CALLS*]->(n:Method)
WHERE m = n AND NOT EXISTS {
    MATCH (m)-[:HAS_DEPTH_LIMIT]->()
} AND NOT EXISTS {
    MATCH (m)-[:CAUGHT_BY_EXCEPTION_HANDLING]->()
}
RETURN m, r, n
```

如此，便可轻而易举的查找到漏洞的调用链，如图 4-6 所示。

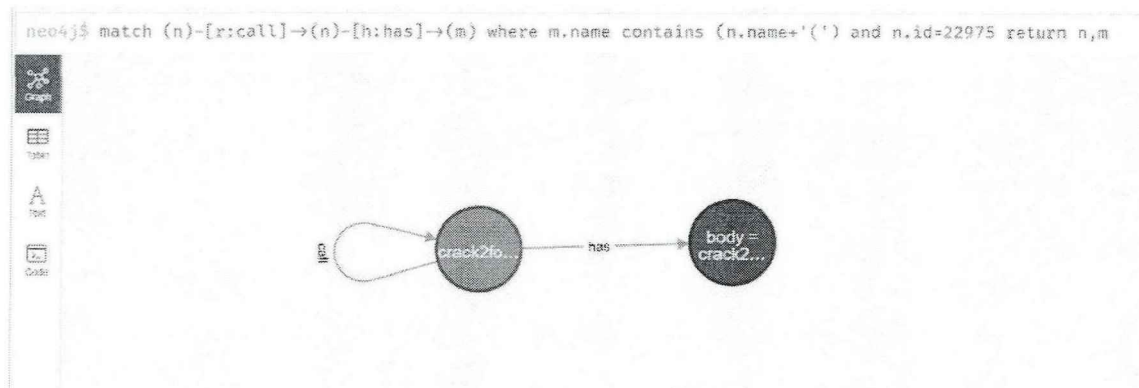


图 4-6 基于前文图形化代码仓库所找到的递归结构

4.4 基于递归结构逻辑理解的漏洞利用载荷构造方法

当前大多数自动化漏洞利用有效载荷生成方法依赖于符号执行和约束求解技术^[115,116]。然而，此类技术在 C 语言环境中更为成熟，而针对 Java、Python 等语言的符号执行技术仍处于发展和探索阶段，迫切需要可靠且高效的测试用例生成技术和工具以加速漏洞利用载荷的构造。目前，大多数符号执行工具只能支持基础且简单的数据类型如 int 和 char 的符号表示，但仍无法完全符号化如 string 和 object 等复杂数据类型^[117]，这一限制制约了在特定漏洞路径上使用符号执行进行约束求解的能力。

针对本章节所关注的目标漏洞类型，即空间型算法复杂度逻辑漏洞（Spatial Algorithm Complexity Vulnerability, S-ACV），本小节提出了一种半自动化、程序化、标准化的有效载荷生成方法，以支持快速的漏洞验证和利用。方法主要包括以下几个步骤：

- 1) 分析递归方法的参数处理逻辑。首先, 根据自动筛选的调用链, 识别并提取递归方法中处理参数的所有语句, 包括从递归方法开始直到参数传递到下一级递归调用的所有操作。
- 2) 构建初始实参。基于提取的参数处理逻辑, 生成应至少满足一次递归调用条件的初始实参。
- 3) 实参的自动迭代处理。将提取的参数处理逻辑应用于初始实参, 并根据编程语言共通特性迭代多次 (例如, 10,000 次迭代), 每次迭代模拟递归调用过程中参数的变化。
- 4) 生成最终攻击载荷。经过自动化迭代处理后, 获取实参的最终值作为 payload。该 payload 代表了经过多次递归分析后参数的最终状态, 可用于进一步的漏洞验证或利用。

4.5 实验评估

本章节最终实现原型工具 ACVFind。针对本章节所提出方法以及原型工具的实验评估主要从三方面进行: 一是对本方案的有效性进行评估, 评估本方案对于热门开源项目逻辑漏洞的挖掘能力; 二是评估本方案的优越性, 与业界最为知名、典型的逻辑漏洞挖掘工具进行对比, 评估本方案的效果及效率; 最后评估本方案对于漏洞验证与利用阶段的辅助作用, 判断基于本方案能否实现更快速的漏洞验证、利用及修复。

本节所有实验数据均来源于开源代码仓库中的热门组件。由于已公开研究的测试数据集大多为闭源数据, 因此本节选择开源代码仓库中的热门项目对本章所提出的漏洞挖掘方法进行验证, 所选择的开源项目包括 Maven 仓库和 GitHub 中的高星项目, 如 org.apache.pdfbox (版本 2.0.27)、org.apache.sling.commons.json (版本 2.0.20)、Jettison (版本 1.5.1)、Hutool-json (版本 5.8.10)、Ini4j (版本 0.5.2)。所有实验均在 Ubuntu 16.04、32 GB 内存、4 vCPUs、128 GB 磁盘空间的环境下进行, 使用的 JDK (Java 开发工具包) 版本为 1.8。

4.5.1 对于知名开源项目的逻辑漏洞挖掘能力

目前, 相关研究领域大多采用 DARPA Space/Time Analysis for Cybersecurity (STAC) 数据集作为测试验证数据集, 但现有研究存在以下局限性: STAC 数据集现已无官方渠道公开, 大部分数据都是经过相关研究人员处理并归档之后的版本; 相关研究大多不公开源代码, 对比实验的复现存在困难。因此, 针对本研究问题, 本小节主要针对开源代码仓库中的热门云平台通用代码层组件进行测试, 以评价本方案挖掘逻辑漏洞的能力。

本小节主要针对 Maven 仓库中的热门项目进行分析，包括 Web Assets 组件 hutool-json 等，Configuration Lib 组件 ini4j 等，PDF（Portable Document Format）相关处理组件 org.apache.pdfbox、JSON（JavaScript Object Notation）相关处理组件 Jettison、org.apache.sling.commons.json 等，所测试的组件均在行业内具有较大影响力，并被广泛应用，所测试的组件版本均为实验实施阶段可找到的最新版本。

表 4-1 本方案对开源代码仓库中热门项目的算法逻辑漏洞检测结果

待测软件	Stars	Forks	Usage	类总数	漏洞路径	路径筛选
Jettison	46	28	1017	47	27	6
hutool-json	27.2k	7.3k	95	74	30	12
ini4j	-	-	192	70	21	2
org.apache.sling.commons.json	-	-	250	45	26	4
org.apache.pdfbox	-	-	692	862	226	48

实验结果如表 4-1 所示。待测软件项目在 GitHub 上获得的 Star/Fork 的数量与 Maven Repository 上获得的 Usage 数量并不呈正相关的原因在于：部分组件通过间接依赖被下游使用，而下游组件的广泛使用并不会增加直接依赖 Usage 的数量；同时部分组件在 Maven 上被广泛引用，开发者无需在 GitHub 上下载代码。第 5 列“类总数”代表了相关组件总共包含的类的数量，第 6 列“漏洞路径”代表本工具总共找到的符合所提出漏洞模型的调用路径总数，第 7 列“路径筛选”代表了外部可访问的路径的数量（即可被攻击者外部触发）。

以 hutool-json 为例，针对该项目利用本工具 ACVFind 总共发现 74 个类，并筛选出 30 条符合漏洞模型定义的调用路径，即存在未限制终止条件的递归调用结构，且该结构的安全问题未被开发者预期并利用异常及错误处理机制捕获。对于这 30 条路径，本文认为其均存在算法逻辑漏洞，但由于其中 18 条路径并非外部可达方法或常用方法，因此将这 18 条路径排除在外，并认为剩余的 12 条路径是真实外部可达的，最后通过构造样本并提交 CVE 机构，并获得官方发布的漏洞编号 3 个（由于官方认为部分路径原理类似，因此对于同类漏洞只授予单个漏洞编号），包括 CVE-2022-45688、CVE-2022-45689、CVE-2022-45690 等。针对上述开源代码组件本文发现大量漏洞，并获得 8 个 CVE 编号，如表 4-2 所示：

表 4-2 本方案挖掘部分 CVE 漏洞及威胁级别

组件名称	漏洞编号	漏洞评级	组件名称	漏洞编号	漏洞评级
org.ini4j	CVE-2022-41404	高危，7.5	hutool-json	CVE-2022-45689	高危，7.5
Jettison	CVE-2022-45685	高危，7.5	hutool-json	CVE-2022-45690	高危，7.5
org.json	CVE-2022-45687	被保留	Jettison	CVE-2022-45693	高危，7.5
hutool-json	CVE-2022-45688	高危，7.5	Apache Sling	CVE-2022-47937	超高危，9.8

4.5.2 与业界先进研究的功能及性能对比

本章节所提出的方案的先进性和优越性主要体现在功能和性能两个角度。从功能的角度进行分析，本方法的有效性主要体现在以下几个方面：1) 本方法能够对 JAR 文件进行详细分析，提取所有类和方法，无需操作源代码。2) 本方法能够对 JAR 文件的调用链进行全面分析，筛选出所有与本章节所提出的漏洞模型相关的潜在漏洞路径；此外，由于本章节所提出模型的多重规则约束，所有识别出的潜在漏洞实际上都可被认定为安全问题；唯一的不确定性在于漏洞利用链的外部可利用性和漏洞原理结构的相似性等方面。3) 所有发现的路径都经过验证或被官方认可为漏洞。

表 4-3 本工具 ACVFind 与业界先进工具的对比

工具	漏洞	分析对象	方法	分析结果	Payload 生成	数据类型
Acquirer ^[63]	T-ACV	源代码	混合	可疑循环	符号执行	基本类型
DISCOVER ^[57]	循环相关	字节码/源代码	静态	循环视图	人工	-
Badger ^[62]	S/T-ACV	字节码	混合	覆盖率提升	Fuzz&符号执行	基本类型
HotFuzz ^[59]	T-ACV	字节码	动态	-	Fuzz&代码插桩	复杂类型
Revealer ^[61]	T-ACV	源代码	混合	可疑漏洞	动态	基本类型
SlowFuzz ^[118]	T-ACV	资源使用信息	动态	-	动态	基本类型
ACVFind	S-ACV	字节码	静态	漏洞路径	基于调用链	复杂类型

如表 4-3 所示，本方法和工具的优越性主要通过与其他工具的对比来展示。本小节选择了近年来最具代表性的研究成果，从可分析的漏洞类型、分析对象、分析方法、静态分析阶段产生的分析结果、payload 生成方式、以及支持的数据类型等多个角度进行对比，以分析本工具的先进性。定性分析结果表明，本章所提出的方案聚焦于新型空间型算法复杂度逻辑漏洞（Spacial ACV, S-ACV），而现有研究大多关注时间型算法逻辑漏洞（Temporal ACV, T-ACV）；所分析的对象不局限于源代码，仅需通过字节码便可实现分析，同时无需插桩修改源代码或字节码；所采用的方式为静态方式，所需的资源开销和时间开销更小；所得到的结果几乎可确定为实际存在的漏洞（截至目前尚未发生误报）；检测结果同时能够有效指导漏洞 payload 的构造，实现快速的漏洞复现与利用；支持复杂类方法递归分析，而现有工具仅支持基本类型分析。

对于性能分析，由于上述工具大部分未开源且未提供标准化复现流程，因此主要将本工具与知名 Java 模糊测试工具 juzzer 以及 Google oss-fuzz 工具（GitHub 9.4k Stars）进行对比。juzzer 以及 oss-fuzz 工具支持针对 Java 语言源代码的模糊测试，以发现空间型算法复杂度逻辑漏洞。由于上述工具需要对源代码进行操作，且需指定入口方法作为模糊测试的入口，因此实验环节对本工具 ACVFind 进同样设置，以保证实验的公平性。实验环节所测试的目标函数入口包括 hutool-json

组件的 `cn.hutool.json.JSONObject.<init>` 方法和 `cn.hutool.json.XML.toJSONObject` 方法，以及 `org.ini4j` 组件的 `org.ini4j.BasicProfile::fetch` 方法。

表 4-4 本工具与 oss-fuzz、jazzer 工具的详细性能对比

入口方法	工具	检测结果	用例数	时间	内存
cn.hutool.json. JSONObject	ACVFind	JSONObject.<init> → ... → .<init>	-	2.5s	98 Mb
	oss-fuzz	-	89,679,747	2h	1212 Mb
	jazzer	-	2,697,140	2h	2194 Mb
cn.hutool.json. XML. toJSONObject	ACVFind	JSONXMLParser.parse → .parse	-	2.4s	88 Mb
	oss-fuzz	-	56,007,748	2h	936 Mb
	jazzer	-	93,650,970	2h	951 Mb
org.ini4j. BasicProfile:: fetch	ACVFind	BasicOptionMap.fetch → ... → .fetch	-	3.1s	66 Mb
	oss-fuzz	-	77,483,648	2h	1026 Mb
	jazzer	-	67,108,864	2h	928 Mb

实验结果如表 4-4 所示，表格列从左到右分别展示了入口方法、工具、测试结果、生成的测试用例数量、时间消耗和平均内存消耗。结果表明，ACVFind 能够准确识别完整漏洞调用链，例如对于入口方法 `cn.hutool.json.JSONObject`，发现调用链：`cn.hutool.json.JSONObject.<init> → ... → cn.hutool.json.JSONObject.<init>`（由于空间限制，表格对“检测结果”一列中的调用链表示进行了简化描述）；而 `oss-fuzz` 的模糊测试引擎 `libfuzzer` 以及 `jazzer`，尽管在有限的时间内生成了大量测试样本（在 2 小时内分别生成 89,679,747 个和 2,697,140 个测试样本），但仍然未能检测到此漏洞。这是由于算法逻辑漏洞与算法逻辑的设计和实现密切相关，而 `oss-fuzz` 及 `jazzer` 采用的是随机种子生成方法，无法触发深层递归逻辑；同时这些工具无法处理复杂对象类型，导致覆盖率低，进而无法发现目标漏洞。

在对比过程中，静态方法可能在效率方面与动态方法相比具有天然的优势，因此将本工具与 `oss-fuzz` 及 `jazzer` 进行比较可能并不完全公平。然而该实验结果旨在展示，对于与特定程序设计逻辑紧密相关的算法逻辑漏洞，本章节基于对算法设计逻辑与编程语言共通特性的理解所提出的模型不仅效率极高、对于时间和内存的消耗极低，而且检测到的调用链几乎都是真实漏洞（到目前为止还没有发现误报），可以大大降低分析人员的复查成本，利于快速发现和验证漏洞。同时实验结果也旨在表明，现有的动态模糊测试工具应更加重视测试程序逻辑，从而提升对程序逻辑的测试效果及效率。

4.5.3 对于漏洞验证与利用的辅助能力

如前所述，本章节提出的漏洞分析方法和检测模型能够更高效、更准确地检测与递归结构相关的漏洞。这是因为本工具筛选出的漏洞链满足以下特征：它们有可能直接或间接触发无休止的递归，可以从外部访问，并且不被异常和错误处

理机制捕获。因此，基于对递归出发条件和参数处理上下文的理解，可以快速构造针对此类漏洞的攻击载荷。

```

1  public class JSONObject implements Serializable {
2      //
3      public JSONObject(JSONTokenizer x) throws JSONException {
4          this();
5          char c;
6          String key;
7
8          if (x.nextClean() != '{') {
9              throw x.syntaxError("A JSONObject text must begin with '{'");
10         }
11
12         for (;;) {
13             c = x.nextClean();
14             switch (c) {
15                 case 0:
16                     throw x.syntaxError("A JSONObject text must end with '}'");
17                 case '}':
18                     return;
19                 case '{':
20                     throw x.syntaxError("Expected a key");
21                 default:
22                     x.back();
23                     key = x.nextValue().toString();
24             }
25         }
26     }
27 }
28
29 public class JSONTokenizer {
30     //
31     public Object nextValue() throws JSONException {
32         char c = nextClean();
33         switch (c) {
34             case '"':
35             case '\\':
36                 return nextString(c);
37             case '{':
38                 back();
39                 return new JSONObject(this);
40             case '[':
41                 back();
42                 return new JSONArray(this);
43         }
44     }
45 }

```

图 4-7 以本方案所挖掘的 CVE-2022-45685 漏洞为例进行漏洞利用载荷构造示例说明

本小节基于所提出递归结构引导的漏洞利用载荷构造方法，以本文所公开披露的漏洞 CVE-2022-45685 为例，对攻击 payload 的快速构建过程进行展示。图 4-7 所示为漏洞源代码，本工具 ACVFind 基于所提出的漏洞模型找到一条间接递归的调用链，即 `JSONObject() → nextValue() → JSONObject`，同时该调用链不存在递归深度限制，且不被异常处理机制捕获。首先查找传入递归方法的参数，即第 3 行 `JSONTokenizer` 输入字符串游标；进一步检查递归结构中从 `JSONObject` 到下一个新的 `JSONObject` 的所有函数语句，分析递归方法参数处理的相关逻辑，包含对 `JSONTokenizer` 字符串游标的所有处理语句，如第 8 - 22 行和第 32 - 42 行

所示；进一步基于相关处理语句构建可以触发递归循环的初始实参，即“{1}”；最后，根据 Java 函数递归的特性对该初始实参进行迭代处理，自动化构造可以触发超过栈深度限制的参数，最终输出即为最终的攻击 payload。

4.6 本章小结

云平台代码层通用组件为开发者加速业务与服务开发提供便利，但由于开发者能力不同以及第三方代码仓库被攻击者恶意投毒，这些组件可能存在安全隐患，并为云平台运营者和用户带来安全和可用性威胁。作为其中一种最为典型的代码层组件的逻辑漏洞，算法逻辑漏洞在各种应用程序和编程语言中广泛存在。领域内相关研究主要集中在基于模糊测试的算法复杂度逻辑漏洞检测上，但由于现有研究主要基于对程序的有限理解进行测试样本构造，并通过随机生成并变异样本实现对目标组件的测试，因此难以深入测试程序算法逻辑，导致漏洞挖掘的效率相对低下。本章节提出一种基于算法调用结构及编程语言共通特性的新型算法逻辑漏洞模型构造方法，揭示该逻辑漏洞的语言无关型成因本质；进一步提出基于该模型的代码调用链搜索方法，高效地发现存在漏洞的调用路径；最终提出基于递归结构理解的利用载荷构造方法，实现漏洞的快速复现验证，成功挖掘多项 0 day 未公开漏洞并获得 8 项 CVE 编号。

与现有方法相比，本工具具有以下优势：漏洞模型更加精确，能够准确识别与递归结构相关的空间型算法复杂度逻辑漏洞；在特定漏洞类型上具有更高的效率和准确性，几乎没有误报；支持快速漏洞验证和利用，分析过程更贴近逻辑漏洞发生的本质。本章节的研究仍然存在一定的局限性，包括：首先，本文尚未充分挖掘所提出的漏洞调用结构发现方法的潜力，目前仅构建针对递归结构的算法逻辑漏洞检测模型，未来可充分发挥框架底座的优势，构建更多模型来发现更多类型的算法逻辑漏洞的利用模式，同时也包括其他类型的代码层逻辑漏洞；其次，受限于动态符号执行工具的发展水平，本章所提出的漏洞利用载荷生成方法目前仍然主要采用半自动化的方式，需要部分手动干预，未来可考虑研究全自动的攻击载荷生成，以进一步提高漏洞验证和利用的效率。最后，可继续深入研究不同编程语言之间相同类型的漏洞检测、验证与利用方法^[119]，利用统一的中间表示来检测更多类型的算法逻辑漏洞以及其他与程序逻辑相关的代码层逻辑漏洞。

第五章 基于动态程序逻辑推演的业务层逻辑漏洞挖掘

云平台业务层通用组件即用于构建云平台关键资源管理业务的各种通用软件及框架,包括 API 网关、虚拟资源编排与管理系统等。云平台向用户提供服务托管业务,用户可以以更高的性能和更低的开销来维护自身的业务、访问云平台厂商所提供的业务。通常情况下,云平台的运营及维护人员、云平台的用户通过各种预先定义的 API 接口来访问、管理并使用云平台所提供的资源。而为了实现接口的规范性和可维护性,通常需定义一种 API 接口的风格标准,其中最常用、最受欢迎且最规范的标准之一即是 REST 架构标准^[120]。REST 架构常被用于定义基于 HTTP 的 Web 服务接口,在此架构下所有云上服务都被抽象为资源,并使用 URI (Uniform Resource Identifier) 进行唯一标识。用户通过 HTTP 方法(如 GET, POST, PUT, DELETE 等)即可实现对资源的获取、创建、修改和删除等操作,并根据 HTTP 响应信息中的状态码来获知操作结果。

在此背景下,几乎所有的云平台业务逻辑都围绕资源来进行设计,几乎所有针对资源的潜在攻击行为都通过业务 API 接口展开。因此相关领域也将通过 REST API 进行云平台可靠性与安全性测试视为发现云服务问题的主流方法^[121, 122],包括微软提出的 RESTler 工具、EMResearch 开发的 EvoMaster 工具等,都对领域发展起到有益效果。然而这些基于 API 模糊测试设计理念开发的工具大多聚焦于单一测试样例,即针对 API 请求的部分字段进行变异测试,而未能深入分析测试云平台的业务逻辑,因此这些工具大多在检测服务端处理异常方面起到了更优秀的效果,而在检测漏洞方面,特别是检测具有复杂触发逻辑的业务相关逻辑漏洞方面效果有限,难以达到宣称的效果。

针对上述问题,本章节提出基于动态程序逻辑推演的业务层逻辑漏洞挖掘方法。具体而言,首先提出一种针对云平台业务层通用组件的逻辑信息获取及处理方法,提出基于云服务 API 文档的测试组归并划分方法,提出云服务运行时逻辑信息的追踪获取方法,以从静态和动态信息的角度分析云平台业务逻辑;其次,提出基于参数推断的云平台 API 设计与实现不一致逻辑缺陷的检测与缓解方法,并提出基于模型推演的测试样本生成方法,构造满足 API 测试组约束、满足 API 请求约束、满足漏洞模型约束的测试样本,从而减少生成无效样本的概率,实现对目标 API 的深度测试;随后,提出基于稀疏调用栈匹配的漏洞检测方法,结合 HTTP 状态码及调用栈信息,通过构建合法或异常行为序列有限状态机,检测预期外的 API 调用结构;最后,实现原型工具 RESTLogic 并针对热门开源云平台业务层通用组件进行逻辑漏洞挖掘,发现权限管理型逻辑漏洞多项。

本章组织架构如下：5.1 节对问题的定义、威胁模型、设计目标以及方案总体框架进行阐述；5.2、5.3、5.4 节分别介绍本章所提出的云平台业务层通用组件的逻辑信息获取及处理方法、基于模型推演以及参数推断的测试样本构造方法、基于稀疏调用栈匹配的漏洞检测方法；5.5 节对方案进行仿真验证，对热门开源业务层组件进行测试，并与业内先进研究进行对比；5.6 节对本章工作进行总结。

5.1 问题描述

本节主要介绍针对云平台通用组件的基于动态程序逻辑推演的业务层逻辑漏洞挖掘方法的有关概念定义、威胁模型、设计目标以及方法总体框架。

5.1.1 概念定义

REST API 设计最佳实践。REST API 旨在为 Web 服务的每个资源定义接口表示，而该表示方式在当今软件开发实践中大多基于 HTTP 实现，并使用 URI 作为资源标识符，用户可以根据 URI 显式地引用目标资源。微软所提出的 REST API 设计最佳实践^[123]鼓励将 URI 按照层次架构进行组织，例如在 OpenStack Compute 组件的 API 中，操作请求 GET /servers 用于获得服务器信息，而操作请求 GET /servers/{server_id} 用于进一步获取特定服务器的详细信息。而对于不同类型资源之间的关联关系，可以将相关资源放置在子路径中进行描述，例如，在 /servers/{server_id} 下的 URI /server/{server_id}/metadata 用于列出与特定服务器资源相关的元数据资源。简而言之，REST API 设计最佳实践引入了分层的 HTTP URI 组织范式，在此范式下与同一资源相关的 URI 通常共享相同的 URI 前缀。

REST API 的前缀归并树。前缀归并树，又称数字树或前缀树，是一种特殊的多叉树结构，主要被用于高效地在一组数据中查找特定的键（比如字符串）。不同于二叉搜索树，前缀归并树的节点并不直接存储与之关联的键值，而是通过节点在树中的位置来定义关联的键值，从而将每个键值的信息分布在整个数据结构中。当对字符串进行查找时，从树的顶端开始，按照字符串的每一个字符依次深入树的下一层，每走一步相当于匹配了字符串的一个字符，通过此方式可以快速定位到目标字符串。简而言之，前缀归并树是一种通过字符串的前缀进行快速查找的数据结构，而这一数据结构与 REST API 的最佳设计实践理念相契合：REST API 的前缀归并树中，相同资源将共享共同前缀；共同前缀越长，API 间的逻辑关联关系越紧密。

测试组。基于云平台业务层通用组件的 API 文档描述信息，可分析 API 的组织结构及逻辑关联关系，进一步基于逻辑关联关系划分 API 测试组。每个测试组包含一系列逻辑上相关联的 API，它们可能共同参与某个业务流程、实现对于某一资源的生命周期管理或在数据流上相互依赖。每一种测试组的 API 组织模

式相似,因此它们可能面临相同类型的逻辑问题,例如当一个测试组中的 API 组织结构可能面临资源权限管理方面的缺陷时,其他同一类测试组都可能面临同样的安全威胁。因此进一步的,每个测试组均可与一个或多个潜在的逻辑漏洞测试模型相关联,在这些测试组上可实现漏洞测试模型的实例化、具象化。

云平台业务逻辑的外在表达。针对云平台业务层通用组件进行逻辑漏洞挖掘应从可获得的云平台业务逻辑的外在表征信息入手,此类外在表征信息可包含静态信息,如 API 的业务层设计、API 的代码层实现、API 设计与代码实现的一致性关系等(由于 REST API 设计最佳实践主张优先设计 API 接口,然后编写实现接口的代码,因此 API 设计与代码实现的异步进行可能导致 API 设计与实现不一致情况的发生,即便同步进行也可能因开发者的主观能力因素而出现此问题);同时可包含动态信息,如运行时特征等,包括 API 调用逻辑、API 接口背后的函数调用逻辑、具体的调用参数及返回值信息等。

5.1.2 威胁模型

如前文所述,云平台业务层通用组件的设计以资源为核心,其访问接口遵循 REST 风格范式和 C/S (Client/Server) 架构,该风格范式的核心概念为“资源”,具有层级架构、统一接口、无状态、基于用户身份标识的权限验证等特点。因此基于云平台漏洞攻击面先验知识与云平台架构设计范式,本章节提出对应的威胁模型如图 5-1 所示。假定用户 A 具有资源 1-1 及其子资源的权限、用户 B 具有资源 2-1 及其子资源的权限,那么曲线 ❶ 代表一条合法的资源访问请求。具体的威胁模型如下所示:

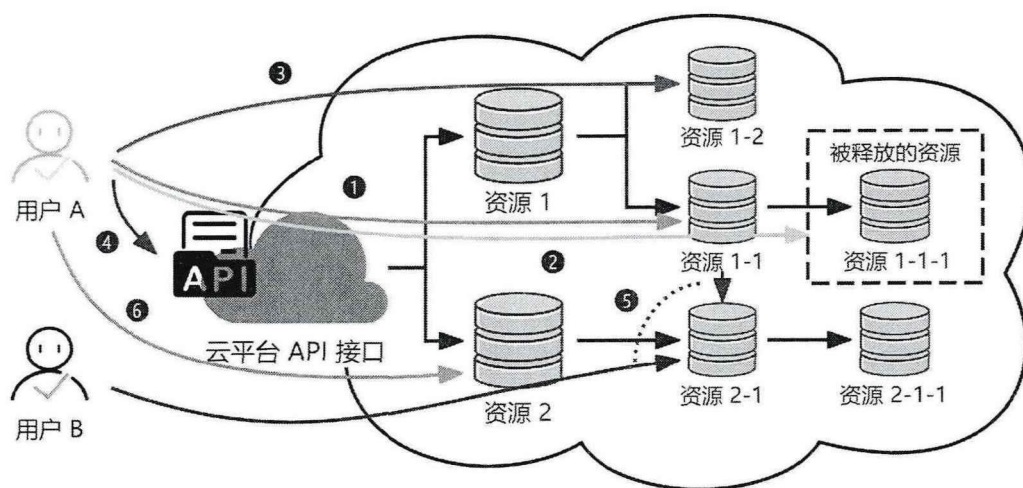


图 5-1 云平台业务层通用组件的逻辑漏洞威胁模型

- 1) 针对资源的威胁。此类威胁主要关注在特定情境下,单一用户对单一资源的访问和操作逻辑可能存在的安全问题。例如曲线 ❷,访问已经释放的资源(比如用户 A 访问已释放的资源 1-1-1)可能导致资源配额绕过或信息泄露,从而威胁其他用户资源的可用性和隐私安全。

- 2) 针对层级架构的威胁。层级架构泛指服务或资源之间的父子关联关系，层级架构的安全缺陷可能导致服务或资源之间发生权限隔离问题。如曲线 ③ 所示，成功访问无权限的父资源或同一父资源下的其他资源（例如用户 A 访问资源 1-2 及其子资源）都是此类缺陷的表现形式。
- 3) 针对统一接口的威胁。REST 架构使用 HTTP 方法为用户提供通过 API 创建、读取、更新和删除（CRUD）资源的能力，所有对资源的操作均需通过 HTTP 方法完成。如曲线 ④ 所示，此类威胁可包括用户请求导致的服务端处理错误。
- 4) 针对无状态性的威胁。无状态性，即所有资源均可通过单独的 URI 进行定位，并且不会因其他资源的变化而变化（也称为幂等性）。因此，不当的隔离机制会导致资源受其他资源变化的影响而发生改变（例如曲线 ⑤ 所示，资源 2-1 受资源 1-1 变化的影响而随之发生意外变化）。
- 5) 针对权限验证的威胁。当用户请求访问特定资源时，通常需要向服务器发送自己的令牌以验证身份及权限。一旦权限验证机制存在漏洞，攻击者可能会伪造令牌绕过验证并任意篡改其他用户的资源（例如曲线 ⑥ 所示，用户 A 通过发送伪造的令牌直接访问和篡改资源 2）。

5.1.3 设计目标

综上所述，测试云平台业务层通用组件的逻辑漏洞通常面临如下挑战：

- 1) 复杂的 API 逻辑。在实际应用场景中，访问 REST 服务通常需要多个 API 的协作支持，这表明对单一逻辑流进行测试可能需要考虑多 API 之间的组合。不同的 API 组合模式可能面临不同的安全威胁，例如“在同一资源上先进行 POST 操作后进行 DELETE 操作”可能面临权限绕过威胁，“在同一资源先进行 DELETE 操作后进行 GET 操作”可能面临资源释放后使用威胁。API 组合模式揭示了资源的生命周期，而“资源”是 REST 架构的核心概念，但此理念在大多数现有研究中很少被考虑。
- 2) 代码与文档的不一致问题。不一致问题是 REST 服务测试过程中面临的另一个关键问题。此问题可能仅在测试过程中发生，而不会对正常应用功能产生不利影响，因此许多现有研究并未对此问题进行考虑。包括请求定义中缺少参数、错误的 URI 路径、参数类型与代码不匹配以及不同 API 之间的参数名称不匹配等不一致问题都可能成为自动化测试的障碍。这些问题可能影响所生成测试用例的质量，进而导致安全分析结果的可靠性降低。
- 3) 分析维度不足。以云平台业务层为代表的通用 REST 服务以黑盒方式通过 API 接口向用户提供服务，这在一定程度上阻碍了安全分析人员对业

务逻辑问题的深入评估。现有测试方法主要关注外部可用信息，如 HTTP 响应（response）的响应头和响应体，无法发现部分看似正常的 HTTP 响应背后可能蕴含逻辑问题。运行时中间信息作为 API 逻辑的外部表达，同样可能揭示逻辑执行的异常，因此可从此视角扩展分析纬度。

因此围绕上述挑战，本章的设计目标主要包括：

- 1) 有效性。随机生成的无效样本可能在测试的早期阶段被丢弃，同样可能无法有效发现逻辑漏洞。面向测试组的基于资源生命周期的样本构造方法旨在生成尽可能不被系统丢弃的测试样本序列，进而测试逻辑问题。
- 2) 覆盖率。覆盖率是另一个影响测试效果的因素，覆盖率的减少可能受到许多因素的影响，如对 API 组合模式的理解不足或代码与文档的不一致问题，通过测试组划分和参数推断可实现针对目标测试组的深度测试。
- 3) 能力。能力指本方案对安全缺陷及逻辑漏洞的检测能力。为了扩展分析维度，本章引入运行时中间逻辑信息分析作为分析维度补充，从而提高检测安全缺陷的可能性。

5.1.4 总体框架

基于本章所提出的概念定义、威胁模型以及设计目标，本小节提出针对云平台业务层通用组件逻辑漏洞的总体测试框架。如图 5-2 所示，该框架主要包括三部分，对应本章的三部分研究内容，即 REST API 相关逻辑信息的获取与处理、基于模型推演与参数推断增强的测试样本生成、基于 API 调用栈辅助的测试结果分析。其中每一部分研究内容的结果为后续部分提供支持，最终实现对于云平台业务层通用组件的逻辑漏洞测试与分析。

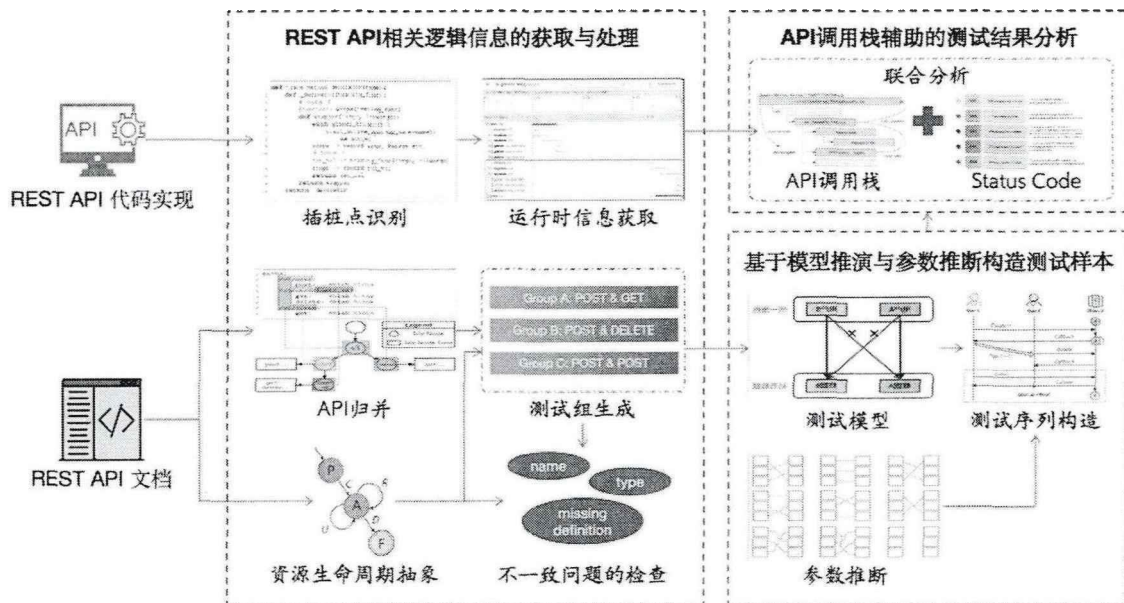


图 5-2 本方案的总体架构

REST API 逻辑信息的获取与处理。此部分的设计目标在于分析 REST API 逻辑的外部表达,并为样本生成和结果分析提供翔实依据。对于静态逻辑信息,本章通过分析 API 文档规范揭示 API 之间可能的逻辑关系,并为可能面临同类安全问题的 API 集合划分测试组。同时,本章对资源生命周期进行抽象,并基于资源属性和测试组信息检查并缓解不一致问题以提高测试覆盖深度。此外,本章提供动态运行时逻辑信息获取方法,以获取 API 运行时的逻辑特征。

基于模型推演与参数推断增强测试样本生成。首先,基于资源生命周期模型、测试组信息以及相关的漏洞模型构造测试用例,测试用例的表现形式为具有依赖或约束关系的 API 请求序列。为解决当前由于 API 设计与实现不一致问题导致的业务逻辑测试深度不足的问题,提出参数推断方案,旨在在样例构造阶段提供参数模板的填充依据,所推断的参数可以有效缓解由于不一致问题导致的依赖分析准确度降低的问题,以进一步提高所生成样本的测试覆盖深度。

基于 API 调用栈辅助的测试结果分析。传统基于 HTTP 响应状态码的测试结果分析通常直观且有效,但也存在一定的局限性,即前文所提到的“无法发现看似正常的响应背后存在的逻辑问题”。因此,从运行时信息中获得的 API 调用栈信息可作为业务逻辑的外在表现而辅助进行测试结果分析,从而扩展分析视角。通过对调用栈结构进行分析,可实现对正常或异常运行时逻辑特征的检测,并结合 HTTP 响应信息进行联合分析以实现深入、多维度的逻辑漏洞检测。

5.2 云平台通用组件 REST API 相关逻辑信息的获取与处理方法

为实现基于动态程序逻辑推演的业务层逻辑漏洞挖掘,首先应获取并处理可获得的逻辑信息。同传统漏洞挖掘获取信息的角度相同,逻辑漏洞挖掘可获得的逻辑信息同样包括静态信息与动态信息,其中静态信息包括云平台通用业务层组件的访问接口信息,进一步的可包括访问接口 API 文档与访问接口的代码实现,以及更深入的接口文档与代码实现的逻辑一致性问题;动态信息可包括 REST API 在处理用户请求过程中的函数调用逻辑。因此本节主要针对上述内容进行研究,提出云平台通用组件 REST API 相关逻辑信息的获取与处理方法。

5.2.1 基于文档信息的 API 归并与测试组生成方法

目前,针对云平台通用组件 REST API 的模糊测试技术主要通过提升测试代码的覆盖率来提升检出程序缺陷或漏洞的概率,并在传统内存破坏型漏洞上展示出了有益效果。然而由于业务逻辑的 API 组合的复杂性,API 之间存在复杂的依赖与调用关系,仅仅提升对于代码的覆盖率难以实现对复杂业务逻辑的全面测试。因此,本章节从一种最为流行的 REST 服务 API 文档规范 OpenAPI Specification

(OAS)^[124]入手,通过对 API 接口逻辑进行理解实现对 API 组合的覆盖测试,以达到更高的针对 API 逻辑测试的覆盖率,从而发现更多的业务逻辑问题。

在 REST 架构下,所有资源类型的实例都存在生命周期,围绕资源生命周期模型划分测试组能够实现同类问题的快速测试。例如在云平台的块存储服务中,整个业务实例的生命周期包括创建状态、可用状态、结束状态:块存储资源首先被创建并分配给用户,接着用户通过读取操作访问该资源,随后用户通过更新操作对资源进行修改,并最终在服务到期或用户指定情况下,该资源被删除。因此,可对所有资源的生命周期利用有限状态机(Finite State Machine, FSM)进行描述,即 $M(\Sigma, S, s_0, \delta, F)$,其中:

- 1) Σ 是输入方法集,即与资源类型相关的生命周期函数集,对于云平台业务层而言包括如{Create, Read, Update, Delete};
- 2) S 是一个有限的非空状态集,在本章所提到的资源生命周期模型中, $S = \{\text{Pre - Creation, Available, Finalized}\}$;
- 3) s_0 为有限状态机的初始状态,即资源的初始状态,为集合 S 中的元素,在本章中,对于大部分资源而言 $s_0 = \text{Pre - Creation}$,即待创建状态;
- 4) δ 为状态转移函数,即 $\delta: S \times \Sigma \rightarrow S$,实现资源状态在状态集合之间的转移,如表格 5-1 所示;
- 5) F 为最终状态的集合,为集合 S 中的子集,即 $F \subseteq S$,在本章中,对于大部分资源而言 $F = \{\text{Finalized}\}$ 。

表 5-1 资源生命周期模型及对应的状态转移函数

FSM 状态转移图	输入方法集中的元素	对应的 HTTP 方法 ^[125, 126]
<pre> graph TD P((P)) -- C --> A((A)) A -- R --> A A -- U --> A A -- D --> F(((F))) </pre>	Creation	POST, PUT
	Read	GET, HEAD
	Update	PUT, PATCH
	Delete	DELETE

如表 5-1 所示,其中第一列“FSM 状态转移图”列出了一个资源实例的生命转移周期,即:资源实例最初处于 Pre-Creation 预创建状态,并通过输入方法集中的元素 Creation 操作(对应 HTTP 定义的 POST 和 PUT 等方法)将自身状态转移至 Available 即可用状态,在此状态下,资源实例可被用户访问;当资源实例处于 Available 状态时,用户可通过 Read 和 Update 等操作,实现对资源实例的读写操作,以实现对资源的操作和维护,在上述操作下资源实例始终处于 Available 状态;最终,当用户不再需要使用此资源时,通过 Delete 操作(对应 HTTP DELETE 方法)实现对资源的删除,资源实例进入 Finalized 状态。

基于资源生命周期模型,对用于描述同一资源的 API 进行分组与归并,可为基于模型的测试样本序列生成提供依据。由于 REST 服务的标准化 API 文档中定义了对资源操作的基本描述并利用 URI 对资源进行标识,同时在 OAS 标准下相同的资源通常具备相似的 URI 前缀,因此基于资源生命周期概念以及标准化文档信息,借助前缀归并树概念可对 API 信息进行归并,从而将庞杂的 API 文档归并为若干测试组,进一步为基于模型的测试样本生成提供依据。

REST API URI 的层级架构表明,越相似的 API URI 结构内在逻辑的相似性越高,其背后蕴含的逻辑问题更可能存在相似性,因此可基于此原则对 API 进行分组与归并。基于概念定义章节所描述的前缀归并树与 REST API 设计最佳实践,可以将 URI 路径中的每一个片段(segment)作为树形结构的属性存储于树形结构的节点中,具备共同前缀路径的 URI 被归并至同一树形分支结构,从根节点进行深度遍历即可获得完整的 URI 路径,具体如算法 5-1 所示:

算法 5-1 基于 REST API 文档构建 URI 树形结构的算法

输入: REST API 的 OAS 规范文档 oas_doc	
输出: URI 树形结构的根节点 root	
1.	初始化 path $\leftarrow$ 获取 oas_doc 中的 path 字段
2.	初始化 root $\leftarrow$ {
3.	segment $\leftarrow$ “”, 用于将路径 segment 存储于树形结构的中间节点
4.	api_unit $\leftarrow$ null, 用于将 API description 存储于树形结构的叶子节点
5.	sub $\leftarrow$ {}, 用于存储孩子节点的 map
6.	}
7.	对于所获取的 path 中的每一条 URI 路径 p:
8.	key_split $\leftarrow$ 利用 “/” 符号分割 URI 路径
9.	如果 key_split.length() > 1, 则弹出 key_split 最顶上的空字符串
10.	如果根节点的孩子节点不包含 key_split 中的第一个片段 key_split[0], 则:
11.	定义当前节点:
12.	current_node $\leftarrow$ {segment $\leftarrow$ key_split[0], api_unit $\leftarrow$ null, sub $\leftarrow$ {}}
13.	将 key_split[0] 以及当前节点 current_node 传递给 root 节点
14.	否则, 取出名为该 key_split[0] 的根节点的孩子节点, 作为 current_node
15.	依次弹出 key_split 中的每一个值 k, 并:
16.	创建临时节点变量 tmp_node $\leftarrow$ {segment $\leftarrow$ k, api_unit $\leftarrow$ null, sub $\leftarrow$ {}}
17.	如果当前节点的孩子节点中不包含 k:
18.	则将 k 值以及 tmp_node 存入当前节点的孩子节点中
19.	current_node $\leftarrow$ 将当前节点 current_node 的孩子节点 k 传递给当前节点
20.	将路径 p 的详细调用参数描述存储于 current_node 的 api_unit 字段

如算法 5-1 所示，其中算法第 8 行对 URI 路径进行处理，按照 “/” 字符对 URI 路径进行分隔；对于每一个路径片段，分析其是否在父节点的孩子节点中，如果不存在则将片段添加为一个孩子节点（第 19 行）；对于每一个 API path，将其 description 字段存储于 path 的最后一个 segment 相关节点的 api_unit 字段中作为该 API 的描述信息（第 21 行）。

图 5-3 对树形结构的构造方式进行示例说明。在如图所示的 OAS 文档中，path 字段下存储三条 URI 路径，分别为路径 1: /v3/user，路径 2: /v3/user/{user_id}，路径 3: /v3/token。可以得知，三条路径的共同前缀节点为 v3 节点，而路径 1 与路径 2 额外存在一个共同前缀节点为 user，因此路径 1 与路径 2 的逻辑关联关系更加紧密，因为二者都是对资源 “user” 进行操作的相关 API 路径，而二者与路径 3 的关联关系相对较弱。因此基于本小节所提出的路径归并算法，可以将 API 归并为树形结构表示。进一步，基于此树形结构，可以进行测试组的过滤及筛选，测试组应具备的基本特征如下：

- 1) 所划分的 API 子树，必须包含非空叶子节点，即至少存在 1 项 API 操作；
- 2) API 子树的叶子节点至少包含 POST 方法或 PUT 方法，即至少存在资源的创建方法，以支持测试环境的初始化。

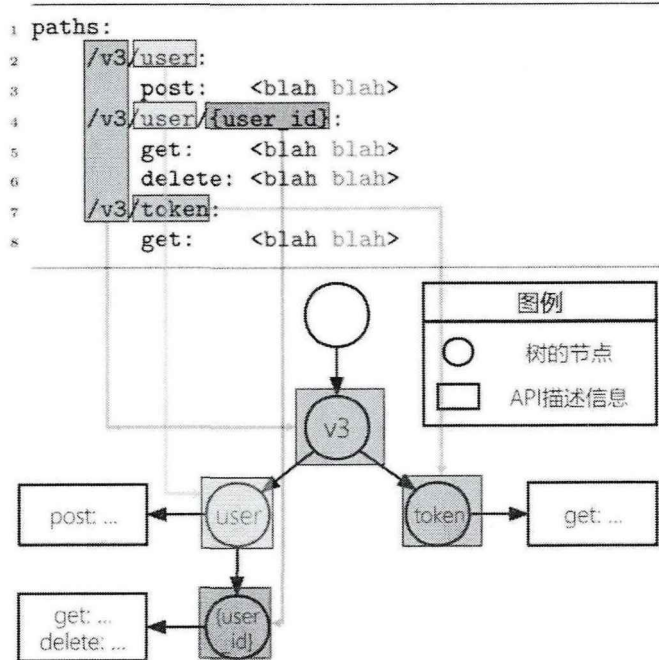


图 5-3 API 路径归并树的示例及其与资源的关联关系

尽管在资源生命周期概念中，几乎所有资源都存在 Pre-Creation、Available、Finalized 状态，但部分业务如特殊日志存储、区块链信息存储等业务的资源不应被删除，因此 DELETE 方法并非总会被实现。因此，基于所提出的测试组筛选基本原则，示例的测试组可以包括但不限于：1) 当存在对同一资源的 POST 或

PUT 请求时, 考虑是否存在越权可能; 2) 当存在对同一资源的 DELETE 请求和 GET 请求时, 考虑是否存在资源释放后使用可能。针对通过不同过滤原则筛选出的测试组, 可以基于模型推演构造动态程序逻辑测试样本, 以发现目标逻辑问题。

5.2.2 基于文档信息与 API 代码实现的不一致问题检测方法

API 的文档描述与代码实现的不一致问题广泛存在, 降低软件质量并制约软件测试的效率。由于文档编写与代码开发在软件工程实践中是两个相对独立的过程, 且由于项目管理不当或开发人员的能力水平不一致, 描述文档与代码实现不一致的问题广泛存在, API 文档并非总能够如实反映代码的实现方式与实现细节。本小节将此类问题定义为“代码与文档不一致问题”, 简称为“不一致问题”, 该问题在当今软件开发实践, 尤其是云平台通用组件等大型软件应用中广泛存在。典型的代码与文档不一致问题可以包括:

- 1) 不同 API 之间的参数名称不匹配: 操作同一资源的不同 API 可能会为相同的资源使用不同的名称。例如在创建资源的 API 中, 资源可能被命名为 `resource_id`, 而在访问该资源的 API 中, 相同的资源可能被命名为 `id`。此类问题定义为问题①。
- 2) 在请求定义中缺乏参数声明: API 文档可能没有完全声明所有需要的参数, 或者某些参数在文档中被遗漏。此问题可能导致开发者在使用 API 时不清楚需要提供哪些参数, 或者在自动化测试中无法生成有效的测试用例。此类问题定义为问题②。
- 3) 错误的 URI 路径: 文档中提供的 URI 路径可能与实际实现不一致。该情况可能由于文档未能及时更新以反映后端代码的变更所致, 因此遵循文档规范构造的请求仍可能因无法匹配任何有效的 API endpoint 而被拒绝。此类问题定义为问题③。
- 4) 参数类型与代码定义不匹配: 文档中声明的参数类型可能与 API 实际接受的类型不一致。例如, 文档可能声明某个参数为字符串类型, 但实际的 API 实现中期望接收的参数是整数类型, 此问题可能导致服务端处理异常。此类问题定义为问题④。

显然, 上述提到的不一致性问题可能降低 REST 服务的可用性、可维护性和可测试性, 甚至引起安全问题。本小节提出一种灰盒测试方法, 通过构造请求发送到测试服务器, 并根据响应来检测代码与文档的不一致性。详细描述如下:

- 1) 请求构造阶段: 通过解析遵循 OAS 规范的 API 文档, 生成相应的具有领域特性语言风格的 SDK (Software Development Kit), 并利用字典值填充 SDK 的请求参数, 用于创建 SDK 实例与待测服务交互。

- 2) 请求发送阶段：进一步向 SDK 提供测试服务器地址，并提供令牌 token 或会话 session 以进行身份验证，创建完整 SDK 客户端实例，并利用该 SDK 实例与目标待测服务交互，收集响应。
- 3) 响应检测阶段：基于上述步骤，可通过分析 HTTP 语义信息（根据 RFC 9110^[127]）实现从测试服务器收集的响应信息中进行代码与文档不一致问题的检测。

如表 5-2 所展示，在 HTTP 响应信息中，当状态码为 20x 时，通常预示请求已经被成功处理，进而表明代码与文档之间可能不存在不一致性问题。然而，当请求的方法不是 GET 方法但返回状态码为 30x 时，往往预示着文档中可能指定了错误的 URI 路径，从而导致了请求的重定向。此外，状态码 400 表示请求负载中可能缺少必要的参数或存在参数类型不匹配的问题，而状态码 405 则表明文档定义了一个实际上不存在的方法。

表 5-2 以 RFC9110 为依据，本小节对代码与文档不一致问题的定义

状态码	RFC 9110 定义	可能存在的 inconsistency 问题
20x	Successful	-
30x	Redirection	③
400	Bad Request	② ④
401	Unauthorized	②
403	Forbidden	-
405	Method Not Allowed	③
5xx	Server error	① ② ③ ④

5.2.3 基于装饰器插桩的 API 运行时逻辑信息获取方法

当前针对云平台通用业务层组件的测试方式主要为黑盒方式，仅能通过构造样本触发的 HTTP 响应分析待测应用可能存在的问题。因此其存在的局限性为：1) 难以获取业务运行时的内在逻辑信息，当前所分析的内容趋于表象；2) 基于 HTTP 响应的问题分析方法难以深入挖掘各形式的安全漏洞，包括潜藏在正常 HTTP 响应背后的逻辑异常。与此同时，逻辑漏洞作为一种与程序逻辑紧密相关的漏洞类型，不仅静态信息背后可能蕴含漏洞逻辑特征，动态信息背后可能蕴含更多能够表征程序执行逻辑的逻辑信息，因此有必要通过获取 API 运行时逻辑信息以扩展分析维度。

传统代码插桩技术主要针对源代码或二进制代码进行插桩，插桩粒度过高，严重降低服务的运行效率；同时云平台应用层通用组件主要通过 Python 语言和 Java 语言进行实现，因此本小节提出一种基于装饰器插桩的 API 运行时逻辑信息获取技术，通过向关键函数插入采集探针实现对目标函数追踪功能的添加^[128, 129]。本小节所提出的追踪框架如图 5-4 所示，根据第三章研究内容分析结果，利

用所开发的追踪函数实现对敏感 API 及函数的自动化插桩与运行信息的获取；将应用进行实际部署之后，采集探针实时获取程序的运行状态，并将运行信息发送至采集代理；进一步基于 hash 值实现函数调用栈结构的恢复与归一化存储。

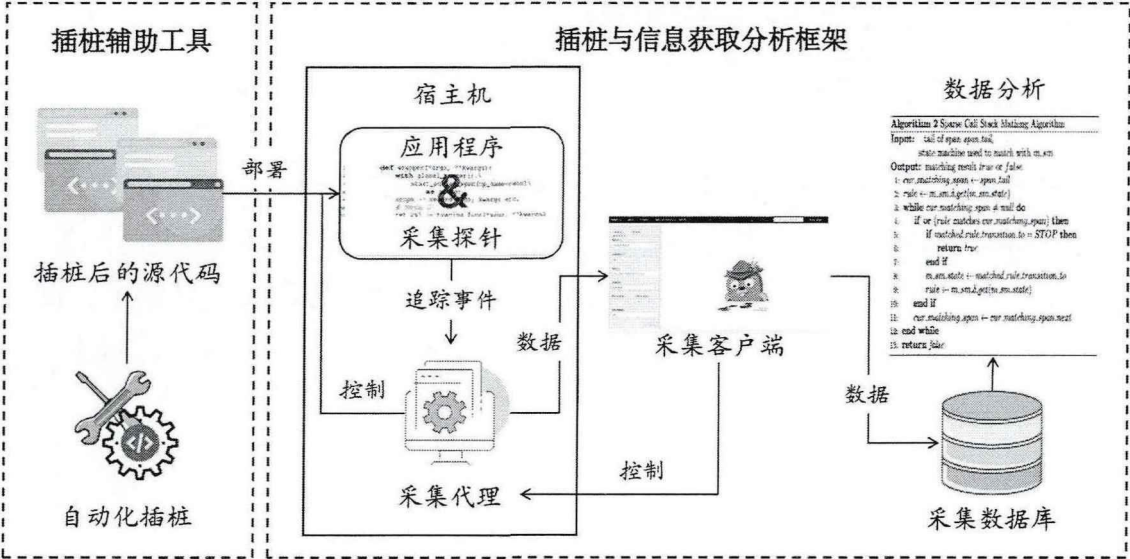


图 5-4 基于装饰器插桩技术的 API 运行时逻辑信息获取方法

装饰器的插桩工作原理如图 5-5 所示，其中：

- 1) `functools.wraps()` 是一个装饰器，用于更新包装函数（wrapper function）属性，例如 `__module__`、`__name__` 等，使得包装函数呈现出类似于被包装的原始函数（wrapped function）的特征。该方式的主要目的是确保包装函数元数据如函数名和模块名等保持不变，以便在调试或其他情况下保持一致性和可读性。
- 2) `*args`, `**kwargs` 此两类参数传递模式被设计用于传递所有函数参数到包装函数的追踪逻辑中，如同将这些参数直接传入原始函数。在作用域关闭时，`args`（位置参数）、`kwargs`（关键字参数）以及 `ret_val`（函数的返回值）会被携带至数据采集代理处，以记录和追踪整个函数的执行路径，辅助构建恢复完整的调用栈信息。

```
1 def trace_method_decorator(name):
2     def _decorator(tracing_func):
3         # Note 1
4         @functools.wraps(tracing_func)
5         def wrapper(*args, **kwargs):
6             with global_tracer().\
7                 start_active_span(op_name=name)\
8                 as scope:
9                 scope -> record args, kwargs etc.
10            # Note 2
11            ret_val := tracing_func(*args, **kwargs)
12            scope -> record ret_val
13            return ret_val
14        return wrapper
15    return _decorator
```

图 5-5 装饰器插桩技术的使用方法

基于上述流程，可实现对函数调用、参数、返回值的完整追踪，以支撑后续的逻辑漏洞分析。

5.3 基于模型推演与参数推断增强的测试样本构造方法

当前漏洞挖掘技术大多以模糊测试技术为代表，通过构造大量随机或满足特定模型约束的样本来对目标应用进行测试，以发现目标应用可能出现的安全问题，相关理念也已被微软等团队引入通用 REST 服务的可靠性测试。然而现有针对 REST 服务的安全测试工具仍然存在一定的局限性，即：所提出的测试方法主要针对单一 API 请求进行测试，难以深入发现 API 组合可能存在的逻辑漏洞；即便根据文档构造具备依赖关系的 API 测试组，测试工作仍然面临着不同 API 组合间由于代码实现错误导致的参数传递异常，进而制约了测试的深度和效果。本部分提出基于资源生命周期模型指导的测试模型构建，并基于参数推断缓解前文所述的代码与文档不一致问题，以解决业界困境。

5.3.1 基于资源生命周期的 IDOR 漏洞检测模型定义

本小节以一种典型的云平台业务层通用组件的逻辑漏洞，即不安全的直接对象引用（Insecure Direct Object References, IDOR），为例进行阐述。当云服务信任用户的输入并提供直接对象操作时，可能产生此类安全问题，攻击者可通过修改请求中的 ID 参数来绕过授权并引用任何对象。围绕资源生命周期模型，提出如下用于漏洞测试的生命周期模型：假设预先创建用户 A 和用户 B，在安全情况下他们不应该访问非自身创建的资源；当用户 B 可以访问用户 A 创建的对象时，存在 IDOR 漏洞。

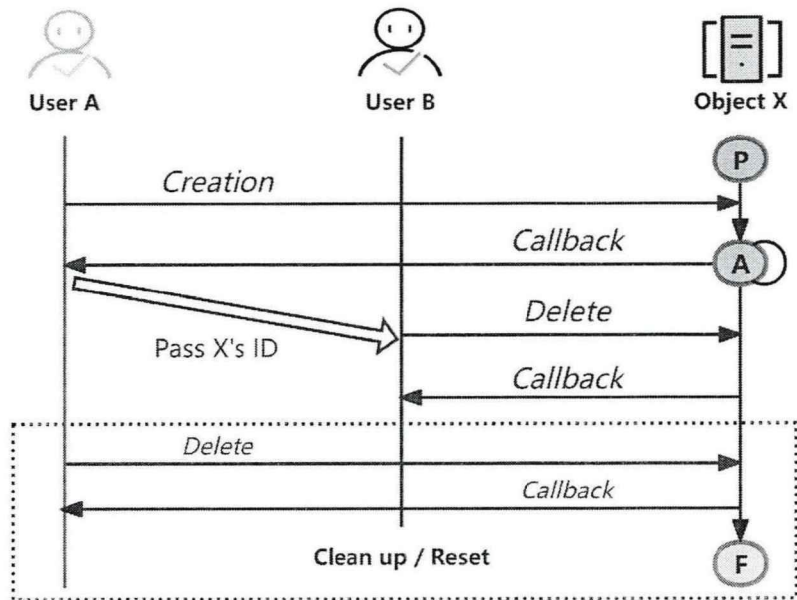


图 5-6 云平台业务层越权型 IDOR 逻辑漏洞模型示例

如图 5-6 所示，Object X 为待测资源实例，如前文所述，其生命周期总共包含 Pre-Creation、Available、Finalized 状态，用户 A 通过 Create 操作使 Object X 进入 Available 状态，并获得创建资源相关响应信息；从响应信息中提取 Object X 的 ID 信息，并使用预先创建的用户 B 通过引用 Object X 的 ID 试图删除 Object X 的实例；根据反馈信息，判断删除操作是否成功，一旦成功则证明存在本小节所描述的 IDOR 漏洞；最后，模拟用户 A 删除资源，以清空并重制测试环境，为后续测试排除干扰。

5.3.2 基于模型推演与参数推断的测试样本构造方法

本小节对 5.3.1 节所提出的样本构造方法进行扩展。通用推演方法包括单一逻辑推演与复合逻辑推演，如图 5-7 所示，“推演”概念重在推理与演化，即在理解资源生命周期的基础上，进行相关测试样本的推演构造。单一逻辑推演以样本的初始逻辑为基础，可进行元语缺失变异、元语内容变异、元语序列异序、元语插入变异等，从而实现单样本相关安全及可靠性问题的测试。复合逻辑推演以 API 组合逻辑为基础，可进行空间异位、时间异位以及主体异位等，空间异位包括 API 组合间的参数替换，部分情况下近似于主体异位；时间异位指 API 调用序列的顺序变异，例如通过 GET 方法和 DELETE 方法的时间异位可发现是否存在资源释放后使用或信息泄露等逻辑漏洞；主体异位则侧重于权限型逻辑漏洞的测试。

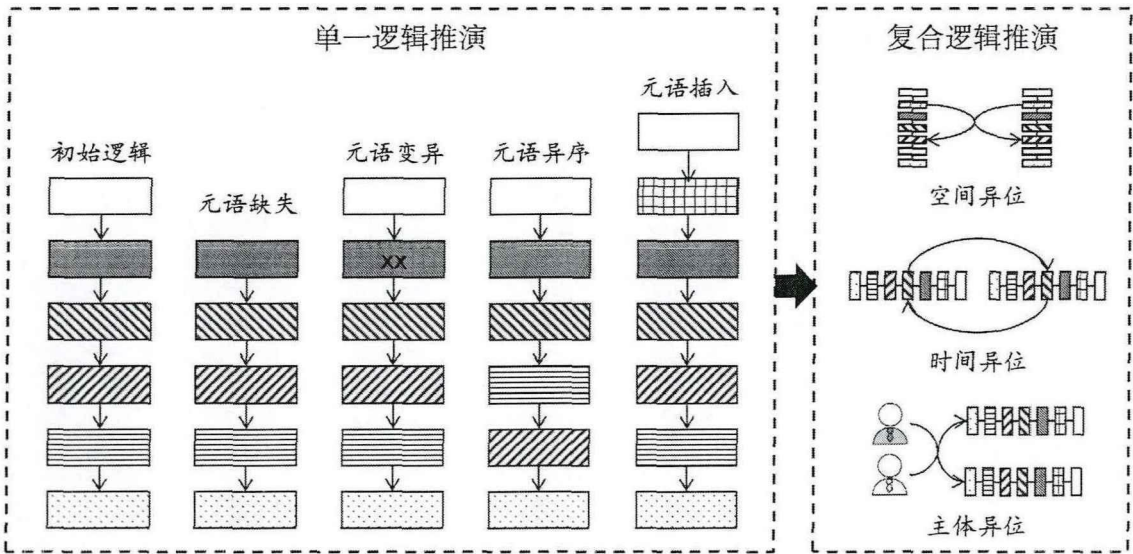


图 5-7 基于逻辑推演的样本构造基本方法

在定义模型推演方法后，通过测试组筛选准则，将基于模型推演的样本构造方法应用于目标云服务的所有符合条件的资源。通过分析相应的 API 归并树或过滤后得到的测试组，筛选满足推演方法基本要求的资源和测试组，按照其对应的漏洞模型推演规则的指导构造测试用例并进行测试，挖掘潜在的逻辑漏洞。以 IDOR 漏洞模型为例，详细的测试用例生成方法如下所示：

- 1) 测试环境初始化。在测试组中搜索 POST 方法 API, 从相应的 SDK 方法获取参数列表, 并根据字典或样本填充待填参数值。使用用户 A 的凭据初始化 SDK 客户端, 然后调用该 POST 方法 API 创建对象。
- 2) 参数解析与推断。在测试组中搜索带有待填参数的 DELETE 方法 API, 该待填参数应该是 ID 型参数, 以利用创建阶段所返回的对象 ID 参数进行删除操作。为了克服文档与代码不一致问题导致的测试深度不足等问题, 当 DELETE API 所“消费”的 ID 型参数与 POST API “生产”的 ID 型参数不一致时, 利用 ID 的相似性进行推断。尽管存在文档与代码不一致问题, 但开发者对参数的命名可能存在一定的规律性, 如用其逻辑含义进行命名, 比如 id 和 resource_id, 或该不一致源自于开发者的编写偏误, 如将 id 误写为 ld。因此可通过计算 ID 参数与创建操作响应中每个参数名的最长公共子串 (Longest Common String, LCS), 如公式 (5-1) 所示, 并选择最接近的参数作为目标 ID 型参数的推断结果。

$$\text{Target_ID} = \arg \max_{\text{ID}_x} \{\text{LCS}(\text{ID}, \text{ID}_x), \text{ID}_x \text{ in Response_param_list}\} \quad (5-1)$$

- 3) 逻辑异常测试样本的构造。使用用户 B 的凭据初始化 SDK 客户端, 并从创建操作的响应中提取 ID 参数值填充到 DELETE 方法的 ID 参数中; 如果响应中没有参数与 DELETE 方法的 ID 参数相匹配, 则进行参数推断并以推断结果进行填充。最后调用 DELETE 的 SDK 方法。
- 4) 样本响应信息的收集。从测试服务器收集响应以及运行时信息, 并将其存储于数据库中以供后续分析。
- 5) 测试环境重置。此环节整体流程类似于逻辑异常测试样本的构造环节, 但是使用合法请求即用户 A 自己的凭据删除对象 X, 以便重置测试环境, 使后续测试不受影响。。
- 6) 测试结果分析。在分析步骤中, 本章提出一种稀疏调用栈匹配的方法来检测追踪结果是否包含目标调用模式, 从而检测可能存在的逻辑漏洞。同时, 可基于测试预言与测试样本的设计直观地从响应状态码中分析出部分成功的漏洞利用, 详细方法将在 5.4 节进行描述。

5.4 基于 API 调用栈辅助的测试样本执行结果分析方法

基于本章所提出的 API 运行时逻辑信息获取技术, 能够在获得 HTTP 响应的同时获得 API 运行时的调用逻辑信息, 并进一步基于此调用逻辑信息提出相应的测试结果分析方法。基于前述方法, 可获得的信息包括三部分: 1) 函数调用属性 Attribute: 此部分内容存储若干键-值对来注释 span 信息, 同时也可以

来携带它跟踪的操作信息；2) Span 状态：描述运行时异常的字段，例如当应用程序发生已知错误时，会将 Span 状态设置为错误状态；3) Parent Span：父 Span ID，用于构造恢复 Span 间父子关系，即操作之间的调用者和被调用者关系，将被调用者的父 Span ID 被设置为调用者的 Span ID。经过上述操作，可构造完整的函数调用栈信息。

基于有限状态机理论，可以描述资源的生命周期状态转移，同时也可用于异常调用逻辑的检测。由于逻辑漏洞通常显示全局效应而不是局部效应，在仅关注局部语句或反馈信息时难以判断所有类型的异常。因此，本小节受资源生命周期有限状态机的启发，提出一种基于稀疏调用栈匹配的异常检测方法来检测逻辑漏洞：调用栈信息也具有其 FSM 特征，通过构建用于检测的 FSM，并利用稀疏匹配算法查找关键调用栈模式以发现逻辑异常，具体如算法 5-2 所示，基于 FSM 的匹配规则可包括：Span 名称、Span 属性和 Span 关系。

算法 5-2 基于稀疏调用栈匹配的目标调用模式检测算法

输入： 函数调用栈，以及序列的最尾部 span_tail，用于被匹配的状态机 m_sm	
输出： 用于指示匹配的结果的 boolean 值	
1.	初始化当前匹配的 span，即 $cur_matching_span \leftarrow$ 序列的最尾部 span_tail
2.	初始化规则 $rule \leftarrow$ 获取 m_sm 的状态 m_sm.state 相关的转移函数 δ
3.	当存在待匹配的 span，即 $cur_matching_span$ 不为空时，
4.	如果有任一规则 rule 与 $cur_matching_span$ 相匹配，则：
5.	如果所匹配到的规则 matched_rule 的下一个匹配状态为 STOP 终止状态
6.	返回 true
7.	m_sm 的下一个匹配状态 $m_sm.state \leftarrow$ matched_rule 所转移至的状态
8.	$rule \leftarrow$ 获取 m_sm.state 相关的转移函数
9.	$cur_matching_span \leftarrow$ $cur_matching_span$ 的下一个 span
10.	返回 false

算法 5-2 对基于 FSM 的稀疏调用栈匹配方法进行了阐述，在第 2 行， δ 为状态转移函数，即 $S \times \Sigma \rightarrow S$ ，m_sm.state 指向当前匹配状态，rule 为状态转移规则。第 3-9 行迭代地对 span 与 rule 进行匹配，只要 span 不为空并且 m_sm 中的任一状态被匹配，匹配规则将跳转到下一个状态（第 7 到 8 行）。在本算法中，第 9 行定义了下一个匹配的 span，而该匹配方向是从最后一个 span 开始向最初一个 span 进行的（第 1 行）。与此同时， δ 同样也可以是相反的方向，即从最后一个状态到最初一个状态，与匹配方向保持一致，此时 STOP 状态为最初一个状态的前一个状态。最后，如果 m_sm 中的状态都被匹配，则证明测试用例所触发的调用栈包含相关 FSM 模式（第 6 行）。

本算法采用了一种稀疏匹配的方式,其合理性在于无论正常或异常逻辑模式,其均反映在基于一定程度抽象的全局表达上而不是局部的执行特征,因此采用传统的黑名单或白名单匹配机制是无必要且不且当的,可能产生大量误报与漏报。本算法旨在通过对 Span Relationship 添加约束,实现不同稀疏程度的匹配,从而发现关键调用模式。此外,本节内容可作为传统基于 HTTP 响应信息的检测方式的补充,当测试样本明确是恶意的但触发具有正常模式的调用栈序列时,可以直接根据状态码检测逻辑缺陷。

5.5 实验评估

本章节最终实现原型工具 RESTLogic。针对本章节所提出方法以及原型工具的实验评估主要从三方面进行:一是评估当今云平台通用业务层组件(REST 服务)所面临的不一致性风险,评估其发生范围并判断其对于自动化逻辑测试的阻碍程度;二是评估本章节所提出的测试样本构造方法的覆盖率和有效性提升效果;最后评估本方法在热门开源云平台业务层通用组件上进行逻辑漏洞测试的效果,评估本方案相对于业界先进研究的优势所在。

本部分的实验内容运行于 2.11.1 版本的 QEMU 模拟器所搭建的 Ubuntu 20.04 LTS 操作系统下,搭载 32 核 2.4GHz CPU 以及 32GB 内存。所依赖的开发环境为 Python 环境,所分析的软件包括 GitHub 热门项目(GitHub 100 星以上项目,即通用 REST 服务),以及目前最为知名的开源云计算平台 OpenStack^[130],其版本为 Wallaby 版本,测试组件包括 Nova 网络管理组件等。

5.5.1 代码与文档不一致风险的分析与检测

为了分析云平台业务层组件及通用 REST 服务组件面临的不一致性风险,本小节首先对 GitHub 开源代码仓库中的热门项目进行分析,使用“REST service”和“RESTful service”作为关键词,“rest-api”和“restful-api”作为标签检索 Java 和 Python 语言开发的相关项目,筛选全部百星以上项目并进行分析,最终得出了 8 组统计分析结果。如图 5-8 所示,柱状图从左至右分别代表利用检索表达式检索到的“总项目数”、“REST 服务的总项目数”、“带有 API 文档的 REST 服务总项目数”、“手动编写 API 文档的 REST 服务总项目数”以及“API 文档符合 OAS 文档规范的 REST 服务总项目数”。结果显示,检索到的大部分项目为“生产”(即用于构造 REST 服务)或“消费”(即对 REST 服务进行分析) REST 服务的项目,如 vitalik/django-ninja 和 microsoft/restler-fuzzer。其中,大约一半 REST 服务具有 API 文档,但大多数基于人工编写或未声明生成方式,只有少数项目的 API 文档具有良好的文档标准及生成规范。因此,REST 服务中的不一致性问题非常普遍,自动化测试此类服务组件将面临巨大的挑战。

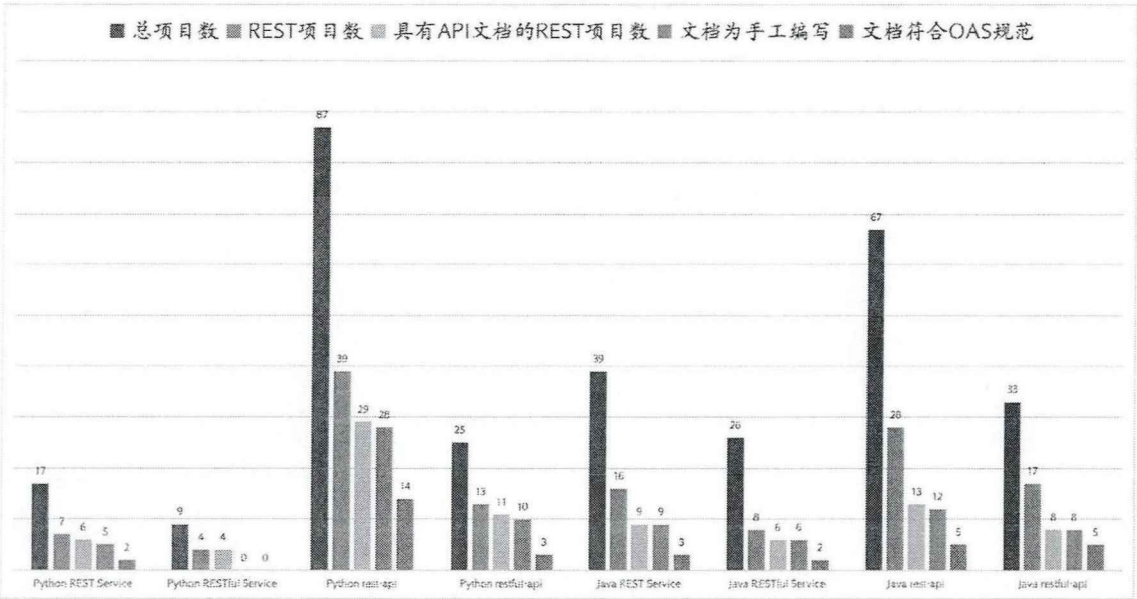


图 5-8 REST API 服务所面临的不一致风险

进一步地，选取其中部分代表性项目进行代码与文档不一致性检测，得到结果如表 5-3 所示，其中：“测试组”表示符合本章所提出的模型约束的目标测试组在全部测试组中的比例；“公共键”表示在所有目标测试组中，不同 API 之间参数对在命名相同的情况下，类型同样相同的组数的占比；“验证”是对公共键分析结果进行人工确认的结果。以 petstore 项目为例，全部 API 总共可划分为 7 个测试组，其中 3 个测试组符合 IDOR 模型准则；这三个测试组中，共有 11 对 API 存在命名相同的参数对，其中 3 组参数对参数类型同样相同，对此 3 组参数对进行人工审核，发现此三组参数对对应的 API 不存在不一致现象，而另外 8 组 API 则存在不一致现象。而 RESTler、EvoMaster、APIFuzzer^[131]、Tcases^[132]等领域内知名开源工具均不支持不一致问题检测，而 RestTestGen^[133]仅支持基于参数名匹配的检测，而不支持类型检查及参数推断。结果显示，不一致性在通用 REST 组件中非常普遍，在知名开源云平台业务层通用组件 OpenStack-nova 中尤为显著：23 个 API 测试组中均存在不一致问题，无法找到任何公共键。

表 5-3 基于本工具检测的代码与文档不一致问题

待测软件	RESTlogic			RESTler [68]	Evo Master [72]	API Fuzzer [131]	Tcases [132]	RestTest Gen [134]
	测试组	公共键	验证					
Gin-vue-admin	0-69	0-0	0-0	-	-	-	-	-
Go-gin-example	2-6	0-5	0-0	-	-	-	-	5
harbor	1-3	0-0	0-0	-	-	-	-	-
hydra	7-14	0-15	0-0	-	-	-	-	15
petstore	3-7	3-11	3-3	-	-	-	-	11
Openstack-nova	23-49	0-0	0-0	-	-	-	-	-

具体而言，本小节对 OpenStack 组件进行详细检测，并向官方提交相关不一致问题检测结果及修复报告，其中部分典型问题如表 5-4 所示：

表 5-4 检出并修复的 OpenStack 代码与文档不一致问题示例

项目名	Issue ID	API 文件路径	问题描述
openstack/nova	805972	api-ref/source/os-cells.inc	缺失请求定义
openstack/ironic	807332	api-ref/source/parameters.yaml	参数位置错误
openstack/keystone	807312	api-ref/source/v3-ext/oauth.inc	缺乏参数声明
openstack/keystone	807315	api-ref/source/v3/application-credentials.inc	缺乏参数声明

5.5.2 测试样本的测试覆盖率和有效性

本小节首先对当前云平台业务层通用 REST 服务组件测试覆盖深度低下的根本原因进行分析。由于大多数开源 REST 服务仅提供基本查询功能，不具备复杂的 API 操作及关联关系，因此本小节选择具有复杂资源操作的 OpenStack，以及最为典型的 REST 样例项目 petstore 来进行验证。部分实验结果如表 5-5 所示，例如：测试组 Group 1 中包含两个 API 请求，其中 API 1 POST 请求返回参数被 API 2 DELETE 请求使用；然而由于后端实现不一致，即编码或文档编写存在问题，API 1 的请求与响应中将 keypair 资源命名为 name，而 API 2 需要传入的参数名称为 keypair_name，但两个名称本质上均映射为资源 keypair。此现象导致了已有工具仅能对单一 API 进行测试，而无法发现 API 1 和 API 2 之间的参数传递关系，进而无法构造符合关联逻辑的测试样本，无法测试潜在逻辑漏洞。

表 5-5 本工具 RESTLogic 与 RESTler 工具在不一致问题缓解方面的能力对比

测试组	测试结果	
Group 1	API 1: POST /v2.1/os-keypairs	
	API 1 Request Parameter: {'keypairs_import_post_req':{'keypair':{'name':'test-keypair-1'},'user_ud':None,...}}	
	API 1 Response Parameter: {'keypair':{'fingerprint':'1e:...','name':'test-keypair-1','user_id':'cedc8...',...}}	
	API 2: DELETE /v2.1/os-keypairs/{keypair_name}	
	参数推断: lcs=name, keypair_name ← name;	依赖分析: Ø
Group 2	API 1: POST /v2.1/os-floating-ips	
	API 1 Request Parameter: {'floating_ips_create_req':{'pool':'public1'}}	
	API 1 Response Parameter: {'floating_ip':{'fixed_ip':None,'id':'9e4d0a51-...','pool':'public1',...}}	
	API 2: DELETE /v2.1/os-floating-ips/{floating_ip_id}	
	参数推断: lcs=id, floating_ip_id ← id;	依赖分析: Ø

续表 5-5 本工具 RESTLogic 与 RESTler 工具在不一致问题缓解方面的能力对比

Group 3	API 1: POST /v3/pet	
	API 1 Request Parameter: {'category': 'Category', 'id': '1', 'name': 'dog', ...}	
	API 1 Response Parameter: {'category': 'Category', 'id': '1', 'name': 'dog', ...}	
	API 2: DELETE /v3/pet/{petID}	
	参数推断: lcs=id, petID ← id;	依赖分析: Ø

其次，评估本章所提出的参数推断方式对于测试覆盖深度的影响。同样如表 5-5 所示，基于参数推断方法，对测试组 Group 1，本工具能够找到的最长公共子串（LCS）为 name，即由于开发者的编码不规范或理解不一致而将同一资源分别命名为 name 及 keypair_name；对于测试组 Group 2，本工具发现 floating_ip_id 与 id 实际上指向同一资源；而对于测试组 Group 3，本工具发现 id 与 petID 实际上指向同一资源，以此类推。对于目前最为先进的 REST 服务测试工具（包括 RESTler、EvoMaster、RestTestGen 等），它们提出的 API 依赖关系分析方法由于其基于参数名称进行 API 间依赖关系分析，因而完全无法解析此类 API 之间的参数传递关系，进而无法测试 API 组间逻辑。基于本工具所推断的结果对 OpenStack Nova API 进行测试，得到结果如图 5-9 所示：

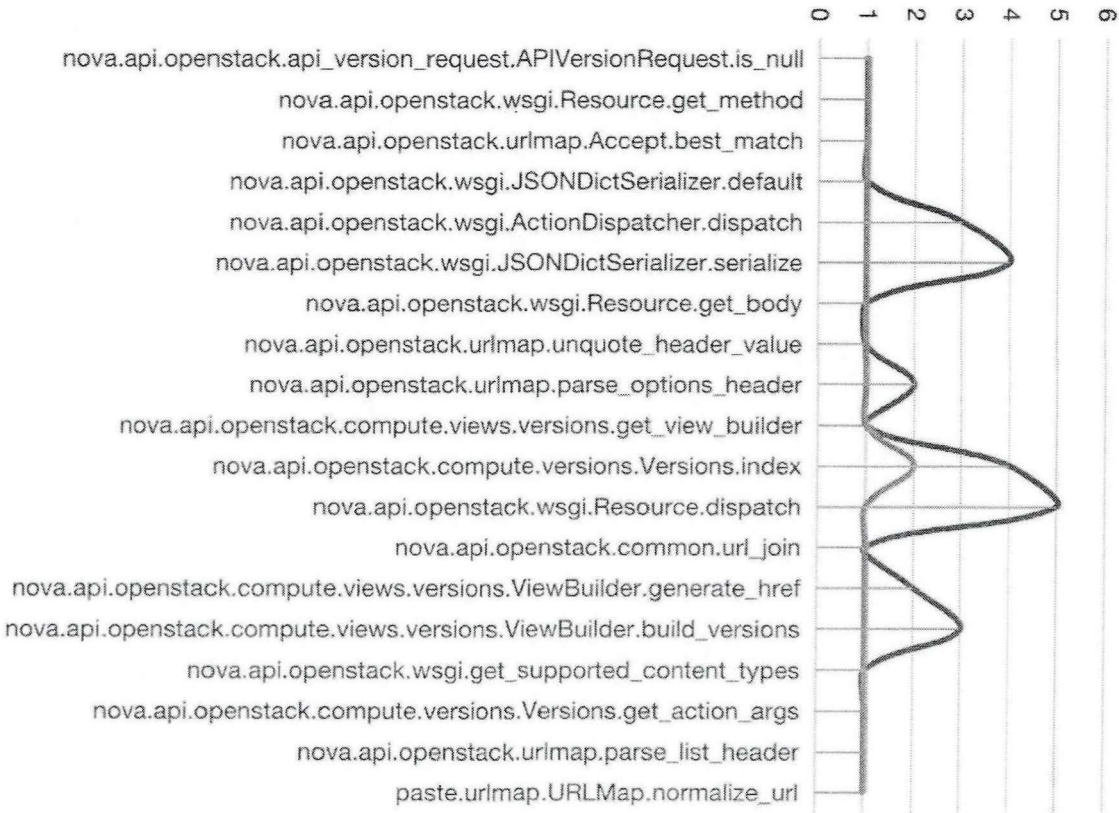


图 5-9 本工具 RESTLogic 与 RESTler 在测试深度覆盖率方面的对比

如图 5-9 所示, 为本工具 RESTLogic 与微软提出的 RESTler 工具在覆盖率方面的对比, 其中蓝色曲线为本文所提出工具所得到的覆盖深度, 而黄色为 RESTler 实现的覆盖深度。首先定义“测试深度”和“测试广度”概念: 测试深度, 即基于所提出的 API 运行时调用栈获取方法, 根据调用栈的树形结构, 获取以某一 API 为入口函数, 直至树形结构的叶子节点的函数调用深度, 该深度即为针对该 API 的测试深度; 测试广度即是否能够测试足够多的 API, 并能对彼此依赖的 API 进行分析。实验结果表明, 以 OpenStack Nova API 为例, 本工具在全部可测 API 的测试深度上得到了更好的效果, 这是由于本工具所构造的样本符合资源生命周期模型约束, 且能够发现并解决代码与文档不一致问题, 因此能够对 API 进行深度测试; 而对于测试广度, 由于本工具相较于 RESTler 能够发现更多 API 之间的关联关系, 如表格 5-5 所示的测试组, 本工具能够将 6 个 API 分成三组 API 进行测试, 而 RESTler 由于无法求解参数之间的传递关系, 故而只能将 6 个 API 划分为 6 个单独的测试目标, 因此无法实现对测试组的覆盖。

5.5.3 对热门公有云平台业务组件的逻辑漏洞挖掘效果

本小节主要讨论本章所提出方法对于热门开源云平台业务层通用组件的逻辑漏洞挖掘效果, 并与业界最先进的工具进行对比, 以证明本方法的有效性, 并探索最适合本方法检测的逻辑漏洞。对于漏洞检测, 需要首先定义测试预言 (即 test oracle), 用于声明预期结果, 符合预期结果则证明存在正常行为或异常行为: 若进行异常检测, 则可预言正常行为, 不符合预期的行为证明为异常; 可预言恶意行为, 符合预期的行为证明为恶意行为。上述预言对应两种检测模式: 1) 基于调用栈的异常检测, 构建异常调用样本, 若运行结果存在异常调用模式, 或异常样本触发正常调用, 则证明存在逻辑漏洞; 2) 基于状态码及调用栈的联合检测, 构建异常调用样本, 若触发成功调用 (即状态码为 20x), 则证明存在逻辑漏洞。

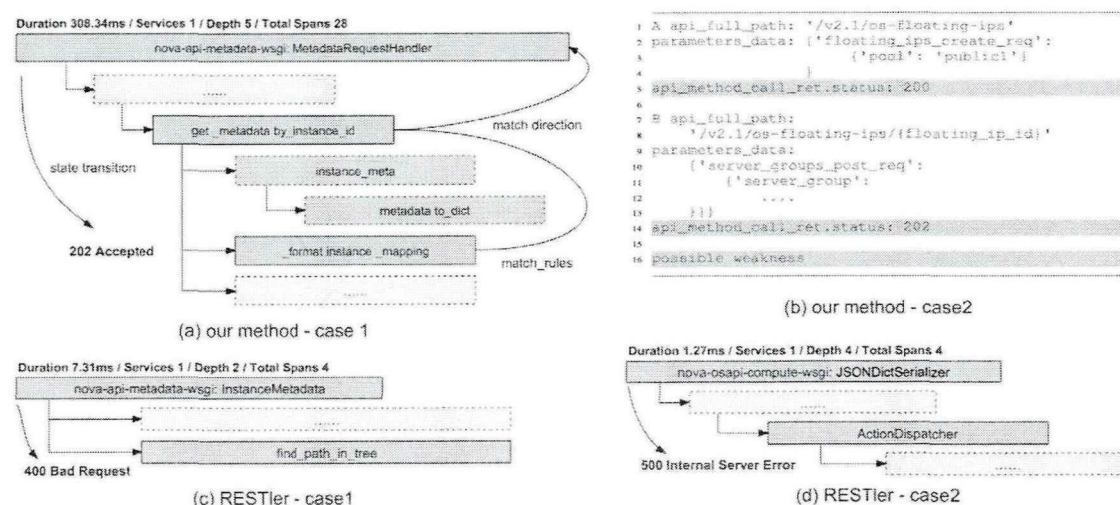


图 5-10 本工具 RESTLogic 与 RESTler 在分析漏洞方面的原理对比

如图 5-10 所示为本工具 RESTLogic 与 RESTler 的检测结果。如图 5-10 (c)、(d)所示, RESTler 使用单一 HTTP 返回信息中的状态码作为测试预言, 当 HTTP 响应信息中出现 4xx 和 5xx 的状态码时, 认为存在异常(如服务端处理错误), 因此 RESTler 适合发现的问题为服务端处理异常及错误等。而本工具对于第一类测试预言, 如图 5-10 (a)所示, 可以根据测试深度发现逻辑问题: 即异常逻辑调用不应触发深层次调用, 应当被后端提前拒绝, 然而恶意调用能够触发同正常调用相同的调用结果, 同时返回 20x 状态码, 则证明存在逻辑漏洞。对于第二类测试预言, 如图 5-10 (b)所示, 异常调用样本反而触发 200 和 202 响应状态码, 则根据该响应可直接判断存在逻辑漏洞。

表 5-6 本工具发现的热门开源云平台业务层通用组件 OpenStack 的逻辑漏洞

组件名称	漏洞位置	漏洞效果
openstack-nova	nova/api/metadata/handler.py#L111	信息泄露型逻辑漏洞
openstack-nova	nova/api/openstack/compute/floating_ips.py#L270	越权型逻辑漏洞

如表 5-6 所示, 本小节对热门开源共有云平台业务层通用组件 OpenStack 进行逻辑漏洞挖掘测试, 发现 OpenStack Nova 组件的元数据泄露安全问题; 并发现 Nova 组件默认配置下为用户分配权限过高, 同时鉴权机制设置不当, 导致的用户可以越权删除其他同级别用户甚至更高级别用户资源的逻辑漏洞, 并已提交漏洞报告至有关机构。相较于业界其他云平台业务层通用组件的测试方法, 本方法更适合挖掘具备复杂 API 调用关系的逻辑漏洞, 具备发现正常调用结果背后潜藏的异常逻辑问题的能力, 而传统工具则更适合发现服务端处理异常等代码质量问题。

5.6 本章小结

云平台业务层通用组件为云平台运营者管理虚拟化资源、云平台用户使用虚拟化资源提供便利, 用户无需了解代码的具体实现, 即可将个人应用或数据托管至平台, 以减轻自身运维负担。然而由于此类应用作为“中间件”, 以及其身为用户与资源间接口的敏感性, 导致其一旦出现逻辑漏洞, 将威胁合法用户、甚至整个云平台的安全。当前业界针对此类应用的安全检测方式大多借鉴传统的模糊测试理念, 基于应用接口文档构造符合文档描述的测试样本, 并基于遗传算法或随机变异, 实现对目标业务组件的模糊测试, 但此类方法更适合发现服务端处理异常等可靠性问题, 对逻辑漏洞挖掘通常无能为力。因此本章提出基于动态程序逻辑推演的业务层逻辑漏洞挖掘方法, 基于业务层通用组件的 RESTAPI 描述文档信息构造逻辑漏洞测试模型, 并基于装饰器插桩技术获取 API 运行时调用栈信息, 最终通过稀疏调用栈匹配方法联合 HTTP 响应信息发现逻辑漏洞。

与业界最新研究进行对比,本章所提出的 API 文档分析方法能够有效构建资源的生命周期描述模型,并指导相关漏洞检测模型的构建以及目标测试组筛选与生成,所提出的 API 运行时逻辑信息获取方法能够捕获关键动态运行时逻辑调用关系,作为分析阶段的支撑与补足;所提出的测试样本构造方法能够构造满足资源生命周期约束的测试样本,能够实现针对 API 测试组的逻辑关联关系测试,并基于参数推断解决测试过程中不同 API 之间参数传递异常导致的测试覆盖率低下等问题,提高发现逻辑漏洞的概率;所提出的基于调用栈信息与 HTTP 状态码联合分析的漏洞检测方法能够有效发现热门开源项目的业务逻辑漏洞。实验结果表明,本方案不仅能够解决传统工具无法更好的发现业务层通用组件逻辑漏洞的问题,同时能够提高对于 API 逻辑测试的覆盖深度,并挖掘热门开源公有云项目的未公开业务层逻辑漏洞多项,取得更好的效果。

在未来的研究过程中,可进一步深化扩展对于逻辑漏洞模型的描述及刻画,并利用本方案框架进行更广泛的验证;针对参数推断可采用智能化方式,以覆盖当无法利用参数名称及参数类型进行推断时的情况。

第六章 基于程序运行时逻辑特征的应用层逻辑漏洞挖掘

云平台应用层通用组件包括各类部署后为用户提供服务的软件。基于亚马逊的责任共享模型（Shared Responsibility Model, SRM），在云平台架构下，云厂商与用户共建云应用，并共同为云平台的安全性负责。引入应用层通用组件便捷了云厂商的开发运维，丰富用户的应用体验，但同时也引入更为隐蔽的安全问题。以当前最为流行的云上托管机器学习模型、大模型应用为例，尽管机器学习相关安全问题已得到人们的广泛关注^[135, 136]，如对抗攻击^[137, 138]、数据投毒^[139]等，但机器学习模型背后潜在的后门逻辑漏洞安全隐患仍未引起人们的足够重视，机器学习模型在此时被视为特殊的代码或数据，恶意模型部署并加载在用户操作系统后，可能导致逻辑异常，甚至引起反序列化代码执行。

在此背景下，学术界与产业界针对机器学习模型后门漏洞的研究开始起步，AI 安全厂商 HiddenLayer 提出在机器模型中嵌入反序列化后门漏洞的方法^[140]，中科院团队在 EvilModel 一文中针对在机器学习模型中嵌入恶意软件对机器学习模型造成的性能损失开展了评估^[13, 141]。TrailofBits 提出 fickling 工具^[142]，用于检测 Python pickle 文件中的序列化安全问题，在一定程度上缓解了机器学习模型反序列化后门漏洞隐患。然而目前针对机器学习模型漏洞检测方式的研究仍然较少，大多现有工具仍然面临可扩展性差、场景适配能力弱、难以发现模型部署阶段的调用逻辑异常等问题，对机器学习模型漏洞的检测能力差，更难以适对其他类型云平台应用层通用组件的漏洞检测。

针对上述问题，本章提出一种基于程序运行时逻辑特征的应用层逻辑漏洞检测方法。首先，针对此类“部署型”云平台应用层通用组件，提出一种基于宿主机运行调试器的程序运行时信息获取方法，以调用栈树形结构描述部署时逻辑特征；其次，针对通常情况下基于调试器的程序运行时信息获取遗漏及效率低下的问题，本章基于漏洞先验知识对现有方法进行优化，以实现追踪性能的提升和追踪内容的优化，并构建此类“伪静态”追踪信息的归一化表示方法，使其与静态程序分析信息融合，实现分析底座覆盖范围的提升；随后，基于所获取的调用栈信息提出基于调用栈序列树的云平台应用层通用组件的后门逻辑漏洞检测方法；最后，实现原型工具 DPDetect 并对 Hugging Face、GitHub 等知名开源代码仓库托管的机器学习模型进行测试，最终发现现实网环境下攻击团队向开源代码仓库投放的 10 余项在野恶意模型。

本章组织架构如下：6.1 节对概念定义、威胁模型、设计目标进行阐述；6.2、6.3 节对本章所提出的核心方法进行阐述，包括机器学习模型的后门逻辑漏洞植

入方法、部署时调用栈信息的优化获取方法、调用栈信息的归一化表示方法、基于调用栈信息的逻辑漏洞检测方法；6.4 节针对知名开源代码仓库托管的恶意模型进行检测，并与业内先进研究进行对比，证明本方案的有效性；6.5 节对本章工作进行总结并提出研究展望。

6.1 问题描述

本节主要介绍针对云平台通用组件的基于程序运行时逻辑特征的应用层逻辑漏洞挖掘方法的有关概念定义、威胁模型以及设计目标。

6.1.1 概念定义

机器学习模型/预训练模型。机器学习模型是机器学习的核心组件，机器学习模型是指能够从数据中学习规律和模式，并应用于预测、分类、聚类等任务的数学模型或计算机程序；而预训练模型则是在某些数据集上提前进行了训练、学习了特定任务的隐含映射关系的模型；在本文的定义中，机器学习模型包含用于执行机器学习任务的模型数据和代码。当前大多数机器学习应用基于 Python 开发，许多机器学习模型在存储时会以 Python 的 pickle 文件格式保存，这种格式具有功能内置、节省内存、方便启动和停止训练、训练后模型可移植等优点。此外，开源网站如 PyTorch Hub 和 Hugging Face 等提供了机器学习预训练模型的共享平台，部分函数库还提供 API 支持自动从 GitHub 库中下载并部署模型。

代码执行后门逻辑漏洞。本文所定义的后门逻辑漏洞不同于现有机器学习安全研究领域最为广泛的后门的概念，是指通过向机器学习模型中嵌入各种形式的恶意代码以引发后续操作，甚至实现对目标系统的远程操作或控制的攻击行为。具体而言，攻击者可能会设计出复杂的手段，利用机器学习模型工作原理中存在的逻辑漏洞风险，在模型中插入恶意代码片段，在满足特定触发条件（如模型部署）时激活并执行预定操作。通过利用此类后门逻辑漏洞，攻击者可以绕过目标系统的安全防护措施，实施未被授权的行为，包括执行任意代码、窃取敏感信息、控制目标系统的关键组件等，甚至破坏目标系统的正常运作。

模型部署特征。模型的部署是一个动态的行为，而模型部署特征是该动态行为的静态特征表现形式。本章所关注的机器学习模型后门逻辑漏洞，其发生阶段要早于传统的机器学习后门，即：传统意义上的机器学习后门由数据投毒等行为造成，对用户的模型使用阶段造成影响；而对于机器学习模型后门逻辑漏洞，由于模型的篡改与投毒并非是一个攻击者持续可控的过程，通常攻击者只会将修改后的模型投放到开源代码仓库，模型一旦被其他用户下载并部署，便会执行恶意代码，因此此类安全问题发生在模型的部署阶段。可以见得，模型的部署行为是相对固定的，模型后门逻辑漏洞并非实时的影响机器学习任务，而是在部署阶段

便彰显其全部特性。因此将此类相对固定的动态行为特征，以动态方式提取并以静态形式表征，进而支撑对后门逻辑漏洞的检测。

6.1.2 威胁模型

与传统方法通过对抗训练或数据注入影响模型性能或输出的安全威胁不同，本文的关注点在于模型的保存与部署过程中可能面临的安全风险，而非训练过程，具体包括：

- 1) 在保存阶段，机器学习模型通常以.pt 或.pth 格式（一种常用的 Python 对象序列化格式）保存。此阶段面临的主要安全威胁包括模型泄露、模型伪装和后门植入。由于模型以未加密的二进制文件形式存储，加之开源模型的广泛使用，使得模型在保存阶段的安全问题尤为重要。
- 2) 在部署阶段，机器学习模型面临的安全问题主要为恶意代码执行。此阶段面临的安全威胁主要为保存阶段攻击行为的具象化表现，即当保存阶段面临的安全威胁真实发生时，这些攻击行为将在部署阶段产生效果，引起恶意代码执行。

将机器学习模型视为一种特殊的软件供应链通用组件，模型泄露、模型伪装和后门植入等问题实质上是对模型的“篡改”，目的在于劫持模型以执行恶意代码，而不仅仅是控制模型的输出。本章的研究重点在于如何有效地检测此类机器学习后门逻辑漏洞，确保机器学习开源模型的安全性，从而保护整个目标系统的安全，并通过此类代表性应用探索云平台应用层通用组件后门逻辑漏洞的一般性检测方法。

6.1.3 设计目标

本章的设计目标包括以下三个部分：

- 1) 提出并总结机器学习模型后门逻辑漏洞的攻击与利用方法。针对所提出的概念定义，总结机器学习代码执行后门逻辑漏洞的隐蔽植入方法，并评估这些方法对模型效果的影响。
- 2) 评估现有检测手段对于机器学习模型后门逻辑漏洞的检测能力。针对代码执行后门，利用现有检测工具进行检测，评估现有检测手段的有效性。
- 3) 提出针对机器学习模型后门逻辑漏洞的新型检测方法。针对现有检测手段难以检测的代码执行后门逻辑漏洞，提出新型检测方法，通过沙箱模拟模型加载行为并结合静态分析和动态分析技术，深入分析隐蔽攻击过程中的异常行为逻辑模式，从而实现更为精准的攻击检测，实现对潜在后门逻辑漏洞的预警和防范。

6.2 机器学习模型应用组件后门逻辑漏洞的植入与利用方法

如前文所述，许多机器学习模型使用 Python pickle 文件作为底层存储格式。pickle 是 Python 的序列化工具，可将对象保存为二进制数据，并通过反序列化操作以还原对象。pickle VM 有大约 70 个操作码，其中大部分与堆栈上值的操作相关；然而为了能够存储类，pickle 同时还实现了可以加载任意 Python 模块并执行方法的操作码，这些指令旨在调用 Python 类的 `__reduce__` 和 `__reduce_ex__` 方法以返回执行类重建所需的所有信息。然而，由于缺乏安全限制，这些操作码可被用于执行带有任何参数的任意 Python 函数，因此 pickle 格式在逻辑本质上是不安全的，如图 6-1 所述^[143]。与此同时，本文发现大量开源项目中由于 pickle 不安全使用导致反序列化代码执行从而控制目标的高危漏洞，包括 Ray AI 分布式计算框架 (Github 30.5k Stars) 以及 gpt_academic 大模型接口 (Github 53.3k Stars)。

Warning: The pickle module is **not secure**. Only unpickle data you trust.
It is possible to construct malicious pickle data which will **execute arbitrary code during unpickling**. Never unpickle data that could have come from an untrusted source, or that could have been tampered with.
Consider signing data with hmac if you need to ensure that it has not been tampered with.
Safer serialization formats such as json may be more appropriate if you are processing untrusted data.

图 6-1 官方对于 pickle 不安全使用的声明

基于此，本节对机器学习模型后门逻辑漏洞的植入与利用方法进行研究总结，具体包括直接注入与隐写注入。

6.2.1 模型保存阶段的直接注入方法

直接注入方法涉及直接在模型的保存文件中插入恶意代码或攻击载荷，通常通过修改模型文件的序列化数据来实现，即在模型序列化（保存）过程中，通过拦截序列化函数（如 Python 的 `pickle.dump` 或 PyTorch 的 `torch.save`）来直接添加额外的恶意对象或代码。此方法可通过扩展或修改序列化库来实现，即重写 pickle 的内部机制，实现特定对象或命令的注入，或者在序列化调用之后执行额外的代码注入。直接注入的关键在于向目标模型注入可导致任意代码执行的函数，笔者对相关可注入的操作进行全面总结，具体如表 6-1 所示。

此种注入方法的特点是：可见性，即由于恶意代码或数据通常直接存在于文件或代码中，因此可以通过简单的文本或代码审查发现；执行方式直接，恶意代码通常会在目标程序或系统的正常运行流程中被执行，导致立即的安全威胁；检测难度小，由于恶意内容的直接可见性，使用标准的安全工具（如病毒扫描器、静态代码分析工具、基于字符串匹配的防火墙）即可相对容易检测地到直接注入的恶意代码。因此针对基于此方法注入的后门逻辑漏洞往往通过直接对模型的二

进制文件进行审查便可直接发现，此方法也是目前 Hugging Face 内置安全检查工具的主要实现原理。

表 6-1 可注入的操作类型

函数类型	函数名称
文件和 IO 操作	open, file, io.open
执行和评估代码	eval, execfile, compile, exec
系统命令和进程	os.system os.popen, os.popen2, os.popen3, os.popen4 subprocess.call, subprocess.check_call, subprocess.check_output, subprocess.Popen
文件和目录操作	os.listdir, glob.glob shutil.copyfile, shutil.copy, shutil.copy2, shutil.move, shutil.make_archive dircache.listdir, dircache.opendir
管道和低级文件操作	os.open, os.pipe posixfile.open, posixfile.fileopen
序列化与反序列化	pickle.load, pickle.loads cPickle.load, cPickle.loads
其他实用工具和函数	map, input os.access, os.execl, os.execle, os.execlp, os.execlpe, os.execv, os.execve, os.execvp, os.execvpe, os.spawnl, os.spawnle, os.spawnlp, os.spawnlpe, os.spawnv, os.spawnve, os.spawnvp, os.spawnvpe linecache.getline timeit.timeit, timeit.repeat sys.call_tracing code.interact, code.compile_command, codeop.compile_command pty.spawn, platform.popen

6.2.2 模型保存阶段的隐写注入方法

由于通过直接注入产生的模型 data.pkl 文件中会包含某些字符串，这些字符串显式的包含了可能导致代码执行、信息泄漏的命令，这些命令在某些恶意 pickle 检测工具（如基于字符串匹配的防火墙）中会直接被认为是恶意的，因此这些恶意模型会被直接过滤，无法达到实际的后门漏洞植入与攻击利用效果。因此，一种结合机器学习模型特性、能够实现隐蔽植入的基于隐写注入技术的后门漏洞植入方式被提出。隐写注入方法不直接修改代码，而是在模型的参数或元数据中隐藏恶意信息，这些信息在模型加载 pickle.load(model)时被解析并动态执行，因此这种方法通常更加隐蔽，大多静态安全检测工具难以及时发现。

一种基于参数（张量或权重，tensor）的植入方式如图 6-2 所示，该方法将攻击载荷直接插入或分段插入到训练数据、模型权重中，一旦满足特定触发条件，后门将会被触发，并通过还原重组攻击载荷达到后门攻击的效果。通常情况下模型权重采用 32 位浮点数进行存储，基于 IEEE 754 标准^[144]可知，对权重 32 位浮点数的后 8 位进行修改通常对该浮点数的实际值影响很小（如浮点数的十六进制

表示 bc000000 与 bc0000ff, 两者的差值仅为 10^{-7} 数量级), 该改变几乎不会对机器学习的总体效率产生太大影响。当参数浮点数的个数足够多时, 通过尾数的 1-8 位将能存储足够大的后门逻辑漏洞载荷。

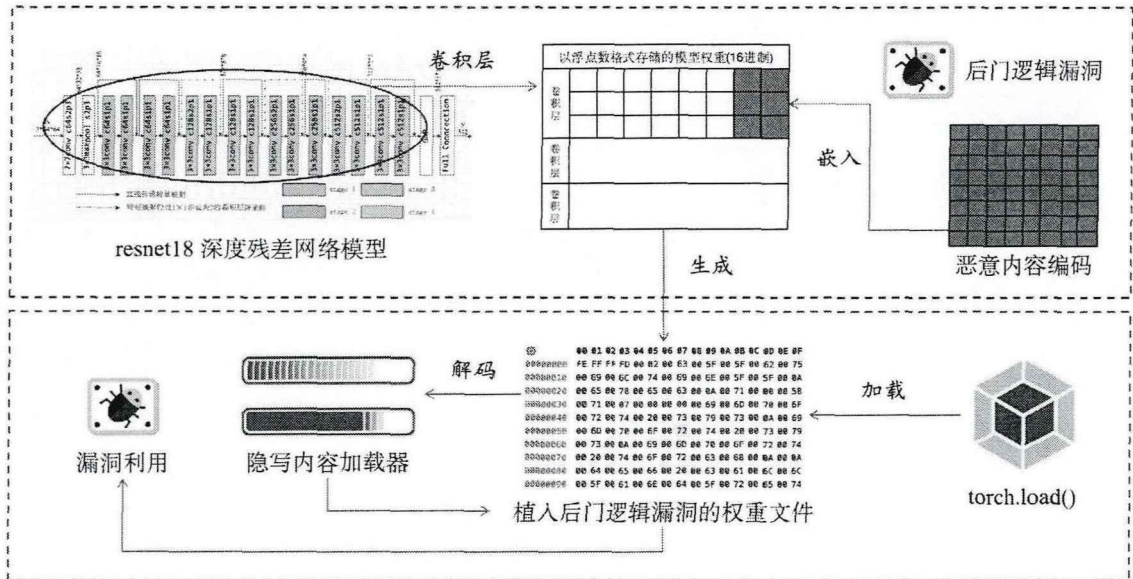


图 6-2 基于隐写注入的后门逻辑漏洞植入方法

6.3 基于运行时逻辑特征的机器学习模型后门逻辑漏洞检测方法

本节对机器学习模型的部署时特征进行分析, 首先将机器学习模型定义为一种一次部署型应用 (One-Time Deployment Application), 用于指代被设计和配置为一次性部署并长期运行, 而无需或很少需要后续更新、维护或干预的软件应用。此类应用的特点是稳定性, 即一旦部署后, 它们就能够在没有持续监控或频繁更新的情况下稳定运行, 云厂商或独立用户可从开源模型仓库, 如 PyTorch Hub、Hugging Face 中下载预训练模型并部署至主机, 实现快速的模型落地应用, 而无需对模型进行进一步修改。即便用户需要对模型进行微调与迁移, 本质上模型也在用户主机上首先完成了“一次性部署”, 因此机器学习模型最终可归类为此类应用。

由于机器学习模型的部署过程其本质上是所存储的二进制格式数据的反序列化的过程, 因此在“一次部署”阶段, 即模型需要被使用时, 二进制数据将会被反序列化以恢复到原来的对象状态, 恶意代码也随之一同恢复。由于反序列化过程通常不会对即将执行的代码进行安全性检查, 因此恶意代码可能像正常代码一样被执行。因此, 嵌入在机器学习模型的后门逻辑漏洞在模型被加载时便可执行各种恶意活动, 如数据泄露、权限提升、远程代码执行等。由于机器学习模型通常被视为黑箱, 模型的内部结构和数据对于使用者而言并不透明, 因此植入的

恶意代码可能很难在常规的安全审查中被发现，从而在模型部署阶段未经检测地被触发。

然而在本质上恶意行为的触发仍然是在宿主机执行恶意命令，因此所有的函数调用逻辑在宿主机均有迹可循。因此本节提出一种沙箱环境下基于运行时逻辑特征的机器学习后门逻辑漏洞检测方法，具体方法如下：

6.3.1 基于宿主机运行调试器的部署时运行信息追踪方法

本节提出一种在沙箱环境下，通过宿主机上运行的调试器来追踪部署时运行信息的方法。本方法的核心是利用 Python 的标准调试器来实现对运行在沙箱中的 Python 应用程序的监控和分析。由于 Python 语言开发的应用程序，尤其是现今流行的机器学习框架，如 TensorFlow、PyTorch 等，出于性能考虑在其底层实现中使用了大量编译后的 C 或 C++函数，因此对于目标应用同样需要考虑对此类函数调用逻辑的追踪。当前对于 Python 语言开发的应用，主要从表 6-2 中的几类事件考虑追踪信息的获取，包括：

表 6-2 需要获取的目标追踪事件描述

事件名称	事件描述
Call	在 Python 函数被调用时触发，代表 Python 的函数调用事件
Return	在 Python 函数完成执行并返回时触发，代表 Python 函数的返回事件
Line	在解释器执行到新的源代码行时触发，以实现逐行调试
Exception	在 Python 函数发生异常时触发此事件，代表 Python 的异常事件
C_call	在 C 函数被调用时触发，代表 C 的函数调用事件
C_return	在 C 函数完成执行并返回时触发，代表 C 函数的返回事件
C_exception	在 C 函数发生异常时触发此事件，代表 C 的异常事件

目前 Python 的内置库 bdb（Python 标准调试器的基础类）已被广泛应用于 Python 应用的跟踪调试^[145]，但仍然存在如下问题，制约了安全问题的分析：1）编码错误，由于 bdb 内置库开发人员的理解错误，开发人员声称可追踪的 c_call 事件实际上无法进行追踪，导致了其无法全面分析机器学习模型部署的完整函数调用逻辑；2）追踪信息不全面：对于 call 及 c_call 事件的追踪不全面，同时无法获得模块被导入阶段的函数调用，即内置__import__函数，导致了部分可被用于执行敏感操作的库函数无法被追踪；3）性能开销过大：内置库 bdb 实现了一种近似于单步调试的追踪方式，追踪信息过于冗余，部分无法表征程序执行逻辑的信息同样被追踪，对准确分析程序调用逻辑产生干扰。

因此本小节提出改进的追踪算法如算法 6-1 所示：

算法 6-1 基于运行调试器的部署时运行信息优化追踪算法

输入： 需要被追踪的函数 <code>traced_func</code>	
输出： 追踪结果，以函数调用栈的形式进行展示	
1.	自定义 Class Mybdb:
2.	初始化追踪事件列表 <code>EventList</code>
3.	函数 <code>trace_dispatch(current_frame, event, arg)</code> :
4.	如果 <code>current_frame</code> 的 <code>event</code> 类型在 <code>EventList</code> 中
5.	调用对应 <code>dispatch_{*}(frame)</code> 函数进行处理，以记录调用栈信息
6.	
7.	函数 <code>generate_call_stack(current_frame, traceback)</code> :
8.	初始化 <code>call_stack = []</code>
9.	当 <code>current_frame</code> 不为空，则对于每一个 <code>current_frame</code> :
10.	将 <code>current_frame</code> 以及对应的代码行号添加到 <code>call_stack</code>
11.	<code>current_frame = current_frame.f_back</code> 的上一个帧
12.	调用栈逆序，以反映实际的调用顺序
13.	如果 <code>traceback</code> 不为空，则从 <code>traceback</code> 开始添加额外的帧:
14.	将 <code>traceback</code> 的帧对象、对应的代码行号添加到 <code>call_stack</code>
15.	<code>traceback = traceback.f_back</code> 的下一个帧对象
16.	返回 <code>call_stack</code>
17.	
18.	主函数:
19.	<code>__builtins__.__import__ = __trace_import__</code> ，设置对内置 <code>__import__</code> 函数的追踪
20.	<code>Mybdb().set_trace(sys._getframe().f_back)</code> ，创建 <code>Mybdb</code> 追踪实例
21.	调用被追踪函数 <code>traced_func</code> ，在本文中即为模型部署函数 <code>pickle.load()</code>

其中，自定义类 `Mybdb` 基于 Python 内置库 `bdb` 进行修改（第 1-16 行），以修复官方原始问题，并适配本文的分析目标。初始化事件追踪列表 `EventList` 实现对 Python 函数调用和 C 函数调用的追踪（第 2 行）；`trace_dispatch` 函数实现追踪任务的委派，将追踪事件委派至对应函数（第 3 行）；函数 `generate_call_stack` 实现面向函数栈帧及 `traceback` 的调用栈生成算法，以构造完整的函数调用栈（第 7 行）。在追踪过程中，仅需加入追踪语句 `Mybdb().set_trace(sys._getframe().f_back)`，即可实现对目标模型部署阶段触发的函数调用栈信息的“无侵入式”获取。

6.3.2 基于运行时信息的函数调用逻辑特征归一表示方法

本文第三章提出了一种逻辑漏洞语言无关型特征提取与表征技术，用于表征静态程序逻辑分析结果。作为动态运行特征的静态表征形式，“一次部署型”应用的部署时调用特性同样可被视为静态分析结果。为提升所表征程序逻辑信息的

丰富性，提升本文所提出的方法框架的可扩展性，同时便于分析结果的可读性，本小节提出一种函数调用逻辑特征归一化表示方法，将所有静态分析结果归一化为 SARIF（Static Analysis Results Interchange Format）静态分析结果标准交换格式，提供统一的格式来表示静态代码分析的输出结果。通过使用 SARIF 格式，不同的代码分析工具可以输出易于互操作和集成的分析结果，使得本文工具的分析结果可以被多种工具和平台共同使用和理解，本小节提供如算法 6-2 所示的调用栈归一化表示算法。

算法 6-2 利用 SARIF 格式对调用栈结果进行归一化表示

输入： 追踪所获得的调用栈信息 <code>stack_info</code>	
输出： 调用栈信息的 SARIF 格式 JSON 文件 <code>sarif_template</code>	
1.	函数 <code>convert_call_stack_to_sarif(stack_info)</code> :
2.	初始化 <code>sarif_template = {SARIF 格式, SARIF 版本, runs: []}</code>
3.	其中, <code>runs = [{"tool": Mybdb, "results": []}]</code> , <code>results</code> 用于存储结果
4.	定义规则集合 <code>rules = { }</code>
5.	对于 <code>stack_info</code> 中的每一个 <code>info</code>
6.	<code>ruleID = "Mybdb 追踪所获调用栈信息"</code>
7.	如果 <code>ruleID</code> 不在已有规则集合 <code>rules</code> 中, 则新建规则存储调用栈信息
8.	静态分析结果 <code>new_result = {</code>
9.	<code>"ruleId"规则 ID: ruleID, "level"等级: "安全/警告",</code>
10.	<code>"message"信息: stack_info 所存储的调用栈信息,</code>
11.	<code>"locations"位置: stack_info 的实际物理位置</code>
12.	<code>}</code>
13.	将 <code>new_result</code> 添加至 <code>sarif_template</code> 中
14.	返回 <code>sarif_template</code>

其中代码第 2 行指明归一化表示所遵循的 SARIF 格式信息、SARIF 版本信息，并利用 runs 字段存储实际的分析结果；代码第 3 行 tool 指明分析结果的来源，在本小节中分析结果来源于本章所提出的追踪工具 Mybdb，并利用 results 字段逐条存储追踪所获得的调用栈信息；代码第 5 行开始对调用栈信息进行逐条分析，并将对应的规则类型、是否存在安全隐患、实际的调用栈信息、调用栈信息的真实物理位置进行存储，最后在第 14 行返回完整的归一化表示文件。

6.3.3 基于部署时运行信息的模型异常调用逻辑检测方法

对于一次部署型应用，每一类应用都具有独特的运行时调用特征。对于同类一次部署型应用，由于其设计目的、功能需求和架构相似，它们在运行时的行为模式（尤其是函数调用的模式）倾向于显示出相似性。它们通常遵循：共享设计和架构原则，如机器学习模型应用通常基于序列化与反序列化实现模型的保存与

加载；相似的功能集，即通常具有一组核心功能片段用于实现功能逻辑；共用库和框架，这些共用元素导致其在运行时表现出相似的调用模式。而对于不同类型的部署型应用，由于其被设计用于完成不同的任务、遵循不同的架构和设计模式、使用特定的库并表现出规律性的依赖关系，因此，它们将具备不同的运行时调用逻辑。对同类部署型应用的部署时行为进行建模并检测异常调用逻辑从而分析逻辑漏洞是行之有效的方案。

以 6.3.1 章节所提出的基于运行调试器的部署时信息追踪方法为基础，利用大量安全机器学习模型对机器学习模型的安全部署逻辑进行画像，并获取安全部署逻辑的函数调用序列树，从而刻画安全模型的部署时行为。以当前应用最为广泛的模型部署加载方案为例，PyTorch 框架通过 `torch.load(f, map_location=None, pickle_module=pickle, **pickle_load_args)` 函数实现机器学习模型 `pkl` 文件的反序列化，从而实现模型的加载。通过对大量安全模型进行分析，所生成的正常逻辑树根节点将为 `<module>()`，即整个模型加载脚本的运行；根节点的子节点将以 `load()` 函数作为节点属性，代表模型通过 `torch.load` 函数进行加载，并可能通过调用 `__enter__()` 方法以实现模型文件处理等操作；进一步根据函数调用关系构建完整的调用序列树，示例的树形结构将在实验评估小节进行展示。

基于安全机器学习模型部署阶段的函数调用特征构造的安全调用逻辑树形结构，提出基于该树形结构的异常调用逻辑检测方法，异常类型包括但不限于如下几类：

- 1) 非标准函数调用结构。如果模型在执行过程中调用了不在安全调用序列树中的函数，或调用了非常规函数类型，或触发了非标准调用结构，则表示模型可能执行了预期外的潜在危险操作。
- 2) 异常调用顺序。即使所有单独的函数调用都在安全序列中，但如果它们的顺序与定义的标准模式不符，也可能表示异常。例如，一个函数被过早或过晚调用，或者在不正常的上下文中被调用。
- 3) 频繁或少见的函数调用，即异常调用频率。即使被调用的函数本身在安全序列中，但其被调用的频率异常增高或异常降低也可能指示异常。部分情况下可等同于无限循环或递归调用模式。
- 4) 循环或递归调用的异常模式。某些异常可能涉及到复杂的调用模式，如无限循环或递归调用，进而导致性能问题或意外行为，最终可能关联拒绝服务型逻辑漏洞。
- 5) 分支结构的异常迁移。特指在函数调用树中，即便函数调用的总体先后顺序未出现异常，但函数调用序列或路径在预期之外的节点实现路径转移，出现了不正常或者预期外的变化。

如图 6-3 所示,父子节点之间的调用顺序用实心箭头表示,兄弟节点之间的强制先后关系(中间无其他调用函数插入,如 E、F 之间的关系)用短虚线表示,兄弟节点之间的非强制先后关系(允许其他调用函数插入,如 B、C 之间的关系)表示为长虚线。因此可得:(1)、(2)为正常的调用结构;(3)存在非标准函数调用 G(及非标准函数调用结构 $A \rightarrow D \rightarrow G$),可能执行预期外命令,进而造成代码执行;(4)为异常调用顺序,可能造成资源泄漏、权限绕过等问题;(5)为频繁函数调用,可能造成资源耗尽与服务降级;(6)为异常的递归调用结构,可能造成拒绝服务;(7)为分支结构迁移,可能发生条件判断错误或循环逻辑故障,可能发生权限绕过等问题;(8)为调用缺失,一旦出现部分关键输入验证调用的缺失,可能造成鉴权异常、权限提升等安全问题;这些异常模式与嵌入的具体攻击载荷类型相关。

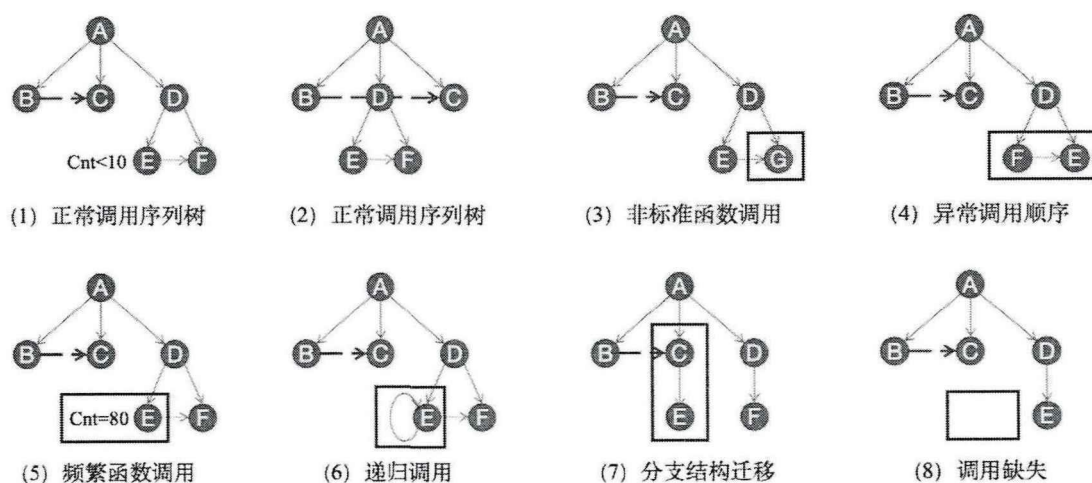


图 6-3 基于运行时调用栈逻辑信息的异常调用逻辑检测

基于调用栈树形结构的异常逻辑检测可以根据异常类型采用不同的检测方式:对于非标准函数调用,一旦出现未在合法范围内的调用函数即可报告异常。当树形结构极其复杂,且存在复杂调用规则,如分支转移、松耦合的调用顺序等时,可同样利用图数据库查询语句对检测规则进行表征,并通过图数据库检索的方式对异常结构进行查询,即:将合法调用逻辑或异常调用逻辑抽象为状态机查询语句,并将查询语句应用于待测模型的函数调用栈以查找异常,这一理念与本文前述第三章节和第五章节相契合并基于其展开验证。

6.4 实验评估

本章节最终实现原型工具 DPDetect。针对本章节所提出方法以及原型工具的实验评估主要从三个方面进行,一是针对模型后门逻辑漏洞的植入效果进行评估,评价多种植入方式对原始应用的影响、现有工具对这些植入方法的检测能力,从而评估更具威胁的植入方式、指明现有工具需要提升的方向;二是评价本工具

的性能,即基于宿主机运行调试器的部署时逻辑信息的提取效率及效果;三是评价本工具对于模型后门逻辑漏洞的检测能力,即评价本工具与当前业界最先进工具对于本文所涉及的最隐蔽的后门逻辑漏洞植入方式的检测效果,并对实网环境下开源代码仓库中的模型进行检测。

本实验环节在 ubuntu20.04 操作系统中进行，搭载 32 核 2.4GHz CPU 以及 32GB 内存，搭建 Docker 虚拟环境，配置 Python 开发环境，安装 Python debug 插件 bdb，测试目标包含 GitHub 和 Hugging Face 等热门开源仓库中的 PyTorch 机器学习模型。

6.4.1 后门逻辑漏洞植入方式的效果评估

如章节 6.3.1 所述，模型后门逻辑漏洞植入方式主要包括直接注入与隐写注入两种方式。其中直接注入的后门逻辑漏洞在模型的二进制文件中具有显式的字符特征，通常基于字符串匹配的漏洞检测工具可进行高效检测；而隐写注入方式通过对模型的参数文件进行操作，具有更好的隐蔽效果，通常需要对模型的部署时调用特征进行分析才能进行准确检测。首先分析直接注入方式对于原始应用，即机器学习模型的影响，主要分析逻辑漏洞注入后的模型变化情况。如图 6-4 所示，为实验示例的模型在植入漏洞后的参数文件变化情况。实验结果表明，直接注入方式所嵌入的后门逻辑漏洞在模型文件中直接显示关键的敏感函数利用载荷，具有显式的字符型特征，如 `os.system`。

[illegible]

图 6-4 基于直接注入方式注入 eval 命令后的模型参数文件特征

分析隐写注入方式对于原始应用，即机器学习模型的影响，主要分析对于机器学习任务性能的影响情况。由于隐写注入方式依赖于对机器学习模型参数的修改，一旦模型性能由于参数修改出现严重下降，则逻辑漏洞有可能在用户使用阶段被用户察觉，无法起到隐蔽植入的效果。因此评价此类方式是否能够起到隐蔽效果的有效途径即评价对模型任务的影响程度。如表 6-3 所示，为隐写注入方式对模型性能的影响。其中表格第一列为预训练模型的模型名称及模型参数文件大

小，第一行为恶意代码名称及恶意代码大小，第二列为预训练模型的基准准确率（Accuracy），后七列为嵌入恶意代码后的模型准确率。

实验结果表明，当以参数文件 32 位浮点数的后 8 位进行后门逻辑漏洞嵌入时：当嵌入的恶意程序大小确定时，模型的参数文件大小越小，模型的准确率受到的影响越大；当模型参数文件大小确定时，嵌入的恶意代码越大，模型的准确率受到的影响越大。根据实验结果可知，通常情况下，当恶意载荷小于模型文件大小的 1%时，模型的准确率几乎不会受到影响；当恶意载荷为模型文件大小的 5%左右时，模型的准确率开始呈现下降趋势；如表格中加粗内容所示，当恶意载荷大小达到模型文件的 10%左右时，模型准确率显著下降，直至恶意载荷大小为模型文件的 15%以上时，模型几乎不可用。这启发攻击者在选择隐写注入的攻击方式时，选择合适的模型文件及攻击载荷以实现隐蔽植入的效果。

表 6-3 嵌入不同大小恶意代码对不同大小模型任务效果的影响

Model	基准 准确率	Equation Drug 372KB	ZeusVM 405KB	NSIS 1.7MB	Mamba 2.3MB	WannaCry 3.4MB	VikingHorde 7.1MB	Artemis 12.8MB
VGG19 548.1MB	74.2%	74.2%	74.2%	74.3%	74.2%	74.2%	74.2%	74.2%
VGG16 527.8MB	73.4%	73.4%	73.4%	73.4%	73.4%	73.4%	73.4%	73.4%
AlexNet 233.1 MB	56.5%	56.5%	56.5%	56.5%	56.5%	56.4%	56.4%	56.4%
ResNet101 170.5MB	77.4%	77.3%	77.3%	77.2%	77.1%	77.0%	76.7%	74.5%
Inception 103.9MB	69.9%	69.9%	69.9%	69.5%	69.6%	69.1%	64.7%	61.3%
ResNet18 44.7MB	69.8%	69.7%	69.6%	69.2%	69.1%	68.9%	67.4%	60.3%
MobileNet 21.1MB	71.9%	71.0%	71.1%	68.5%	60.6%	39.7%	0.1%	-

数据来源：表中部分数据来源于 EvilModel 一文^[13]。

模型嵌入的后门逻辑漏洞及恶意软件本质上与 Python pickle 的反序列化逻辑漏洞紧密相关，而目前对于反序列化相关漏洞的检测以针对 Java、PHP 语言的反序列化研究为主，针对 Python 的反序列化安全检测，特别是针对机器学习模型的反序列化安全研究相对较少。目前最为主流的工具包括 TrailofBits 团队提出的 fickling 工具，用于检测 pkl 格式的 Python 反序列化文件风险；以及 Verma 提出的 Insecure Deserialization Detection in Python（以下实验中简称为“反序列化扫描器”）^[146]。

本小节利用上述工具对两种后门植入方式进行检测，评价现有工具对模型后门逻辑漏洞的检测能力。检测结果如表 6-4 所示，其中 fickling 只支持 pickle 类型文件的检测，同时 fickling 0.0.4 版本只能检测 eval、exec、subprocess 等命令的注入，不能检测 system 以及 runpy 等命令的注入，造成此结果的根本原因在

于：1) fickling 工具仅能获取静态信息，因此只支持对 pickle 文件（即模型二进制文件）的检测，不支持对完整模型文件的检测；2) fickling 工具仅支持对黑名单敏感函数的检测，所构建的检测规则并不完善，未能覆盖所有可能导致代码执行的函数调用逻辑。而 Python 反序列化扫描器实现能力有限，无法对任何 Python 机器学习模型反序列化逻辑问题进行检测，仅能检测部分传统 Web 应用反序列化安全问题。因此基于隐写注入的后门逻辑漏洞植入方式具有极强的隐蔽性。

表 6-4 目前主流检测工具对于模型相关安全问题的检测能力

工具	恶意行为类型			
	pickle 文件	PyTorch 模型	Tensorflow 模型	Python Web 反序列化
fickling	部分	×	×	×
反序列化扫描器	×	×	×	部分

6.4.2 部署时运行信息追踪的效果及效率

首先，本小节对追踪结果进行示例分析。如表 6-5 所示，本小节收集 113 个 PyTorch Hub、Hugging Face 等开源机器学习模型托管平台提供的知名机器学习预训练模型，并确认其安全性，并进一步以此为基准收集信息构造正常机器学习模型的调用序列树。

表 6-5 所构造的安全模型的调用结构（部分节选）

<module>()	
├──	/usr/local/lib/python3.8/dist-packages/torch/serialization.py--load()->'--C_Call--:keys'
	├── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--__enter__()->'--Call--:__enter__'
	├── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--__exit__()->'--C_Call--:close'
	├── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--__exit__()->'--Call--:__exit__'
	├── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--_is_zipfile()->'--C_Call--:append'
	├── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--_is_zipfile()->'--C_Call--:len'
	├── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--_is_zipfile()->'--C_Call--:read'
	├── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--_is_zipfile()->'--C_Call--:seek'
	├── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--_is_zipfile()->'--C_Call--:tell'
	├── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--_is_zipfile()->'--Call--:_is_zipfile'
	├── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--_legacy_load()->'--C_Call--:__build_class__'
	─── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--UnpicklerWrapper()->'--Call--:UnpicklerWrapper'
	─── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--_check_seekable()->'--C_Call--:seek'
	─── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--_check_seekable()->'--C_Call--:tell'
	─── /usr/local/lib/python3.8/dist-packages/torch/serialization.py--_check_seekable()->'--Call--:_check_seekable'
...	

所得到的安全调用序列树包含 1281 个节点，树形结构最大深度为 21 层，调用序列的根节点为<module>()，代表模型加载脚本执行；树形结构的首个子节点 /usr/local/lib/python3.8/dist-packages/torch/serialization.py-load() -> 'keys'代表某些模型可通过调用 load 函数，以字典方式加载模型状态；进一步地，后继子节点 __enter__()、__exit__()等调用与 Python 的上下文管理器(context manager)有关，表明某些模型通过 torch.load()加载模型的过程中使用 with 语句或类似机制实现适当地打开和关闭资源（如文件）；此外，树形结构利用--Call--与--C_Call--对函

数调用类型分别进行标注，包括 Python 函数调用以及跨语言底层 C 语言函数调用；如图 6-5 所示，展示了该树形结构中函数的调用频次，其中纵轴为函数调用名称，横轴为以 10 为底函数调用频次的对数（由于部分函数调用的频次过高，如 `serialization.py--_maybe_decode_ascii()->'--C_Call--:isinstance'` 函数被频繁地用于 `tensor` 底层数据相关处理，因此利用对数函数对调用频次进行简化处理以作图进行展示）；总而言之，该树形结构示例地表明了机器学习模型应用组件在部署阶段应具备的安全调用模式，刻画了模型部署的正常行为逻辑。

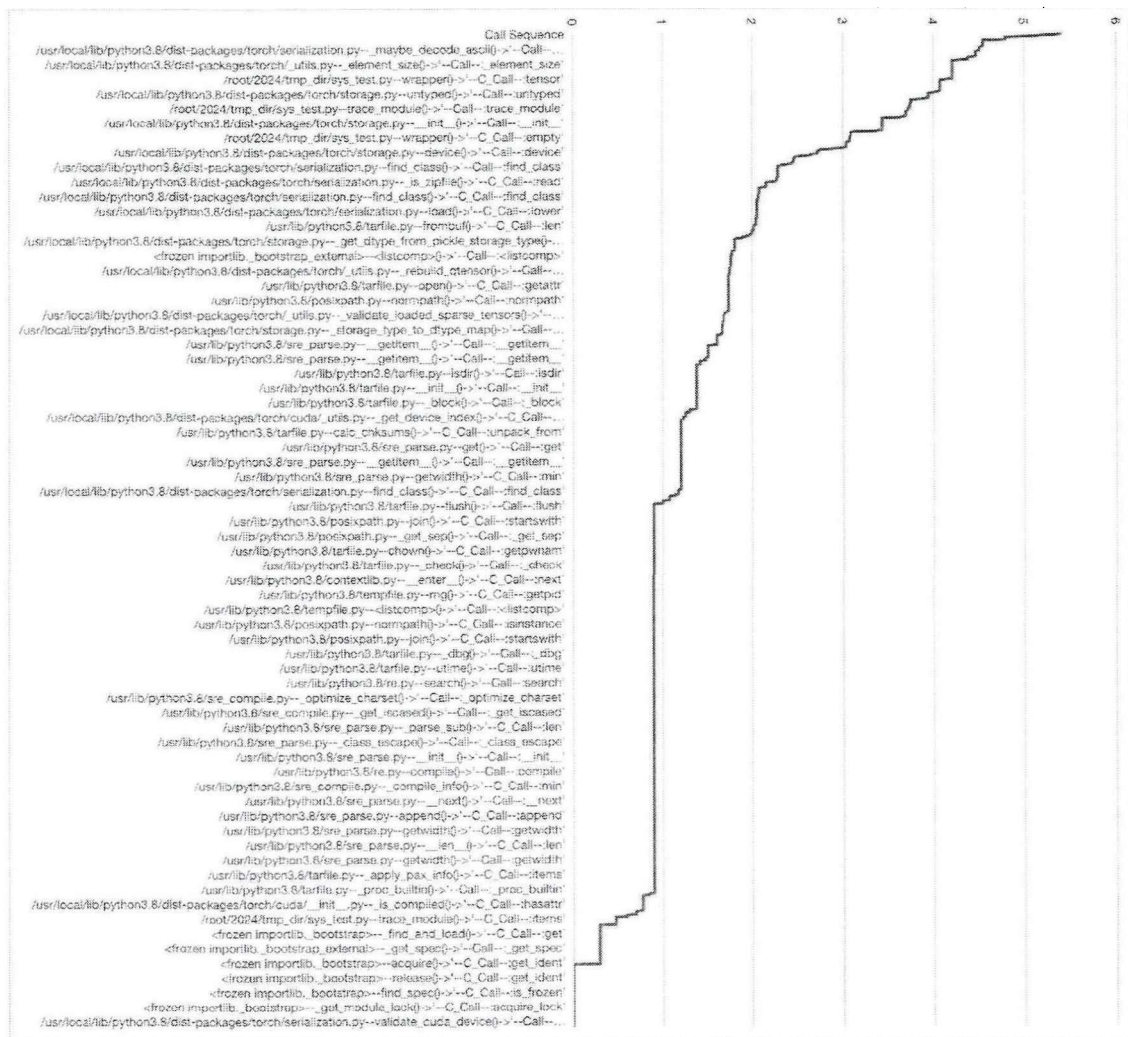


图 6-5 树型结构中的函数调用频次特征

其次，本小节对所提出调用结构的收敛特性进行分析，即分析是否存在一种调用结构能够近乎涵盖所有安全调用，如图 6-6 所示。子图(d)结果表明，当按照加载模型后新增调用节点数量进行降序排列，所加载的安全模型数约为 15 时，所构造的正常调用序列树实现收敛，这表明：1) 机器学习预训练模型作为一类功能相似的应用层通用组件，尽管其部署过程存在一定的差异性，但也存在着更为显著的收敛性，1281 个节点几乎覆盖了所有的正常调用逻辑；2) 不同类型的模型之间的调用特性存在一定差别，如子图(e)所示的 VGG 系列模型和子图(f)所

示的 ResNet 系列模型,但每一族独立的机器学习预训练模型内部也具有收敛性,即遵循该系列模型独立的设计规范。实验证明,通过收集大量具有代表性的正常模型,可保证所构造的正常调用序列树能够几乎涵盖所有安全机器学习模型应有的设计规范,进而支撑对异常调用模式的检测。

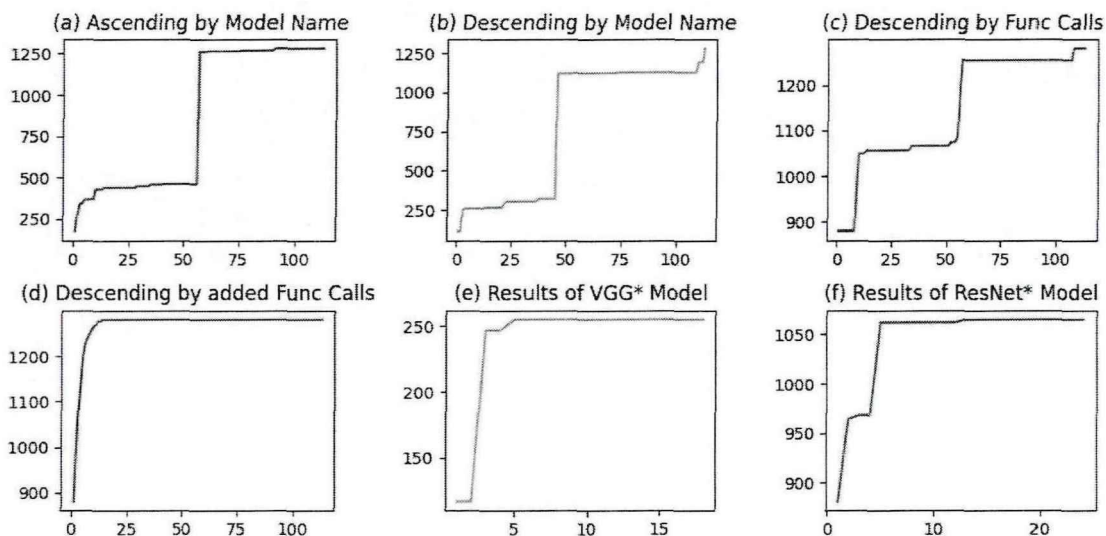


图 6-6 安全调用结构的收敛情况

进一步地,本小节对运行时信息的获取效果及效率进行对比,所对比的对象包括目前应用最为广泛的基于 bdb 单步调试执行的追踪信息获取方法,以及本章所提出的改进后的关键逻辑信息提取方法,即 Mybdb。其中 Mybdb 又分为 Mybdb,即原始改进优化方案,以及 eMybdb,即补充追踪 C_Call 事件的信息获取方案。如表 6-6 所示,为本工具的追踪优化结果,根据实验结果对以下几方面的信息进行分析:1)本方案的性能优化程度,即 Mybdb 相较于原始 bdb 在各测试模型上的耗时缩短情况;2)本方案的追踪效果优化程度,即 eMybdb 追踪获得的 C_Call 事件在全部追踪得到的调用中的占比。

实验结果表明,基于原始 bdb 的单步调试信息获取方法在性能上表现出极大的时间消耗,对于较小模型,如 ResNet18(模型参数文件 44.7MB),运行时长超过 54 min(3249 s),而本章所提出的 Mybdb,3 min 22 s 即可完成全部信息的获取,而在补充追踪 C_Call 与 import 事件后,仍然能够做到 24min 左右完成全部事件的追踪,所补充的 C_Call 事件及 import 事件追踪占比超过 80%,达到 81.08%,总体性能优化为 93.8%。而对于参数文件较大的预训练模型,如 VGG19 模型(模型参数文件 548.1MB),原始 bdb 追踪耗时为 6 h 26 min 6 s,而 Mybdb 的追踪方式在 1 min 11 s 即可完成全部追踪,补充追踪 C_Call 事件与 import 事件的最终耗时为 2 min 9 s,所补充的 C_Call 事件及 import 事件追踪占比约为 75.1%,总体性能优化比例为 99.7%。

表 6-6 本工具 Mybdb 模块及 eMybdb 模块实现的追踪效果及效率优化情况

模型信息		追踪耗时				追踪内容		
模型名称	模型大小	bdb	Mybdb	eMybdb	性能优化	Call	C_Call	追踪优化
ResNet18	44.7 MB	3249	202	1420	93.78%	5057	21668	81.08%
VGG11	506.8 MB	3661	42	81	98.85%	2973	9699	76.54%
VGG13	507.5 MB	4824	47	92	99.02%	3021	9755	76.35%
VGG11_bn	506.9 MB	7252	95	175	98.69%	3519	10093	74.15%
ResNet34	83.3 MB	9056	296	2456	96.73%	6323	31250	83.17%
VGG13_bn	507.6 MB	9977	112	207	98.87%	3699	10249	73.48%
ResNet50	97.8 MB	12789	391	3648	96.94%	7668	36332	82.57%
AlexNet	233.1 MB	14409	108	4850	99.25%	3604	11762	76.55%
VGG16	527.8 MB	16653	62	112	99.62%	3189	9807	75.46%
VGG16_bn	527.9 MB	19851	141	260	99.28%	3969	10483	72.54%
DenseNet121	30.8 MB	19922	939	1746	95.28%	11799	17269	59.41%
VGG19	548.1 MB	23166	71	129	99.69%	3279	9885	75.09%
VGG19_bn	548.1 MB	32923	169	371	99.48%	4239	10717	71.66%
Inception_v3	103.9 MB	32964	655	1350	98.01%	8517	16167	65.50%
ResNet101	170.5 MB	33296	687	5393	97.93%	11701	57198	83.02%
DenseNet169	54.7 MB	34436	1302	2438	96.21%	15399	20389	56.97%
ResNet152	230.4 MB	36694	881	7129	97.59%	14326	73232	83.64%
DenseNet201	77.4 MB	36940	1551	2892	95.80%	17799	22469	55.80%
DenseNet161	110.4 MB	38974	1244	2324	96.80%	14799	19869	57.31%

综上所述，实验结果证明，对于问题 1），本章所提出的方案的平均性能优化超过 95%，实现了极大幅度的信息追踪获取的性能提升。对于问题 2），实验结果表明，在模型部署阶段，import 事件与底层 C 语言函数调用事件的发生比例一般情况下超过 50%，对于部分模型，能够达到 80%以上（调用占比与模型的具体设计有关，包括是否进行归一化处理、是否包含更多的参数与计算），证明模型部署阶段对于 import 内置库及 C 语言函数调用占比达到极高水平，部署模式呈现极强的规律性特征；同时对于非 import 及 C 语言调用事件，即 Python 函数调用，仍然具备规律的行为模式，因此基于此分析调用异常将具备极强的现实性与可行性。本文将在下一小节，即 6.4.3 小节对基于此信息的后门逻辑漏洞检测效果进行评估。

6.4.3 对开源模型后门逻辑漏洞及实网攻击的检测结果

对于本研究问题，实验的检测对象包括自构造的嵌入后门逻辑漏洞的机器学习模型，以及 PyTorch Hub、Hugging Face 上的待测开源机器学习模型。本小节基于大量开源安全机器学习预训练模型构建标准模型部署时函数调用树形结构，

进一步基于所提出的基于部署时运行逻辑特征的模型异常调用逻辑检测方法对包含模型后门逻辑漏洞的恶意模型进行检测。检出包含非标准函数调用结构的预训练模型，所检出的恶意序列包括：system 命令相关调用序列如 `Serialization.py-load()->'tell'`，`serialization.py-load()->'load'`，`sys_test.py-wrapper()->'system'`，此调用序列指示命令行命令执行问题，以及 exec 命令相关调用序列如 `serialization.py-load()->'tell'`，`serialization.py-load()->'load'` `sys_test.py-wrapper()->'exec'`，此调用序列指示恶意程序或代码的执行，如图 6-7 所示：

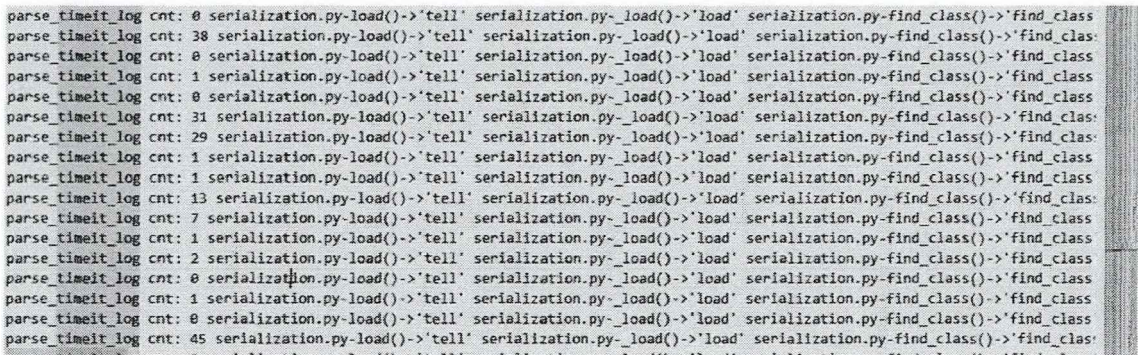


图 6-7 基于本工具 DPDetect 检测出的恶意调用序列

如表 6-7 所示，为本工具 DPDetect 与 fickling、反序列化扫描器以及 Hugging Face 内置安全检测工具的能力对比结果。实验结果表明，对于非标准函数类后门逻辑漏洞，本工具的检测能力优于目前最先进的工具 fickling、反序列化扫描器以及 Hugging Face 内置安全检测工具：其中 fickling 缺乏对非流行异常逻辑触发方式的检测能力；反序列化扫描器缺乏对全部隐写注入类后门逻辑漏洞的检测能力；Hugging Face 内置安全检测工具与 fickling 工具的检测能力几乎相同，且其扫描周期约为 5-7 天，难以及时发现安全问题；而本工具具备对流行与非流行异常后门逻辑漏洞的检测能力。实验结果表明，本工具的检测能力优于现有工具，能够发现更多恶意模型的异常调用模式，检测可能造成代码执行的安全隐患。

表 6-7 本工具 DPDetect 与业界工具的检测能力对比

工具名称	eval	exec	system	runpy	timeit	subprocess
fickling ^[142]	√	√	√	×	×	×
反序列化扫描器 ^[146]	×	×	×	×	×	×
Hugging Face 内置功能	√	√	√	×	×	×
DPDetect	√	√	√	√	√	√

此外，本节针对实网环境下知名开源代码仓库托管的机器学习预训练模型进行检测，发现 Hugging Face 上用户 Star23 从 2023 年 10 月 14 日至 2024 年 2 月 14 日一共上传了 9 份恶意模型，其中 star23/baller13 项目在 2024 年 1 月的下载量达到了 331 次，所有样本中的 C2 平台（Command & Control 平台，用于远程

管理和控制恶意软件或系统的服务器)IP 包括德国 136.243.156.104 端口号 27303 的地址以及斯里兰卡 IP 为 192.248.1.167 端口号为 4242 的地址，所检测得到的恶意样本内容为先判断是 Linux 还是 windows 平台，然后反弹对应 shell 到 C2 平台的接收端口。此外还有用户 ScanMe 在 2024.2.9 日上传了携带安全隐患的 PyTorch、Keras、tf2、NumPy 模型或 pkl 文件；用户 mcpotato 也在 2022 年 9 到 12 月上传了具有反序列化隐患的 pickle 文件、PyTorch 模型文件，如表 6-8 所示：

表 6-8 本工具 DPDetect 检测到的部分实网环境下的模型攻击案例

用户名	时间周期	攻击类型	IP 及端口
Star23	2023/10/14 - 2024/02/14	恶意模型	德国：136.243.156.104:27303
Star23	2024/01/13 - 2024/01/31	恶意模型	斯里兰卡：192.248.1.167:4242
ScanMe	2024/2/9	安全隐患	未反弹 shell
mcpotato	2022/09/01 - 2022/12/31	反序列化漏洞	未反弹 shell

实验结果表明，尽管模型托管平台如 Hugging Face 以及机器学习模型安全检测工具提供了一定的安全检测能力，但无法准确检测部分基于隐写注入的后门逻辑漏洞。此外，本工具检测出实网环境下知名开源代码仓库 Hugging Face 上存在的恶意模型（即植入代码逻辑漏洞的模型），证明此类问题已进入相关研究人员与研究团队的视野，需要对此类问题，包括所有通用部署类应用的逻辑安全问题保持关注。

6.5 本章小结

云平台应用层通用组件便捷了云应用的开发与使用，但也同时引入一系列安全问题：一旦开源代码仓库中的应用层通用组件存在安全隐患，如被攻击者植入后门漏洞，将极大地威胁用户乃至云平台运营者的安全。当前业界对于此类漏洞的检测方式仍然为传统的基于恶意代码字符型特征的检测，这对于部分更为隐蔽植入的漏洞如基于隐写注入的后门漏洞的检测则无能为力。对于云平台大部分应用层通用部署类应用组件，由于其设计目的、功能需求和体系架构的相似性，其部署阶段具备相似的逻辑特征，因此本章提出一种基于运行时逻辑特征的逻辑漏洞检测方法，针对机器学习预训练模型这一最为典型的部署型应用层通用组件，首先提出机器学习模型后门逻辑漏洞的植入方法并评估植入效果；其次提出部署时运行逻辑信息的无侵入式获取与分析方法，提出基于运行时逻辑特征的机器学习后门逻辑漏洞检测方法。

与业界最新研究进行对比，本章首先针对机器学习模型后门逻辑漏洞的植入方式进行分析，以评估植入恶意代码对预训练模型任务效果的影响，证明本文所

研究的反序列化后门逻辑漏洞在实网环境中的危害性与隐蔽性；其次对本章所提出的基于宿主机运行调试器的部署时运行信息追踪方法进行评估，实验结果表明，本章所提出的 Mybdb 工具能够实现平均 95% 以上的追踪性能提升，eMybdb 能够实现 60% 左右、对于部分模型能够达到 80% 以上的追踪信息补充，极大提升追踪信息的获取效率与丰富程度；最后，本章对自身构造的恶意模型以及开源代码仓库托管的机器学习模型进行后门逻辑漏洞检测，并与业界最为先进的工具进行对比，实验结果表明本章所提出的 DPDetect 工具能够实现更好的针对包括但不限于非标准函数调用类、异常调用结构类后门逻辑漏洞的检测，并成功发现实网环境下知名开源代码仓库中十余个被植入后门逻辑漏洞的机器学习预训练模型。同时本工具的检测结果采用归一化的结构进行存储与表示，支持与传统静态分析结果的融合表示，便捷标准化分析。

在未来的研究过程中，可进一步深化对不同类型云平台应用层通用组件运行时特征的描述及刻画，建模不同类型应用层通用组件的标准运行模式，进而支撑更准确、更高效地检测包含隐蔽植入的后门逻辑漏洞在内的部署型应用层通用组件的安全问题。

第七章 总结与展望

7.1 工作总结

随着云计算、大数据、5G 技术的飞速发展,软件形态和软件开发方式发生深刻变革,云平台通用组件作为新时代下软件供应链的关键要素,正逐渐扩大对于当今信息社会的影响,一旦云平台通用组件出现安全问题,将极大地危害个人用户、企业用户、乃至所有受到云平台通用组件所辐射的机构和团体。而逻辑漏洞作为其中最具危害的安全问题之一,其具有难建模、难检测、难分析等特点,持续困扰着相关研究人员。当前针对逻辑漏洞挖掘的技术尚不成体系,在分而治之的各项漏洞挖掘与分析技术中,相关程序分析技术难以直观的辅助漏洞的复现与成因分析,静态漏洞挖掘技术尚未解释逻辑漏洞的成因本质,动态漏洞挖掘技术难以实现对程序逻辑的覆盖测试,这些现状极大地制约了逻辑漏洞挖掘的能力和云平台的安全水平。本文针对云平台通用组件的程序逻辑表征、逻辑漏洞挖掘两个大方面开展研究,所提出的逻辑漏洞挖掘技术覆盖三大类云平台通用组件,具体研究内容如下:

1、逻辑漏洞语言无关型特征提取与表征技术

本部分研究了一种针对逻辑漏洞的语言无关型特征的提取与表征技术,首先从程序纵向分析与横向分析的角度,提出程序层级架构的逻辑信息语言无关化提取技术、程序执行语句的逻辑信息语言无关化提取技术,将程序文件、类、方法、语句间的纵向逻辑关系抽象为语言无关的从属关系与调用关系,并将具体的程序语句进行高层次抽象,消除不同编程语言特性间的差异;随后提出相应的逻辑关系表征技术,将程序逻辑抽象为图形化表示,并提供图形化的程序逻辑查询及分析方法,便捷复杂型逻辑漏洞的复现与建模。本部分最终实现原型工具 Luger,经过实验验证,本方案能够支撑对多种,包括代码执行型、拒绝服务型、条件竞争型逻辑漏洞的复现与建模,相较于现有的漏洞检测工具如 Bandit,能够分析出更多更完整的漏洞利用链、能够降低误报、提供漏洞的更多详细信息,并最终指导挖掘多项 0 day 未公开逻辑漏洞。

2、基于静态程序逻辑理解的代码层逻辑漏洞挖掘

本部分研究了一种针对云平台代码层通用组件的逻辑漏洞挖掘方法,基于逻辑漏洞语言无关型特征的提取与表征技术,对最具代表性的算法逻辑漏洞进行建模及挖掘。首先提出算法逻辑漏洞分析与挖掘的一般 workflow,涵盖逻辑漏洞的建模、挖掘与利用样本构造的全过程;其次提出基于算法结构安全策略违背的新型

算法逻辑漏洞模型,揭示算法逻辑漏洞的语言无关型成因本质;随后提出基于调用链及图形化代码仓库的算法逻辑漏洞分析方法,指导目标漏洞调用结构的快速筛选;最后提出基于递归结构逻辑理解的漏洞利用载荷构造方法,提高漏洞验证与利用的效率。本部分最终实现原型工具 ACVFind,经过实验验证,本方法相较于 oss-fuzz 及其他业界先进工具,能够实现对新型算法逻辑漏洞的检测,能够高效率、低开销的实现目标应用的检测,所检测的漏洞利用链均被验证为漏洞,最终挖掘多个 0 day 未公开算法逻辑漏洞并获得 8 个 CVE 编号。

3、基于动态程序逻辑推演的业务层逻辑漏洞挖掘

本部分研究了一种针对云平台业务层通用组件的逻辑漏洞挖掘方法,提出针对业务层静态逻辑信息与动态运行逻辑的获取与表征方法,并提出针对业务层通用组件的逻辑漏洞挖掘方法。首先提出针对云平台通用组件 REST API 相关逻辑信息的获取与处理方法,获得外部文本视角的静态逻辑信息以及 API 内部视角的动态逻辑信息;其次基于所提出的资源与逻辑漏洞的生命周期模型,通过模型推演以及参数推断实现针对 API 逻辑的测试样本构造,提升对业务逻辑的测试深度与测试广度;最后提出基于 API 调用栈辅助的测试样本执行结果分析技术,基于稀疏调用栈匹配,实现对异常 API 逻辑的检测。本部分最终实现原型工具 RESTlogic,经过实验验证,本方法相较于 RESTler 等其他业界先进工具,能够实现更深入、更全面的业务层逻辑漏洞的动态测试,实现测试深度的提升,最终发现多个开源云平台业务层通用组件的逻辑漏洞,并提交漏洞报告。

4、基于程序运行时逻辑特征的应用层逻辑漏洞挖掘

本部分研究了一种针对云平台应用层通用组件的逻辑漏洞挖掘方法,提出应用层组件运行时逻辑信息的获取与表征方法,并提出针对应用层通用组件的逻辑漏洞挖掘方法。以最为典型的应用层通用组件应用场景,即机器学习模型为例,提出针对此类部署型应用组件的逻辑漏洞检测方法,首先提出机器学习模型后门逻辑漏洞的植入与利用方法,评估不同植入方法的隐蔽性对漏洞检测带来的挑战;其次,提出基于运行时逻辑特征的机器学习模型后门逻辑漏洞检测方法,提出针对此类部署型应用的部署时运行逻辑信息的无侵入式获取方法,并基于所获得函数调用逻辑进行异常调用逻辑的检测。本部分最终实现原型工具 DPDetect,经过实验验证,本方法相较于 fickling 等其他业界先进工具,能够实现对此类应用层后门逻辑漏洞的检测,能够提高调用信息获取效率的同时,补充大量关键逻辑调用信息,能够检测其他工具无法发现的后门逻辑漏洞,并发现实网环境下十余个攻击者向知名开源模型托管平台上传的恶意模型。

综上所述,本文针对云平台通用组件的逻辑漏洞与挖掘分析技术进行深入且广泛的研究,所提出的逻辑漏洞语言无关型特征提取与表征技术能够有效支撑云

平台代码层组件、业务层组件、应用层组件的逻辑漏洞分析与挖掘。对于基于静态程序逻辑理解的代码层逻辑漏洞挖掘、基于动态程序逻辑推演的业务层逻辑漏洞挖掘、基于程序运行时逻辑特征的应用层逻辑漏洞挖掘三项逻辑漏洞挖掘关键技术,本文均遵循逻辑漏洞挖掘的工作流程进行方法设计,包括基础逻辑信息分析方法、逻辑漏洞挖掘模型构建方法、逻辑漏洞的检测分析方法,涉及对静态代码信息、静态文本信息、动态运行时信息的分析,包含静态漏洞挖掘、动态漏洞挖掘、混合漏洞挖掘方法,每项技术方法均有所不同并互为补充,呈现递进关系,共同支撑云平台通用组件逻辑漏洞挖掘的方法体系。

7.2 未来展望

云平台通用组件的逻辑漏洞挖掘与分析技术在近些年受到广泛关注,虽然本文在代表性方向和领域进行了相关研究并取得部分成果,然而随着软件形态的不断发展以及新型通用组件的开发,云平台通用组件面临的逻辑漏洞风险将不断增加,云平台通用组件的逻辑漏洞挖掘与分析技术也将面临不断的挑战。针对本文的阶段性研究,仍有以下几个方面有待进一步深化:

1、云平台通用组件的相关逻辑信息的获取与分析有待进一步深入。本文所提出的逻辑信息分析对象包括静态逻辑信息,即文档信息与代码信息;包括动态逻辑信息,即运行时 API 与函数调用信息。本文所涵盖的逻辑信息可能并不完全,也可能存在冗余,未来需根据相关漏洞挖掘与分析技术的发展不断深入完善优化,并利用智能化方法进一步提取和挖掘关键逻辑信息。

2、云平台通用组件的逻辑漏洞挖掘模型有待持续构造。本文所提出的漏洞挖掘模型聚焦云平台通用组件的三个典型方面,并提出相应的通用逻辑信息获取方法和漏洞挖掘方法。而逻辑漏洞挖掘是经验型的工作,可以充分利用本文所提取与表征的逻辑信息和所提出的逻辑漏洞挖掘方法流程,进一步构建通用及专用逻辑漏洞挖掘模型,提升对于新型逻辑漏洞的挖掘能力,并优化实时性与准确性。

3、云平台通用组件的逻辑漏洞挖掘与分析方法有待推广验证与持续集成。本文所提出的逻辑漏洞挖掘与分析方法聚焦于云平台通用组件,云平台真实生产环境极为复杂,尽管本文所提出的逻辑漏洞挖掘与分析方法已针对实际云平台业务层及应用层典型通用组件进行验证,但为保证方法的通用性及实用性,应持续对方法进行实际生产环境的验证评估,并与自动化安全测试工具链进行集成。

参考文献

- [1] Sunyaev A. Cloud Computing [M]// Sunyaev A. Internet Computing: Principles of Distributed Systems and Emerging Internet-Based Technologies. Switzerland: Springer Cham. 2020: 195-236.
- [2] 中国信息通信研究院. 云计算白皮书[R]. 北京, 2023:1
- [3] 陈芸芸, 邱刚. 美国政府“云优先政策”效果分析[J]. 物联网技术, 2014, 4(05): 9-10.
- [4] 蔡翠红, 张若扬. “技术主权”和“数字主权”话语下的欧盟数字化转型战略[J]. 国际政治研究, 2022, 43(01): 9-36.
- [5] Reuters. EU clears up to 1.2 bln euros of aid for cloud computing[EB/OL]. (2023-12-05)[2024-03=24]. <https://www.reuters.com/technology/eu-clears-up-12-bln-euros-state-aid-cloud-computing-2023-12-05/>.
- [6] Ministry of Defence, UK. (2023-02-02)[2024-03-24]. <https://www.gov.uk/government/publications/cloud-strategic-roadmap-for-defence/cloud-strategic-roadmap-for-defence#en>.
- [7] TIOBE. TIOBE Index for February 2024[EB/OL]. (2024-02-29)[2024-03-04]. <https://www.tiobe.com/tiobe-index/>.
- [8] Hassija V, Chamola V, Gupta V, et al. A Survey on Supply Chain Security: Application Areas, Security Threats, and Solution Architectures[J]. IEEE Internet of Things Journal, 2021, 8(8): 6222-6246.
- [9] Felmetsger V, Cavedon L, Kruegel C, et al. Toward automated detection of logic vulnerabilities in web applications[C]// Proceedings of the 19th USENIX conference on Security, Washington, DC. USENIX Association, 2010: 10.
- [10] Red Hat. CVE-2020-12689[EB/OL]. (2024-02-16)[2024-03-04]. <https://access.redhat.com/security/cve/cve-2020-12689/>.
- [11] National Vulnerability Database. CVE-2021-44228[EB/OL]. (2023-06-11)[2024-03-04]. <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>.
- [12] Wu T, He S, Liu J, et al. A Brief Overview of ChatGPT: The History, Status Quo and Potential Future Development[J]. IEEE/CAA Journal of Automatica Sinica, 2023, 10(5): 1122-1136.
- [13] Wang Z, Liu C, Cui X. EvilModel: Hiding Malware Inside of Neural Network Models[C]// 2021 IEEE Symposium on Computers and Communications (ISCC). IEEE Computer Society, 2021: 1-7.
- [14] Cable J, Gregory D, Izhikevich L, et al. Stratosphere: Finding Vulnerable Cloud Storage

- Buckets[C]// Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses, San Sebastian, Spain. Association for Computing Machinery, 2021: 399–411.
- [15] 王丰辉. 漏洞相关技术研究 [D]. 北京: 北京邮电大学, 2006.
- [16] Krsul I V. Software vulnerability analysis [D]. Ann Arbor, Michigan: Purdue University, 1998.
- [17] Li X, Chang X, Board J A, et al. A novel approach for software vulnerability classification[C]// 2017 Annual Reliability and Maintainability Symposium (RAMS). 2017: 1-7.
- [18] Lee I. Business Logic Flaw[EB/OL]. (2024-02-26)[2024-03-04]. <https://www.wallarm.com/what/business-logic-flaw>.
- [19] OWASP. Business logic vulnerabilities[EB/OL]. (2024-03-04)[2024-03-04]. https://owasp.org/www-community/vulnerabilities/Business_logic_vulnerability.
- [20] 国家市场监督管理总局, 国家标准化管理委员会. 信息安全技术 网络安全漏洞分类分级指南: GB/T 30279-2020[S]. 北京: 中国标准出版社, 2020.
- [21] 孙成昊, 杨一未, 易锦, 等. 漏洞分类分级标准在国家信息安全漏洞库的应用[J]. 信息技术与标准化, 2022, (05): 82-86.
- [22] MITRE Corporation. Common Weakness Enumeration: CWE[EB/OL]. (2024-02-29)[2024-03-04]. <https://cwe.mitre.org/>.
- [23] Siegmund J. Program comprehension: Past, present, and future[C]// 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER). IEEE, 2016: 13-20.
- [24] Maalej W, Tiarks R, Roehm T, et al. On the comprehension of program comprehension[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2014, 23(4): 1-37.
- [25] 金芝, 刘芳, 李戈. 程序理解:现状与未来[J]. 软件学报, 2019, 30(01): 110-126.
- [26] Shahbazi R, Fard F. APIContext2Com: Code Comment Generation by Incorporating Pre-Defined API Documentation[C]// 2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC). IEEE Computer Society, 2023: 13-24.
- [27] Godefroid P, Lehmann D, Polishchuk M. Differential regression testing for REST APIs[C]// Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, USA. Association for Computing Machinery, 2020: 312–323.
- [28] Zheng Q, Xia X, Zou X, et al. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X[C]// Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Long Beach, CA, USA. Association for Computing Machinery, 2023: 5673–5684.

- [29] Chen Y, Xing L, Qin Y, et al. Devils in the guidance: predicting logic vulnerabilities in payment syndication services through automated documentation analysis[C]// Proceedings of the 28th USENIX Conference on Security Symposium, Santa Clara, CA, USA. USENIX Association, 2019: 747–764.
- [30] Gorski P L, Möller S, Wiefling S, et al. “I just looked for the solution!” On Integrating Security-Relevant Information in Non-Security API Documentation to Support Secure Coding Practices[J]. IEEE Transactions on Software Engineering, 2022, 48(9): 3467–3484.
- [31] Haryono S A, Kang H J, Sharma A, et al. Automated Identification of Libraries from Vulnerability Data: Can We Do Better?[C]// 2022 IEEE/ACM 30th International Conference on Program Comprehension (ICPC). 2022: 178–189.
- [32] 张健, 张超, 玄跻峰, 等. 程序分析研究进展[J]. 软件学报, 2019, 30(01): 80–109.
- [33] Geer D. Eclipse becomes the dominant Java IDE[J]. Computer, 2005, 38(7): 16–18.
- [34] Vallée-Rai R, Co P, Gagnon E, et al. Soot - a Java bytecode optimization framework[C]// Proceedings of the 1999 conference of the Centre for Advanced Studies on Collaborative research, Mississauga, Ontario, Canada. IBM Press, 1999: 13.
- [35] Vallée-Rai R, Co P, Gagnon E, et al. Soot: a Java bytecode optimization framework[C]// CASCON First Decade High Impact Papers, Toronto, Ontario, Canada. IBM Corp., 2010: 214–224.
- [36] Tan T, Li Y. Tai-e: A Developer-Friendly Static Analysis Framework for Java by Harnessing the Good Designs of Classics[C]// Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, Seattle, WA, USA. Association for Computing Machinery, 2023: 1093–1105.
- [37] Weiser M. Program slicing[J]. IEEE Transactions on software engineering, 1984, (4): 352–357.
- [38] Tao C, Zhan Q, Hu X, et al. C4: contrastive cross-language code clone detection[C]// Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension, Virtual Event. Association for Computing Machinery, 2022: 413–424.
- [39] Feng Z, Guo D, Tang D, et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages[C]// Findings of the Association for Computational Linguistics: EMNLP 2020, Online. Association for Computational Linguistics, 2020: 1536–1547.
- [40] 任泽众, 郑晗, 张嘉元, 等. 模糊测试技术综述[J]. 计算机研究与发展, 2021, 58(05): 944–963.
- [41] Srivastava A. A practical system for intermodule code optimization at link-time[J]. Journal of programming Languages, 1993, 1(1): 1–18.

- [42] Zhang M, Qiao R, Hasabnis N, et al. A platform for secure static binary instrumentation[C]// Proceedings of the 10th ACM SIGPLAN/SIGOPS international conference on Virtual execution environments, Salt Lake City, Utah, USA. Association for Computing Machinery, 2014: 129–140.
- [43] Luk C-K, Cohn R, Muth R, et al. Pin: building customized program analysis tools with dynamic instrumentation[J]. *Acm sigplan notices*, 2005, 40(6): 190-200.
- [44] Cantrill B, Shapiro M W, Leventhal A H. Dynamic Instrumentation of Production Systems[C]// USENIX Annual Technical Conference, General Track. 2004: 15-28.
- [45] Azmandian F, Moffie M, Alshawabkeh M, et al. Virtual machine monitor-based lightweight intrusion detection[J]. *ACM SIGOPS Operating Systems Review*, 2011, 45(2): 38-53.
- [46] Jones S T, Arpaci-Dusseau A C, Arpaci-Dusseau R H. Antfarm: Tracking Processes in a Virtual Machine Environment[C]// USENIX Annual Technical Conference, General Track. 2006: 1-14.
- [47] He S, He P, Chen Z, et al. A survey on automated log analysis for reliability engineering[J]. *ACM computing surveys (CSUR)*, 2021, 54(6): 1-37.
- [48] Cândido J, Aniche M, van Deursen A. Log-based software monitoring: a systematic mapping study[J]. *PeerJ Computer Science*, 2021, 7: e489.
- [49] Bento A, Correia J, Filipe R, et al. Automated analysis of distributed tracing: Challenges and research directions[J]. *Journal of Grid Computing*, 2021, 19: 1-15.
- [50] Andrianto I, Liem M M I, Asnar Y D W. Web application fuzz testing[C]// 2017 International Conference on Data and Software Engineering (ICoDSE). 2017: 1-6.
- [51] Jabiyeve B, Sprecher S, Onarlioglu K, et al. T-Reqs: HTTP Request Smuggling with Differential Fuzzing[C]// Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea. Association for Computing Machinery, 2021: 1805–1820.
- [52] Livshits V B, Lam M S. Finding Security Vulnerabilities in Java Applications with Static Analysis[C]// USENIX security symposium. 2005: 18-18.
- [53] Zaazaa O, El Bakkali H. Dynamic vulnerability detection approaches and tools: State of the Art[C]// 2020 Fourth International Conference On Intelligent Computing in Data Sciences (ICDS). IEEE, 2020: 1-6.
- [54] Chakraborty S, Krishna R, Ding Y, et al. Deep learning based vulnerability detection: Are we there yet[J]. *IEEE Transactions on Software Engineering*, 2021, 48(9): 3280-3296.
- [55] Cheng X, Wang H, Hua J, et al. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network[J]. *ACM Transactions on Software Engineering and*

- Methodology (TOSEM), 2021, 30(3): 1-33.
- [56] 李韵, 黄辰林, 王中锋, et al. 基于机器学习的软件漏洞挖掘方法综述[J]. 软件学报, 2020, 31(07): 2040-2061.
- [57] Awadhutkar P, Santhanam G R, Holland B, et al. Discover: Detecting algorithmic complexity vulnerabilities[C]// Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2019: 1129-1133.
- [58] Holland B, Santhanam G R, Awadhutkar P, et al. Statically-Informed Dynamic Analysis Tools to Detect Algorithmic Complexity Vulnerabilities[C]// 2016 IEEE 16th International Working Conference on Source Code Analysis and Manipulation (SCAM). 2016: 79-84.
- [59] Blair W, Mambretti A, Arshad S, et al. HotFuzz: Discovering Temporal and Spatial Denial-of-Service Vulnerabilities Through Guided Micro-Fuzzing[J]. ACM Trans Priv Secur, 2022, 25(4): 1-35.
- [60] Wei J, Chen J, Feng Y, et al. Singularity: Pattern fuzzing for worst case complexity[C]// Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2018: 213-223.
- [61] Liu Y, Zhang M, Meng W. Revealor: Detecting and exploiting regular expression denial-of-service vulnerabilities[C]// 2021 IEEE Symposium on Security and Privacy (SP). IEEE, 2021: 1468-1484.
- [62] Noller Y, Kersten R, Păsăreanu C S. Badger: complexity analysis with fuzzing and symbolic execution[C]// Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis. 2018: 322-332.
- [63] Liu Y, Meng W. Acquirer: A Hybrid Approach to Detecting Algorithmic Complexity Vulnerabilities[C]// Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. 2022: 2071-2084.
- [64] Kim M, Xin Q, Sinha S, et al. Automated test generation for rest apis: No time to rest yet[C]// Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. 2022: 289-301.
- [65] Godefroid P, Huang B-Y, Polishchuk M. Intelligent REST API data fuzzing[C]// Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA. Association for Computing Machinery, 2020: 725-736.
- [66] Hatfield-Dodds Z, Dygalo D. Deriving semantics-aware fuzzers from web API schemas[C]// Proceedings of the ACM/IEEE 44th International Conference on Software Engineering:

- Companion Proceedings, Pittsburgh, Pennsylvania. Association for Computing Machinery, 2022: 345–346.
- [67] Martin-Lopez A, Segura S, Ruiz-Cortés A. RESTest: automated black-box testing of RESTful web APIs[C]// Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual, Denmark. Association for Computing Machinery, 2021: 682–685.
- [68] Atlidakis V, Godefroid P, Polishchuk M. Restler: Stateful rest api fuzzing[C]// 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). IEEE, 2019: 748–758.
- [69] Atlidakis V, Godefroid P, Polishchuk M. Checking security properties of cloud service REST APIs[C]// 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). IEEE, 2020: 387–397.
- [70] Deepa G, Thilagam P S, Praseed A, et al. DetLogic: A black-box approach for detecting logic vulnerabilities in web applications[J]. Journal of Network and Computer Applications, 2018, 109: 89–109.
- [71] Liu Y, Li Y, Deng G, et al. Morest: model-based RESTful API testing with execution feedback[C]// Proceedings of the 44th International Conference on Software Engineering. 2022: 1406–1417.
- [72] Arcuri A. Evomaster: Evolutionary multi-context automated system test generation[C]// 2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST). IEEE, 2018: 394–397.
- [73] Arcuri A. RESTful API automated test case generation with EvoMaster[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2019, 28(1): 1–37.
- [74] Arcuri A, Galeotti J P, Marculescu B, et al. EvoMaster: A search-based system test generation tool[J]. Journal of Open Source Software, 2021, (6(57)): 2153.
- [75] Arcuri A. Automated black-and white-box testing of restful apis with evomaster[J]. IEEE Software, 2020, 38(3): 72–78.
- [76] Zhang M, Arcuri A. Adaptive Hypermutation for Search-Based System Test Generation: A Study on REST APIs with EvoMaster[J]. ACM Trans Softw Eng Methodol, 2021, 31(1): 1–52.
- [77] Binxiang L, Gang Z, Ruoying S. A Deep Reinforcement Learning Malware Detection Method Based on PE Feature Distribution[C]// 2019 6th International Conference on Information Science and Control Engineering (ICISCE). 2019: 23–27.
- [78] Kolter J Z, Maloof M A. Learning to Detect and Classify Malicious Executables in the Wild[J].

- J Mach Learn Res, 2006, 7: 2721–2744.
- [79] Nataraj L, Karthikeyan S, Jacob G, et al. Malware images: visualization and automatic classification[C]// Proceedings of the 8th International Symposium on Visualization for Cyber Security, Pittsburgh, Pennsylvania, USA. Association for Computing Machinery, 2011: 1-7.
- [80] 刘建松, 张磊, 方勇. 基于行为关系网络的恶意代码检测方法[J]. 四川大学学报(自然科学版), 2022, 59(02): 77-83.
- [81] Wang Y, Wang Q, Qin X, et al. DockerWatch: a two-phase hybrid detection of malware using various static features in container cloud[J]. Soft Computing, 2023, 27(2): 1015-1031.
- [82] Liu X, Zhang Y, Yu Q, et al. SmartEagleEye: A Cloud-Oriented Webshell Detection System Based on Dynamic Gray-Box and Deep Learning[J]. Tsinghua Science and Technology, 2024, 29(3): 766-783.
- [83] Hua J, Wang K, Wang M, et al. MalModel: Hiding Malicious Payload in Mobile Deep Learning Models with Black-box Backdoor Attack[J]. arXiv preprint arXiv:240102659, 2024.
- [84] MITRE Corporation. cve[EB/OL]. (2024-03-09)[2024-03-09]. <https://cve.mitre.org/>.
- [85] OWASP. OWASP Top Ten[EB/OL]. (2021-09-30)[2024-03-09]. <https://owasp.org/www-project-top-ten/>.
- [86] Zhang J, Wang X, Zhang H, et al. A Novel Neural Source Code Representation Based on Abstract Syntax Tree[C]// 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). 2019: 783-794.
- [87] Koschke R, Falke R, Frenzel P. Clone Detection Using Abstract Syntax Suffix Trees[C]// 2006 13th Working Conference on Reverse Engineering. 2006: 253-262.
- [88] Yamaguchi F, Golde N, Arp D, et al. Modeling and Discovering Vulnerabilities with Code Property Graphs[C]// 2014 IEEE Symposium on Security and Privacy. 2014: 590-604.
- [89] Li Q, Song J, Tan D, et al. PDGraph: A Large-Scale Empirical Study on Project Dependency of Security Vulnerabilities[C]// 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). 2021: 161-173.
- [90] GitHub. CodeQL[EB/OL]. (2024-03-09)[2024-03-09]. <https://github.com/github/codeql>.
- [91] Gobbi M F, Kinder J. Poster: Using CodeQL to Detect Malware in npm[C]// Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, Copenhagen, Denmark. Association for Computing Machinery, 2023: 3519–3521.
- [92] Youn D, Lee S, Ryu S. Declarative static analysis for multilingual programs using CodeQL[J]. Software: Practice and Experience, 2023, 53(7): 1472-1495.
- [93] GEEKSFORGEEKS. Introduction to Intermediate Representation(IR)[EB/OL]. (2022-06-07)[2024-03-09]. <https://www.geeksforgeeks.org/introduction-to-intermediate->

[representationir/](#).

- [94] 林泽琦, 赵俊峰, 谢冰. 一种基于图数据库的代码结构解析与搜索方法[J]. 计算机研究与发展, 2016, 53(3): 531-540.
- [95] 王飞, 刘井平, 刘斌, 等. 代码知识图谱构建及智能化软件开发方法研究[J]. 软件学报, 2020, 31(1): 47-66.
- [96] Python. ast — Abstract Syntax Trees[EB/OL]. (2024-03-08)[2024-03-09]. <https://docs.python.org/3/library/ast.html>.
- [97] PyCQA. bandit[EB/OL]. (2024-03-08)[2024-03-09]. <https://github.com/PyCQA/bandit>.
- [98] Semgrep. semgrep[EB/OL]. (2024-03-31)[2024-04-01]. <https://github.com/semgrep/semgrep>.
- [99] Snky. snyk/cli[EB/OL]. (2024-03-28)[2024-04-01]. <https://github.com/snyk/cli>.
- [100] Devopedia. Algorithmic Complexity[EB/OL]. (2022-02-20)[2024-03-10]. <https://devopedia.org/algorithmic-complexity>.
- [101] S.Adamchik V. Algorithmic Complexity[EB/OL]. (2009)[2024-03-10]. <https://viterbi-web.usc.edu/~adamchik/15-121/lectures/Algorithmic%20Complexity/complexity.html>.
- [102] Wikipedia. Algorithmic complexity attack[EB/OL]. (2023-10-19)[2024-03-10]. https://en.wikipedia.org/wiki/Algorithmic_complexity_attack.
- [103] Crosby S A, Wallach D S. Denial of service via algorithmic complexity attacks[C]// Proceedings of the 12th conference on USENIX Security Symposium - Volume 12, Washington, DC. USENIX Association, 2003: 3.
- [104] Czubak A, Szymanek M. Algorithmic Complexity Vulnerability Analysis of a Stateful Firewall[C]// Information Systems Architecture and Technology: Proceedings of 37th International Conference on Information Systems Architecture and Technology – ISAT 2016 – Part II, Cham. Springer International Publishing, 2017: 77-97.
- [105] Weidman A. Regular expression Denial of Service - ReDoS[EB/OL]. (2023-07-19)[2024-03-10]. https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS.
- [106] Kirrage J, Rathnayake A, Thielecke H. Static analysis for regular expression denial-of-service attacks[C]// International Conference on Network and System Security. Springer, 2013: 135-148.
- [107] Wüstholtz V, Olivo O, Heule M J, et al. Static detection of DoS vulnerabilities in programs that use regular expressions[C]// Tools and Algorithms for the Construction and Analysis of Systems: 23rd International Conference, TACAS 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-

- 29, 2017, Proceedings, Part II 23. Springer, 2017: 3-20.
- [108] Davis J C, Coghlan C A, Servant F, et al. The impact of regular expression denial of service (ReDoS) in practice: an empirical study at the ecosystem scale[C]// Proceedings of the 2018 26th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering. 2018: 246-256.
- [109] Wang X, Zhang C, Li Y, et al. Effective ReDoS Detection by Principled Vulnerability Modeling and Exploit Generation[C]// 2023 IEEE Symposium on Security and Privacy (SP). IEEE, 2023: 2427-2443.
- [110] Staicu C-A, Pradel M. Freezing the Web: A Study of ReDoS Vulnerabilities in JavaScript-based Web Servers[C]// 27th USENIX Security Symposium (USENIX Security 18). 2018: 361-376.
- [111] Maury J. How to Use Input Sanitization to Prevent Web Attacks[EB/OL]. (2023-02-28)[2024-03-10]. <https://www.esecurityplanet.com/endpoint/prevent-web-attacks-using-input-sanitization/>.
- [112] Barlas E, Du X, Davis J C. Exploiting input sanitization for regex denial of service[C]// Proceedings of the 44th International Conference on Software Engineering, Pittsburgh, Pennsylvania. Association for Computing Machinery, 2022: 883–895.
- [113] GeeksforGeeks. Recursion in Java[EB/OL]. (2023-05-20)[2024-03-10]. <https://www.geeksforgeeks.org/recursion-in-java/>.
- [114] Awadhutkar P, Santhanam G R, Holland B, et al. Intelligence amplifying loop characterizations for detecting algorithmic complexity vulnerabilities[C]// 2017 24th Asia-Pacific Software Engineering Conference (APSEC). IEEE, 2017: 249-258.
- [115] Burnim J, Juvekar S, Sen K. WISE: Automated test generation for worst-case complexity[C]// 2009 IEEE 31st International Conference on Software Engineering. IEEE, 2009: 463-473.
- [116] Luckow K, Kersten R, Păsăreanu C. Symbolic complexity analysis using context-preserving histories[C]// 2017 IEEE International Conference on Software Testing, Verification and Validation (ICST). IEEE, 2017: 58-68.
- [117] Luckow K, Dimjašević M, Giannakopoulou D, et al. JD art: a dynamic symbolic analysis framework[C]// Tools and Algorithms for the Construction and Analysis of Systems: 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings 22. Springer, 2016: 442-459.
- [118] Petsios T, Zhao J, Keromytis A D, et al. Slowfuzz: Automated domain-independent detection of algorithmic complexity vulnerabilities[C]// Proceedings of the 2017 ACM SIGSAC

- conference on computer and communications security. 2017: 2155-2168.
- [119] Dietrich J, Jezek K, Rasheed S, et al. Evil pickles: DoS attacks based on object-graph engineering[C]// 31st European Conference on Object-Oriented Programming (ECOOP 2017). Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2017:
- [120] Fielding R T. Architectural styles and the design of network-based software architectures [M]. Irvine, California: University of California, 2000.
- [121] Lyu C, Xu J, Ji S, et al. MINER: a hybrid data-driven approach for REST API fuzzing[C]// Proceedings of the 32nd USENIX Conference on Security Symposium, Anaheim, CA, USA. USENIX Association, 2023: 4517-4534.
- [122] Deng G, Zhang Z, Li Y, et al. NAUTILUS: automated RESTful API vulnerability detection[C]// Proceedings of the 32nd USENIX Conference on Security Symposium, Anaheim, CA, USA. USENIX Association, 2023: 5593-5609.
- [123] Microsoft. Best practices in cloud applications[EB/OL]. (2022-12-17)[2024-03-10]. <https://learn.microsoft.com/en-us/azure/architecture/best-practices/index-best-practices>.
- [124] OpenAPI Initiative. OpenAPI Specification v3.1.0[EB/OL]. (2021-02-15)[2024-03-10]. <https://spec.openapis.org/oas/v3.1.0>.
- [125] Fielding R, Reschke J. RFC 7231: Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content [M]. USA: RFC Editor, 2014.
- [126] Dusseault L, Snell J. RFC 5789: PATCH Method for HTTP [M]. USA: RFC Editor, 2010.
- [127] Fielding R, Nottingham M, Reschke J. RFC 9110: HTTP Semantics [M]. USA: RFC Editor, 2022.
- [128] OpenTracing. OpenTracing API[EB/OL]. (2024-01-27)[2024-03-10]. <https://github.com/opentracing>.
- [129] Jaeger. Jaeger: open source, distributed tracing platform[EB/OL]. (2024-03-10)[2024-03-10]. <https://www.jaegertracing.io/>.
- [130] Rosado T, Bernardino J. An overview of openstack architecture[C]// Proceedings of the 18th International Database Engineering & Applications Symposium. 2014: 366-367.
- [131] KissPeter. APIFuzzer[EB/OL]. (2023-09-18)[2024-04-01]. <https://github.com/KissPeter/APIFuzzer>.
- [132] Cornutum. tcases[EB/OL]. (2024-03-06)[2024-04-01]. <https://github.com/Cornutum/tcases>.
- [133] Corradini D, Zampieri A, Pasqua M, et al. Automated black-box testing of nominal and error scenarios in RESTful APIs[J]. Software Testing, Verification and Reliability, 2022, 32(5): 1808-1840.
- [134] Viglianisi E, Dallago M, Ceccato M. Resttestgen: automated black-box testing of restful

- apis[C]// 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). IEEE, 2020: 142-152.
- [135] Barreno M, Nelson B, Joseph A D, et al. The security of machine learning[J]. Machine Learning, 2010, 81(2): 121-148.
- [136] Vorobeychik Y, Kantarcioglu M, Brachman R, et al. Adversarial machine learning [M]. Switzerland: Springer Cham, 2018.
- [137] Rosenberg I, Shabtai A, Elovici Y, et al. Adversarial Machine Learning Attacks and Defense Methods in the Cyber Security Domain[J]. ACM Comput Surv, 2021, 54(5): 1-36.
- [138] Oprea A, Vassilev A. Adversarial machine learning: A taxonomy and terminology of attacks and mitigations[R]. Gaithersburg, MD: National Institute of Standards and Technology, 2023:
- [139] Chen X, Liu C, Li B, et al. Targeted backdoor attacks on deep learning systems using data poisoning[J]. arXiv preprint arXiv:171205526, 2017.
- [140] Eoin W, Marta J, Tom B. Weaponizing ML Models with Ransomware[EB/OL]. (2022-12-06)[2024-03-27]. <https://hiddenlayer.com/research/weaponizing-machine-learning-models-with-ransomware/>.
- [141] Wang Z, Liu C, Cui X, et al. EvilModel 2.0: Bringing Neural Network Models into Malware Attacks[J]. Computers & Security, 2022, 120: 102807.
- [142] trailofbits. fickling[EB/OL]. (2024-03-08)[2024-03-10]. <https://github.com/trailofbits/fickling>.
- [143] Python. pickle — Python object serialization[EB/OL]. (2024-03-23)[2024-03-24]. <https://docs.python.org/3/library/pickle.html>.
- [144] Kahan W. IEEE standard 754 for binary floating-point arithmetic[J]. Lecture Notes on the Status of IEEE, 1996, 754(94720-1776): 11.
- [145] Python. bdb — Debugger framework[EB/OL]. (2024-03-23)[2024-03-24]. <https://docs.python.org/3/library/bdb.html>.
- [146] Verma A. Insecure Deserialization Detection in Python [D]. San Jose, CA: San Jose State University, 2023.